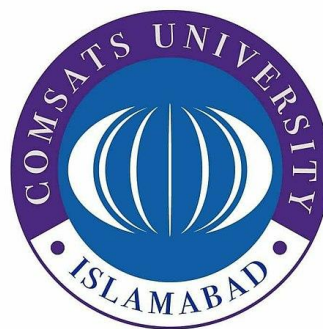


Automated Web Application Penetration Testing Using Machine Learning Models

Final Year Project

Session 2022-2026

A project submitted in partial fulfillment of the
requirements for the Degree
of
BS in Software Engineering



Department of Computer Science
COMSATS University Islamabad (CUI), Lahore Campus

Project Details

Project ID (for office use)				
Type of project	☐ Traditional ☐ Industrial ☒ Continuing			
Nature of project	☐ Development ☒ Research & Development			
Sustainable Development Goals(SDGs)	☐ Good Health and Well-Being ☐ Quality Education ☒ Industry, Innovation, and Infrastructure ☐ Gender Equality ☐ Decent Work and Economic Growth ☐ Climate Action			
Area of specialization	☐ Artificial Intelligence (AI) ☐ Blockchain ☒ Cybersecurity ☐ Data Science and Analytics ☐ Game Development ☐ Internet of Things (IoT) ☐ Natural Language Processing (NLP) ☐ Mobile App Development ☐ Web Development			
Project Group Members				
Sr.#	Reg. #	Student Name	Email ID	Signature
(i)	SP22-BSE-088	Hannan Ali	Sp22-bse-088@cuilahore.edu.pk	
(ii)	SP22-BSE-094	Ahmad Naeem	Sp22-bse-094@cuilahore.edu.pk	
(iii)	SP22-BSE-118	Mujtaba Kamil Mehmood	Sp22-bse-118@cuilahore.edu.pk	
Declaration: The candidates confirm that the work submitted is their own and appropriate credit has been given where reference has been made to the work of others.				

Plagiarism Free Certificate

This is to certify that, I am Hannan Ali S/D/o Muhammad Idrees, group leader of FYP under registration no CUI/SP22-BSE-088/LHR at the Computer Science Department, COMSATS University Islamabad, Lahore Campus. I declare that my FYP proposal is checked by my supervisor and the similarity index is _____% that is less than 20%, an acceptable limit by HEC. The report is attached herewith as Appendix A.

Date: _____ Name of Group Leader: Hannan Ali Signature: _____

Name of Supervisor: DR. Shahbaz Akhter Co-Supervisor (if any): _____

Designation: _____ Designation: _____

Signature: _____ Signature: _____

HoD: _____

Signature: _____

ABSTRACT

This project aims to develop an automated web application vulnerability scanner based on machine learning to enhance security scanning. The aim is to assist developers and security testers in identifying common vulnerabilities such as SQL Injection and XSS more efficiently. The system combines standard scanning techniques with a trained machine learning system which detects known attack patterns. Developed in Python, the scanner navigates web applications, inspects their structure, and applies the ML model to identify potential security flaws. Machine learning assists in minimizing false positives by detecting actual vulnerabilities more precisely and adapting how it scans according to application feedback. The software has a friendly interface to present results and suggest remedies. It was experimented with on sensitive applications and was found to be more effective than human testing. As a whole, the project demonstrates how machine learning can be applied to cybersecurity and sets the stage for more intelligent, automated security tools.

Table of Contents

Chapter 1. Introduction	12
1.1 Overview	12
1.2 Problem Statement	12
1.3 Proposed Solution	13
1.4 Main Objectives	13
1.5 Assumptions & Constraints	13
1.5.1 Assumptions	13
1.5.2 Constraints	14
1.6 Project scope	14
1.6.1 Project Goals	14
1.6.2 Deliverables	14
1.6.3 Features and Functions	14
1.7 Software Development Lifecycle Model	15
1.7.1 Why Used Agile?	16
1.7.2 Advantages to This Project	16
Chapter 2. Requirement Analysis	17
2.1 Literature Review	17
2.1.1 Automated Penetration Testing Tools:	17
2.1.2 Machine Learning in Cybersecurity:	17
2.1.3 Challenges in Vulnerability Detection:	17
2.1.4 Literature Review Summary	18
2.1.5 OWASP ZAP	18
2.1.6 Burp Suite	18
2.1.7 Acunetix	18
2.1.8 Nessus	19
2.1.9 Nikto	19
2.2 Research Gap Identification	19
2.3 Requirements Elicitation	19
2.3.1 Functional Requirements	19
2.3.2 Non-functional Requirements	25
2.3.3 Requirements Traceability Matrix	28
2.4 Use Case Description	29
2.4.1 User Registration	29
2.4.2 Login	30
2.4.3 Logout	31
2.4.4 Password Recovery	32
2.4.5 Profile Management	33
2.4.6 View Projects	34
2.4.7 Add Project	35
2.4.8 Open Project	36

2.4.9 Target Configuration	37
2.4.10 Passive Information Gathering	38
2.4.11 Scan Execution	39
2.4.12 Vulnerability Analysis and Prediction.....	41
2.4.13 Data Collection and Model Training.....	42
2.4.14 Remediation Suggestion Based on ML Prediction.....	43
2.4.15 Report Generation	44
2.4.16 Report Management	45
Chapter 3. System Design/Methodology	47
3.1 Activity Diagram	47
3.1.1 User Registration	47
3.1.2 Login	48
3.1.3 Logout	49
3.1.4 Password Recovery	50
3.1.5 Profile Management	51
3.1.6 View Projects.....	52
3.1.7 Add project	53
3.1.8 Open Project	54
3.1.9 Target Configuration	55
3.1.10 Passive Information Gathering	56
3.1.11 Scan Execution	57
3.1.12 Vulnerability Analysis and Prediction.....	58
3.1.13 Data Collection and Model Training.....	59
3.1.14 Remediation Suggestion Based on ML Prediction.....	60
3.1.15 Report Generation	61
3.1.16 Report Management	62
3.2 Software Architecture Diagram	63
3.3 Database Diagram.....	64
3.4 Dataset Details	65
3.4.1 Dataset Source	65
3.4.2 Dataset Creation Process	65
3.4.3 Dataset Size and Structure.....	65
3.4.4 Dataset Cleaning Summary	66
3.5 Algorithm/Model Selection	66
3.5.1 Problem Framing.....	66
3.5.2 Model Selection Rationale	66
3.5.3 Feature-Based Learning Approach.....	66
3.5.4 Threshold-Based Decision Strategy	66
Chapter 4. Implementation	66
4.1 Work Breakdown Structure (WBS).....	66
4.2 Team Roles and Responsibilities	68
4.3 Tools and Technologies.....	69

4.3.1	Core Technologies.....	69
4.3.2	Database & Storage	69
4.3.3	Security & Scanning Tools.....	70
4.3.4	DevOps & Deployment	70
4.3.5	Data Analysis & Utilities.....	70
4.4	Implementation Details.....	70
4.4.1	System Architecture	70
4.4.2	Scanning Logic Implementation.....	70
4.4.3	Data Flow & Storage	71
4.4.4	Pseudocode	71
4.5	Screenshots of Prototype / System.....	75
4.5.1	Home Page.....	75
4.5.2	Landing Page	75
4.5.3	Authentication & Authorization	76
4.5.4	Dashboard.....	76
4.5.5	Password Recovery	77
4.5.6	Project Management Interface.....	77
4.5.7	Security Scan Management Interface	78
4.5.8	User Profile Management.....	78
4.5.9	Project Creation and Target URL Configuration Interface	79
4.5.10	Project Details and Scan Overview Interface	79
4.5.11	Security Scan Configuration Interface	80
4.5.12	Website Vulnerability Assessment and Security Analysis Results	82
4.6	Challenges During Implementation	84
4.6.1	Integration of Multiple Security Tools	84
4.6.2	High Computational and Resource Requirements.....	84
4.6.3	Data Quality and Inconsistent Scanner Output.....	84
4.6.4	Lack of Ground-Truth Labels for Machine Learning.....	84
4.6.5	Feature Selection and Information Leakage Risk.....	84
4.6.6	Model Evaluation and Interpretation.....	84
Chapter 5.	Results & Analysis.....	85
5.1	Experimental Setup.....	85
5.1.1	Test Cases.....	85
5.1.2	System Under Test	100
5.1.3	Learning Paradigm	100
5.1.4	Evaluation Metrics.....	100
5.2	Results.....	101
5.2.1	Alert Reduction Performance	101
5.2.2	Precision and F1-Score (Internal Validation).....	102
5.2.3	Overfitting Analysis	102
5.2.4	FP Confidence Distribution	102
5.2.5	Risk Preservation.....	103

5.3	Comparitive Analysis with Existing Work	104
5.4	Discussion on Findings	104
Chapter 6.	Conclusions	105
6.1	Summary	105
6.2	Recommendations for Future Work.....	105
Chapter 7.	References	106

List of Tables

Table 1.1 Agile Lifecycle	15
Table 2.1 Literature Review Summary	18
Table 2.2 FR-001 Signup.....	19
Table 2.3 FR-002 Login	20
Table 2.4 FR-003 User Log out	20
Table 2.5 FR-004 Password Recovery	20
Table 2.6 FR-005 User Profile Management.....	20
Table 2.7 FR-006 View Projects.....	21
Table 2.8 FR-007 Add Project.....	21
Table 2.9 FR-008 Open Project	21
Table 2.10 FR-009 Target Configuration	22
Table 2.11 FR-010 Basic Passive Information Gathering	22
Table 2.12 FR-011 Scan Execution	23
Table 2.13 FR-012 Vulnerability Analysis and Prediction.....	23
Table 2.14 FR-013 Data Collection and Model Training	23
Table 2.15 FR-014 Vulnerability-Based Remediation Suggestions	24
Table 2.16 FR-015 Report Generation	24
Table 2.17 FR-016 Report Management	24
Table 2.18 NFR-001 Security.....	25
Table 2.19 NFR-002 Simple and Minimalistic Design	25
Table 2.20 NFR-003 Reliability	25
Table 2.21 NFR-004 Performance.....	26
Table 2.22 NFR-005 Usability.....	26
Table 2.23 NFR-006 Data integrity	26
Table 2.24 NFR-007 Availability	27
Table 2.25 NFR-008 Interoperability	27
Table 2.26 NFR-009 Efficient Session Management	27
Table 2.27 Requirements Traceability Matrix	28
Table 2.28 Use Case Description-001 Signup	29
Table 2.29 Use Case Description-002 Login.....	30
Table 2.30 Use Case Description-003 Logout.....	31
Table 2.31 Use Case Description-004 Password Recovery.....	32
Table 2.32 Use Case Description-005 Profile Management.....	33
Table 2.33 Use Case Description-006 View Projects	34
Table 2.34 Use Case Description-007 Add Project	35
Table 2.35 Use Case Description-008 Open Project	36

Table 2.36 Use Case Description-009 Target Configuration.....	37
Table 2.37 Use Case Description-010 Passive Information Gathering	38
Table 2.38 Use Case Description-011 Scan Execution	39
Table 2.39 Use Case Description-012 Vulnerability Analysis and Prediction.	41
Table 2.40 Use Case Description-013 Data Collection and Model Training	42
Table 2.41 Use Case Description-014 Remediation Suggestion Based on ML Prediction	43
Table 2.42 Use Case Description-015 Tournament Analysis Reporting.....	44
Table 2.43 Use Case Description-016 Report Management.....	45
Table 4.1. Individual Tasks	68
Table 5.1 Test Case 001-User Registration	85
Table 5.2 Test Case 002-Login.....	86
Table 5.3 Test Case 003-Logout.....	86
Table 5.4 Test Case 004-Password Recovery.....	87
Table 5.5 Test Case 005-Profile Managemnet.....	88
Table 5.6 Test Case 006-View Projects.....	89
Table 5.7 Test Case 007-Add Projects.....	90
Table 5.8 Test Case 008-Open Project	90
Table 5.9 Test Case 009-Target Configuration	91
Table 5.10 Test Case 010-Passive Information Gathering	92
Table 5.11 Test Case 011-Sacn Execution	93
Table 5.12 Test Case 012 Vulnerability Analysis and Prediction	94
Table 5.13 Test Case 013-Data Collection and Model Training	96
Table 5.14 Test Case 014-Remediation Suggestion Based on ML Prediction	97
Table 5.15 Test Case 015- Report Generation.....	98
Table 5.16 Test Case 016-Report Management.....	99
Table 5.17 Results of Scan	101

List of Figures

Figure 1.1 Agile-Model	16
Figure 3.1 Activity Diagram-001 User Registration.....	47
Figure 3.2 Activity Diagram-002 Login.....	48
Figure 3.3 Activity Diagram-003 Logout.....	49
Figure 3.4 Activity Diagram-004 Password recovery	50
Figure 3.5 Activity Diagram-005 Profile Management.....	51
Figure 3.6 Activity Diagram-006 View Projects	52
Figure 3.7 Activity Diagram-007 Add project.....	53
Figure 3.8 Activity Diagram-008 Open Project.....	54
Figure 3.9 Activity Diagram-009 Target Configuration.....	55
Figure 3.10 Activity Diagram-010 Passive Information Gathering.....	56
Figure 3.11 Activity Diagram-011 Passive Information Gathering.....	57
Figure 3.12 Activity Diagram-012 Vulnerability analysis and prediction.....	58
Figure 3.13 Activity Diagram-013 Data Collection and Model Training.....	59
Figure 3.14 Activity Diagram-014 Remediation Suggestion Based on ML Prediction	60
Figure 3.15 Activity Diagram-015 Report Generation	61
Figure 3.16 Activity Diagram-016 Report Generation	62
Figure 3.17 Software Architecture Diagram.....	63
Figure 3.18 Database Diagram	64
Figure 4.1 Work Breakdown Structure.....	67
Figure 4.2 Gantt Chart	69
Figure 4.3 Home Page	75
Figure 4.4 Landing Page.....	75
Figure 4.5 Authentication and Authorization	76
Figure 4.6 Dashboard.....	76
Figure 4.7 Password Recovery	77
Figure 4.8 Project Management Interface.....	77
Figure 4.9 Project Health Overview Dashboard.....	78
Figure 4.10 Security Scan Status and Results Listing	78
Figure 4.11 User Profile Management.....	79
Figure 4.12 Project Creation and Target URL Configuration Interface	79
Figure 4.13 Project Details and Scan Overview Interface.....	80
Figure 4.14 Passive Scan Configuration.....	80
Figure 4.15 Active Scan Configuration	81
Figure 4.16 Comprehensive Scan Configuration.....	81
Figure 4.17 Security Scan Results Dashboard.....	82

Figure 4.18 Security Recommendations 82

Figure 4.19 Scan Analytics and Statistics..... 83

Figure 4.20 Passive Reconnaissance Summary 83

Figure 5.1 Test results of previous unseen real-world scan..... 101

Figure 5.2 Test results random forestt model 102

Figure 5.3 FP Confidence Distribution..... 103

Figure 5.4 Severity Predervation Analysis 103

Chapter 1. Introduction

In this chapter, we discuss the growing importance of web application security in today's digital era and introduce our proposed automated vulnerability scanner. We describe the integration of machine learning techniques to improve detection accuracy, decrease false positives, and offer scanning capabilities for detecting common web vulnerabilities like security misconfigurations, SQL Injection, and Cross-Site Scripting (XSS).

1.1 Overview

The web applications are perhaps the most integral aspects of the current digital environment. Easy operation of governments, businesses, and daily routines cannot be thought of without web applications. Talking from the smallest matter of shopping to the complex issues related to health issues, such technologies are facilitating communication and the transfer as well as conducting transactions, making them necessities. As organizations continue to shift their bases online to reach users and offer services, the requirement for ultimate security measures becomes increasingly vital.

Hackers have been able to take advantage of platform vulnerabilities since the beginning of this era. The practices involved stealing data and disrupting services and gaining unauthorized access to systems. Security failures will produce severe consequences which include monetary losses together with negative impacts on organizational reputation and potential legal consequences. The public becomes aware of security protocol requirements for organizations when major data breaches and web service attacks receive media attention. Web application security faces a critical problem because third party components become vulnerable when developers misuse them particularly when these components emerge from AI and automated system development. The development process becomes faster while costs decrease but these methods contain concealed threats. Developers sometimes add outdated libraries and poorly constructed APIs and weak programming code to their projects without realizing it. The use of Artificial Intelligence (AI) generated code for security testing becomes problematic when developers depend too heavily on it because AI code lacks the thorough analysis needed to detect vulnerabilities. The security issue will become difficult to identify and resolve because abandoned vulnerabilities remain unaddressed. The tool requires development to solve existing problems while creating new methods which will enhance web application security assessment capabilities. The project developed an automated web application penetration testing tool which incorporates machine learning to enhance both speed and precision of vulnerability identification. The tool will enhance its ability to detect SQL Injection and Cross-Site Scripting (XSS) vulnerabilities through its implementation of sophisticated machine learning methods during the scanning procedure. The new testing approach will simplify security assessment procedures while enabling developers and security researchers to obtain essential data which will enhance web application security frameworks.

1.2 Problem Statement

The web application has become a crucial part of our web world, permitting everything from electronic commerce to government services. However, the increasing reliance has brought about serious security risks. As more organizations migrate their services online, their web applications have become increasingly appealing to cyber attackers. Broad-ranging attacks such as SQL Injection, Cross-site scripting, and Access Control Vulnerabilities continue to pose risks, even though they have been well-explicated. Most current vulnerability tools are static and lack the ability to keep up with the ever-evolving sophisticated attacks. They will miss the sophisticated attacks or show too much noise in the form of false positives, which would require expert knowledge to filter out manually. In addition, most current vulnerability tools are quite complex for the uninitiated to work with.

Moreover, the code being developed often involves third party code or code developed using the help of artificial intelligence, which might have certain security flaws in them. Such flaws are even harder to detect using the scanners than the ones discovered using traditional methods. There is

definitely a requirement for smarter scanners using machine learning algorithms, which can decrease the rate of false positives along with the capability to address the newest categories of threats. An intelligent scanning approach can help in securing the complex web applications.

1.3 Proposed Solution

The proposed solution would come in as an automated web application vulnerability scanner that utilizes machine learning to enable accurate and efficient scanning. This system would incorporate conventional scanning methodologies along with the abilities provided by a trained machine learning engine focused on detecting prevalent attack patterns, which may include SQL Injection, XSS, security misconfiguration, as well as outdated libraries, through a Python-based scanning engine that leverages the strengths offered by open source tools. It should also enable real-time reporting through an interactive interface, which would advise technical as well as non-technical individuals on existing vulnerabilities, along with appropriate remediation strategies. Through an amalgamation involving datasets related to existing vulnerabilities, synthetic attack variant development, as well as iterative learning, it ensures that it remains scalable and an continuously improving system. Through its implementation, it gives a sustainable, intelligent, as well as viable mechanism for web application security by combining traditional penetration analysis and predictive attack engines powered by machine learning.

1.4 Main Objectives

The main objective that the project aims to achieve is to design and build an advanced machine learning based web application vulnerability scanner that goes beyond traditional approaches based on rules. Below are the objectives:

- To design a system for autonomous web application vulnerability detection using machine learning for high risk attacks like SQL Injection, Cross-Site Scripting (XSS), and other prevalent web security vulnerabilities.
- To minimize manual efforts by automating vulnerabilities detection and report generation.
- To enhance detection accuracy and speed by training machine learning models on data collected from multiple sources.
- To offer more frequent and non disruptive scans.
- To provide actionable remediation recommendations which depend on both the severity level and the extent of system impact.
- To create an intuitive user interface accessible to technical and non-technical users.

1.5 Assumptions & Constraints

In the process of creating our web application vulnerability scanner powered by machine learning, some assumptions and constraints have been realized that will inform our implementation strategy and set practical boundaries for our project. These factors are vital to project feasibility and successful project delivery within the given timeframe.

1.5.1 Assumptions

- Assume all team members possess sufficient knowledge in machine learning, web development, and cybersecurity to operate efficiently.
- Assume necessary hardware and test environments will be available throughout the project.
- Assume appropriate training data on web vulnerabilities will be available for model training and testing.
- Assume web frameworks have structures that our scanner can scan correctly.
- Assume that the tool is to be utilized only for ethical and permitted security testing, not malicious activities.

- Assume that users will have some knowledge of web security to be able to carry out scan results and implement recommendations.

1.5.2 Constraints

- The project has to be finished within the academic duration (Dec 2024 - Dec 2025), restricting the size and complexity of features.
- Limited hardware could influence the extent and intensity of machine learning model training and testing, including computational cloud resources.
- Inability to have access to varied real-world environments can necessitate the utilization of synthetic or controlled test data.
- The scope of interest is limited to OWASP Top 10 [6] vulnerabilities, particularly Injection, Broken Access Control, Security Misconfiguration, and Outdated Components.
- The solution needs to meet legal and ethical requirements, limiting some of the scanning methods that might interfere with live systems.
- With just three team members, the scope of the project needs to be handled with caution in order to keep deadlines and deliverables on track.
- CI/CD integration can be restricted to generic or certain platforms because of resource and time limitations.

1.6 Project scope

The scope of this project includes:

1.6.1 Project Goals

- Create a machine learning-based automated vulnerability scanner to improve web application security evaluations.
- Improve detection accuracy for vulnerabilities such as SQL Injection, Cross-Site Scripting (XSS), and other prevalent web security vulnerabilities.
- Minimize false positives through machine learning-based vulnerability pattern recognition.
- Create a user-friendly interface that simplifies vulnerability reporting and offers actionable insights for resolving security flaws.

1.6.2 Deliverables

- A useful vulnerability scanner with machine learning enhancements that may identify common online vulnerabilities on its own.
- Adaptive scanning system that dynamically adjusts based on web application responses.
- Comprehensive vulnerability reporting with remediation suggestions.
- Intuitive interface for configuring scans, reviewing results, and accessing guidance on resolving vulnerabilities.
- Compatibility with many web application frameworks.

1.6.3 Features and Functions

- Machine learning-powered vulnerability detection that adapts based on application responses.
- Minimized false positives through a trained model based on common vulnerability patterns.
- Real-time analysis and reporting of detected vulnerabilities.
- Support for SQL Injection, XSS and other prevalent web security vulnerabilities.

1.7 Software Development Lifecycle Model

For the development of the Automated Web Application Penetration Testing with Machine Learning Models, the **Agile Software Development Life Cycle** model is used because it is flexible, it supports iterative development, and it is responsive to changing security needs. The nature of this project including security scans, integration with machine learning modules, and report generation is greatly helped by ongoing feedback and modular releases, so Agile is the best fit. Agile Software Development Lifecycle applied to the Development of an Automated Web Application Penetration Testing Tool Using Machine Learning Models.

Table 1.1 describes the phases of Agile Software Development Lifecycle that are applied on our project. Each phase is described respectively.

Table 1.1 Agile Lifecycle

Phase	Description
Requirement Gathering	Functional as well as non-functional requirements were achieved by brainstorming and proposal writing. Agile user stories have been created for critical modules.
Sprint Planning	The collective development is divided into several 2-week sprints. Each sprint focuses on a major feature of the application (e.g., Scan Module, Report Generation, ML-Prediction).
Design	Use case diagrams, class diagrams, and architecture are created iteratively depending upon changing sprint objectives.
Development	The code is incrementally written. Functional components such as user login, scan configuration, and
Testing	Unit tests, integration testing, and security testing are carried out after every sprint for ensuring quality.
Evaluation	Demonstrations and presentations are carried out to stakeholders (peers and supervisors) at the end of every sprint to obtain feedback.
Documentation & Review	User documentation, reports, and technical documentation are revised sprint-wise for ongoing completeness.



Figure 1.1 Agile-Model

Figure 1.1 shows the Agile software development Methodology used for the Automated Web Application Penetration Testing using Machine Learning Models project. The diagram displays the project lifecycle phases including Requirements, Design, Development, Testing, Deployment, and Review.

1.7.1 Why Used Agile?

Rationale for Using Agile:

- **Iterative Development:** User authentication, scan running, passive information collection, and vulnerability reporting features can be built in sprints.
- **High-Frequency Feedback Loops:** Agile enables frequent interaction with superiors, colleagues, and security domain specialists to improve features and scanning semantics based on real-time feedback.
- **Early Issue Discovery:** Security modules and ML predictions can be tested early, avoiding expensive repairs later.
- **Accommodation to change:** Important changes in scan targets, report formats or machine learning integration can be managed effectively during sprint planning.

1.7.2 Advantages to This Project

- Enhanced modularity of features such as Scan Execution, Passive Recon, Integration of machine learning models, and Reporting.
- Ensures the iterative improvement of accuracy for vulnerabilities identification and remediation reasoning based upon initial test outcomes.
- Facilitates collaboration among developers and testers, which is crucial for its orientation towards security accuracy and reliability.

Chapter 2. Requirement Analysis

The functional and non-functional requirements necessary for creating the automated web application penetration testing tool and literature review are covered in this chapter.

2.1 Literature Review

The literature review emphasizes three main areas: Automated penetration testing tools, Machine learning for cybersecurity, and Challenges facing vulnerability detection. These areas form a comprehensive knowledge circle of the present state concerning web application security and the potential for innovations to take place.

2.1.1 Automated Penetration Testing Tools:

- OWASP ZAP and Burp Suites are commonly used vulnerability scanners. They are rule based, using regex expressions to identify known vulnerabilities. The downside to these vulnerability scanners is that they do not work well with zero day exploits or new attack technologies. They have a very high rate of false positives, resulting in a lot of manual work to determine the validity of the results.
- Nikto and Nessus are very efficient in identifying vulnerabilities related to servers. But these tools are not capable of assessing logical layers of the applications dealing with dynamic user inputs, which are essential in identifying SQL Injection and Cross-Site Scripting (XSS) vulnerabilities.

2.1.2 Machine Learning in Cybersecurity:

Current developments in machine learning technologies have proven the potential of the latter to improve vulnerability detection. For example:

- Supervised models like Random Forest and Support Vector Machines have reached an accuracy of 85% and above in identifying malicious inputs like SQLi payloads and XSS scripts.
- Techniques from Natural Language Processing (NLP) have been applied in the analysis of HTTP responses to identify anomalies such as unexpected database errors or malicious JavaScript processes.
- Despite these achievements, ML models encounter some difficulties when it comes to real-world applications. This is particularly because they need a substantial amount of labeled data for training, and they find it challenging to generalize over different web application architectures.

2.1.3 Challenges in Vulnerability Detection:

- **False Positives:** A major drawback of rule based scanning is that it identifies harmless anomalies that require manual validation, wasting time and efforts.
- **Scalability:** Manual testing tools like Burp Suite require lots of resources. They are not ideal for large-scale implementation.

2.1.4 Literature Review Summary

The literature review highlights major gaps in existing solutions, which this project aims to address.

Table 2.1 describes the literature review summary of our project and also gaps fulfilled by our project.

Table 2.1 Literature Review Summary

Aspect	Existing Tools	Gaps Addressed by This Project
Detection Method	Rule-based, static signatures	Hybrid approach: Combines rule-based scanning with ML pattern recognition
False Positives	High (e.g., OWASP ZAP flags 30% false positives)	ML reduces false positives by validating anomalies and learning from historical data
Adaptability	Limited to predefined rules	Reinforcement learning adapts scanning strategy based on application behavior
Cost	Expensive (e.g., Burp Suite Pro)	Open-source stack (Python, OWASP tools) reduces costs and increases accessibility
Integration	Limited to specific frameworks	Compatible with diverse architectures and DevSecOps workflows

2.1.5 OWASP ZAP

Owasp zap is a free and popular web application vulnerability scanner[1]. It is capable of offering a variety of options that could be used manually or automatically, hence appropriately used in CI/CD systems. However, it follows a rule-based approach that is not amenable to emerging threats. The method used is largely dependent on predefined rules; hence, it is bound to produce a large number of false positives.

2.1.6 Burp Suite

Famed for its powerful set of testing tools, Burp Suite [2] is the darling of professional security testers with richly detailed configurability and a deep level of testing ability. Although highly potent for the most complex security assessments, it is mainly manual and rule-based and demands extensive time and expertise to configure all properly. This makes it way too costly to use the Professional Edition in small teams, and without machine learning, it also misses adaptability to perpetually updating vulnerabilities.

2.1.7 Acunetix

Acunetix [3] is a high speed and high-performance vulnerability scanner commercially available and known to be accurate. It suits large enterprises well that seek fast security testing. More so, Acunetix has great integration options for all sorts of DevSecOps workflows with really detailed reporting, but its proprietary high cost models focus on bigger organizations as it is not so accessible to smaller teams. Moreover, the flexibility of custom testing of Acunetix is relatively lesser compared to its open-source alternatives. It also lacks the adaptability of machine learning-based optimizations hence making it less effective against the new threats.

2.1.8 Nessus

Nessus [4] is widely known for its scanning tool which identifies and assesses security risks cross cutting web applications, networks and systems. Developed by Tenable, Nessus provides a solution in thorough detection of high-impact vulnerabilities that may include misconfigurations of software, outdated software versions, and known exploits.

2.1.9 Nikto

Nikto [5] is an open-source web server scanner intended to detect security vulnerabilities and misconfigurations for web servers. It thoroughly examines HTTP and HTTPS servers for most common vulnerabilities, including cross-site scripting (XSS), SQL injection, RFI, and outdated software versions. It further identifies security issues like insecure CGI scripts, missing security patches, and improper HTTP headers. Nikto provides detailed reports in several formats, including HTML, CSV, and TXT, so the areas under attack will be identified. It is one of the irreplaceable tools for penetration testers and security consultants to use rapidly in assessing web server security and working over weaknesses.

2.2 Research Gap Identification

The majority of existing vulnerability scanners, like OWASP ZAP, Burp Suite, Acunetix, and Nessus, have a strong reliance on static, rule-based detection. This makes them inefficient at discovering new or evolving attack patterns. Most of these tools report high numbers of false positives, need expert configuration, and are not very adaptive in the case of changes in structure or behavior of web applications. Current research mainly focuses on traditional scanning techniques, while the integration of machine learning for dynamic, predictive vulnerability detection remains underexplored and insufficiently implemented in practical tools. Furthermore, most studies do not address the need for continuous learning, adaptive attack simulation, or real-time response analysis. There is an emerging need for an intelligent system that can learn from attack histories, dynamically readjust its strategies in response to application reactions, and provide more precise, automated assessments. Our project addresses this gap with the development of a machine learning-enhanced vulnerability scanner that improves detection accuracy, **reduces false positives**, and adapts to modern web application architectures, hence offering a more robust and scalable security solution.

2.3 Requirements Elicitation

This subsection briefly discusses the functional and non-functional requirements of the system along with the requirements traceability matrix.

2.3.1 Functional Requirements

In Software Engineering, functional requirement defines the scope of the project and working of the system components in a way that is feasible for both the client and the developing team. Below is the functional requirement for Automated Web Application Penetration Testing Using Machine Learning Models:

User Registration

Table 2.2 presents a detailed and structured overview of the functional requirements of the Signup module.

Table 2.2 FR-001 Signup

FR01-01	System shall provide a registration form with email, username, and password fields.
---------	---

FR01-02	System shall validate that the password meets complexity requirements (e.g., minimum 8 characters, a special symbol, and a number).
FR01-03	System shall send an email verification link to the user after successful registration.

Login

Table 2.3 presents a detailed overview of the functional requirements of the Login module.

Table 2.3 FR-002 Login

FR02-01	System shall authenticate users with email/username and password.
FR02-02	System shall display appropriate error messages for failed login attempts (e.g., invalid credentials, inactive account).

User Log out

Table 2.4 presents a detailed overview of the functional requirements of the Log out module.

Table 2.4 FR-003 User Log out

FR03-01	System shall allow authenticated users to log out of their account.
FR03-02	System shall terminate the current session and invalidate session tokens or cookies.
FR03-03	System shall redirect the user to the login or landing page after successful logout.

Password Recovery

Table 2.5 presents a detailed overview of the functional requirements of the Password recovery module.

Table 2.5 FR-004 Password Recovery

FR04-01	System shall provide a password reset feature via the "Forgot Password" option.
FR04-02	System shall send a secure, time-limited password reset link via email.

User Profile Management

Table 2.6 presents a detailed overview of the functional requirements of the User Profile Management module.

Table 2.6 FR-005 User Profile Management

FR05-01	System shall allow users to edit personal information such as name and contact details.
---------	---

FR05-02	System shall allow users to upload and manage a profile picture.
FR05-03	System shall allow users to configure privacy settings related to profile visibility.
FR05-04	System shall provide users with notification preferences (on/off).

View Projects

Table 2.7 presents a detailed overview of the functional requirements of the Project Management module.

Table 2.7 FR-006 View Projects

FR06-01	System shall display a dashboard of recent projects associated with the logged-in user.
FR06-02	If no projects are present, the system shall show a message like “No projects found.”
FR06-03	System shall allow users to view basic details of each project, including project name and target URL.

Add Project

Table 2.8 presents a detailed overview of the functional requirements of the Project Management module.

Table 2.8 FR-007 Add Project

FR07-01	System shall provide a form for creating a new project with fields: Project Name and Target URL.
FR07-02	System shall validate the format and reachability of the target URL before saving.
FR07-03	System shall save the project under the user’s account.
FR07-04	System shall redirect the user to the project details page after successful creation.

Open Project

Table 2.9 presents a detailed overview of the functional requirements of the Project Management module.

Table 2.9 FR-008 Open Project

FR08-01	System shall allow users to open a selected project from the dashboard.
---------	---

FR08-02	System shall display the project's information, such as name, target URL, and creation timestamp.
FR08-03	System shall provide navigation options to access scan configuration, scanning, and report modules.

Target Configuration

Table 2.10 presents a detailed overview of the functional requirements of the Target Configuration Management module.

Table 2.10 FR-009 Target Configuration

FR09-01	System shall accept a website URL as a scan target.
FR09-02	System shall verify website reachability using an HTTP status check.
FR09-03	System shall allow users to select scan depth, intensity, and whether to respect robots.txt.
FR09-04	System shall allow configuration of advanced options such as user-agent, request delay, and custom headers.

Basic Passive Information Gathering

Table 2.11 presents a detailed overview of the functional requirements of the Passive Information Gathering module.

Table 2.11 FR-010 Basic Passive Information Gathering

FR10-01	System shall perform a quick initial information gathering on the target URL.
FR10-02	System shall use a simple DNS lookup function to retrieve a records for the domain.
FR10-03	System shall attempt to retrieve the raw text of the robots.txt file (if it exists).
FR10-04	System shall attempt to retrieve the raw XML content of the sitemap.xml file (if it exists).
FR10-05	System shall collect web server and technology details from the initial HTTP response headers.
FR10-06	System shall store all collected passive information for future use.

Scan Execution

Table 2.12 presents a detailed overview of the functional requirements of the Scan Execution module.

Table 2.12 FR-011 Scan Execution

FR11-01	System shall initiate an automated scan on the configured target.
FR11-02	System shall crawl the target and collect forms, cookies, headers, discovered URLs, and web structure.
FR11-03	System shall log progress and display real-time scan status.
FR11-04	System shall allow users to manually stop the scan during execution.
FR11-05	System shall pass the collected information, including passive and active reconnaissance data, to the ML Detection Module for further analysis.

Vulnerability Analysis and Prediction

Table 2.13 presents a detailed overview of the functional requirements of the Vulnerability Analysis and Prediction module.

Table 2.13 FR-012 Vulnerability Analysis and Prediction

FR12-01	System shall receive all data collected during the scanning process (active + passive).
FR12-02	System shall analyze the received data using the trained ML model to identify possible vulnerabilities.
FR12-03	System shall generate a list of predicted vulnerabilities, each with a confidence score.
FR12-04	System shall forward these findings to the Reporting & Visualization Module.

Data Collection and Model Training

Table 2.14 presents a detailed overview of the functional requirements of the Data Collection and Model Training module.

Table 2.14 FR-013 Data Collection and Model Training

FR13-01	System shall collect and preprocess data from previous scan results for ML model training.
FR13-02	System shall use machine learning models to classify security threats and predict potential risks based on the received data.
FR13-03	System shall periodically update the ML models using new data and feedback.

Vulnerability-Based Remediation Suggestions

Table 2.15 presents a detailed overview of the functional requirements of the Vulnerability-Based Remediation Suggestions module.

Table 2.15 FR-014 Vulnerability-Based Remediation Suggestions

FR14-01	System shall map predicted vulnerabilities to remediation best practices using a predefined knowledge base.
FR14-02	System shall display these suggestions to the user in the scan result interface.
FR14-03	System shall include these suggestions in the downloadable scan report.
FR14-04	System shall group suggestions by vulnerability category (e.g., Authentication, Input Validation, etc.).

Report Generation

Table 2.16 presents a detailed overview of the functional requirements of the Result Report Generation module.

Table 2.16 FR-015 Report Generation

FR15-01	System shall generate a summary report of the scan results.
FR15-02	System shall categorize vulnerabilities by severity (Critical, High, Medium, Low).
FR15-03	System shall provide actionable remediation guidance for each vulnerability.

Report Management

Table 2.17 presents a detailed overview of the functional requirements of the Report Management module.

Table 2.17 FR-016 Report Management

FR16-01	System shall allow users to view, organize, and manage all scan reports per project.
FR16-02	System shall allow exporting reports in PDF format.
FR16-03	System shall maintain a report history for each project and scan session.

2.3.2 Non-functional Requirements

Non-functional requirements describe the system behaviour. It represents the quality or attribute of the system. Below is the non-functional requirement for Automated Web Application Penetration Testing Using Machine Learning Models:

Security

Table 2.18 describes the security related non-functional requirements of the proposed system.

Table 2.18 NFR-001 Security

NFR01-01	The system shall ensure secure authentication using hashed passwords and encrypted communication (SSL/TLS).
NFR01-02	Users shall only access their own data, except where explicitly allowed.
NFR01-03	All API endpoints shall be protected against common attacks including but not limited to XSS, and injection attacks.
NFR01-04	The system shall sanitize all input and output to prevent command injection and related vulnerabilities.

Simple and Minimalistic Design

Table 2.19 describes the Simple and Minimalistic Design related non-functional requirements of the proposed system.

Table 2.19 NFR-002 Simple and Minimalistic Design

NFR02-01	The user interface shall utilize a clean, minimalist design with intuitive navigation and clear visual hierarchies.
NFR02-02	The system shall implement a consistent color scheme that enhances readability and reduces visual fatigue
NFR02-03	The system shall use straightforward language and avoid technical jargon in user-facing elements, with optional tooltips for complex terms.
NFR02-04	The system shall maintain a consistent interface pattern across different functional areas to reduce the learning curve.
NFR02-05	Error messages shall be clear, concise, and provide actionable solutions when possible.

Reliability

Table 2.20 describes the reliability related non-functional requirements of the proposed system.

Table 2.20 NFR-003 Reliability

NFR03-01	In the event of a system crash, the recovery process shall restore the system to its previous state within 45 minutes.
NFR03-02	The system shall validate all scan results for consistency and completeness before presenting them to users.

NFR03-03	The system shall gracefully degrade functionality rather than completely fail when operating under high load or resource constraints.
NFR03-04	The system shall maintain an uptime of at least 95% during standard operating hours (excluding scheduled maintenance).

Performance

Table 2.21 describes the performance related non-functional requirements of the proposed system.

Table 2.21 NFR-004 Performance

NFR04-01	The system shall maintain a maximum page load time of 3 seconds for the dashboard and results pages under typical load conditions.
NFR04-02	The system shall be able to handle at least 30 simultaneous user sessions without degradation in response time.
NFR04-03	API endpoints shall respond within 10 second for 95% of requests under normal load conditions.

Usability

Table 2.22 describes the usability related non-functional requirements of the proposed system.

Table 2.22 NFR-005 Usability

NFR05-01	The system shall provide an intuitive interface that allows users to initiate a vulnerability scan with no more than 5 clicks from the dashboard.
NFR05-02	Error messages shall be human-readable, precisely indicate the issue, and suggest corrective actions when possible.
NFR05-03	The system shall maintain consistency in terminology, layout, and interaction patterns across all components.

Data Integrity

Table 2.23 describes the data integrity related non-functional requirements of the proposed system.

Table 2.23 NFR-006 Data integrity

NFR06-01	Scan reports and logs shall be protected from unauthorized modifications.
NFR06-02	The system shall prevent data corruption during concurrent access by implementing proper transaction management.
NFR06-03	The system shall perform regular integrity checks to ensure data consistency.

Availability

Table 2.24 describes the availability related non-functional requirements of the proposed system.

Table 2.24 NFR-007 Availability

NFR07-01	The system shall support multiple concurrent users without performance degradation.
NFR07-02	The system shall implement load balancing to handle traffic spikes during peak usage periods.
NFR07-03	Users shall be notified in case of downtime or system maintenance.

Interoperability

Table 2.25 describes the interoperability related non-functional requirements of the proposed system.

Table 2.25 NFR-008 Interoperability

NFR08-01	The system shall provide REST API endpoints for integration with third-party security tools and platforms.
NFR08-02	The system shall implement standard authentication protocols (OAuth 2.0, API keys) for secure API access.

Efficient Session Management

Table 2.26 describes the session management related non-functional requirements of the proposed system.

Table 2.26 NFR-009 Efficient Session Management

NFR09-01	User sessions shall automatically timeout after 30 minutes of inactivity to prevent unauthorized access.
NFR09-02	The system shall allow users to maintain multiple active sessions across different devices.
NFR09-03	When a user changes their password, all existing sessions shall be invalidated for security purposes.

2.3.3 Requirements Traceability Matrix

Table 2.27 describes the requirements traceability matrix of the proposed system.

Table 2.27 Requirements Traceability Matrix

ID	Associate ID	Requirements	Use case ID	Test Case ID	Status
1.	FR01	User Registration	UC ID 001	TC ID 001	Pass
2.	FR02	Login	UC ID 002	TC ID 002	Pass
3.	FR03	Logout	UC ID 003	TC ID 003	Pass
4.	FR04	Password Recovery	UC ID 004	TC ID 004	Pass
5.	FR05	User Profile Management	UC ID 005	TC ID 005	Pass
6.	FR06	View Projects	UC ID 006	TC ID 006	Pass
7.	FR07	Add Project	UC ID 007	TC ID 007	Pass
8.	FR08	Open Project	UC ID 008	TC ID 008	Pass
9.	FR09	Target Configuration	UC ID 009	TC ID 009	Pass
10.	FR10	Basic Passive Information Gathering	UC ID 010	TC ID 010	Pass
11.	FR11	Scan Execution	UC ID 011	TC ID 011	Pass
12.	FR12	Vulnerability Analysis And Prediction	UC ID 012	TC ID 012	Pass
13.	FR13	Data Collection and Model Training	UC ID 013	TC ID 013	Pass
14.	FR14	Vulnerability Based Remediation Suggestoins	UC ID 014	TC ID 014	Pass
15.	FR15	Report Generation	UC ID 015	TC ID 015	Pass
16.	FR16	Report Management	UC ID 016	TC ID 016	Pass

2.4 Use Case Description

This subsection briefly discusses the use case description for the system's functional requirements.

2.4.1 User Registration

Table 2.28 describes the use case for user registration, including its purpose, involved actors, preconditions, main workflow, and expected outcomes.

Table 2.28 Use Case Description-001 Signup

Use case ID 001	
Use case name User Registration (sign up) Priority High	Primary Actor User Other Participating Actor Admin
Use Case Summary A user registers an account by providing an email, username, and password, or by using Google authentication, followed by account verification.	Pre-condition 1- User is not logged in. 2- No existing account with the same email or username.
Normal Course of Events	
1. User accesses the registration page. 2. User fills in the registration form with email, username, and password. 2a. Alternatively, the user clicks "Sign up with Google", and the system redirects to Google OAuth. 2b. After Google confirms authentication, the system creates and verifies the user account automatically. 3. System validates the form. 4. If password does not meet complexity requirements, the system provides an error message. 5. System sends a verification email with a link. 6. User clicks the verification link to verify the email. 7. Account is successfully created and verified.	
Alternative Path	
1. User submits an invalid email or username, and the system displays an error message. 2. Password doesn't meet complexity requirements, and the system asks the user to correct it. 3. If the Google authentication fails or is cancelled, the system displays an appropriate error message.	
Conclusion	The user securely registers an account either through standard registration with email verification or through Google authentication.
Post Conditions	
1. A new user account is created and stored in the database. 2. User account is verified and ready for login. 3. User may proceed to login using valid credentials.	
Implementation, constraints, and Specifications	
1. Password must contain at least 8 characters, one special symbol, and one number. 2. Email verification link should expire within 15 minutes. 3. Google OAuth 2.0 protocol will be used for Google authentication.	

Use Case Cross References		
Includes Email Verification, Google Authentication (for Google sign up)	Extends None	Exceptions 1. Invalid email format. 2. Weak password. 3. Verification link expires or is invalid. 4. Server issues preventing email dispatch. 5. Google returns an invalid or expired token

2.4.2 Login

Table 2.29 describes the use case for user login, including its purpose, involved actors, preconditions, main workflow, and expected outcomes.

Table 2.29 Use Case Description-002 Login

Use case ID 002	
Use case name User Login Priority High	Primary Actor User Other Participating Actor None
Use Case Summary A user logs in by providing their email/username and password, and the system authenticates them.	Pre-condition 1-User has already registered and created an account. 2- User is currently logged out.
Normal Course of Events	
1. User accesses the login page. 2. User enters their email/username and password. 3. System validates input format. 4. System checks credentials against the database. 5. If credentials are valid and account is active, system logs the user in. 6. User is redirected to the dashboard or home page.	
Alternative Path	
1. User enters incorrect email/username or password. 2. System displays an error message indicating the login failure.	
Conclusion	The user successfully logs in after the system authenticates their credentials, granting access to their dashboard or home page.
Post Conditions	
1. User is logged into the system. 2. A session is created for the user. 3. Login activity is optionally logged for security auditing.	
Implementation, constraints, and Specifications	
1. Email/username must match a verified registered account. 2. Password must be correct and securely compared. 3. Rate limiting must be enforced after multiple failed attempts. 4. Session should expire after defined inactivity time.	

Use Case Cross References		
Includes Credential Validation	Extends None	Exceptions 1. Invalid email/username format. 2. Incorrect password.

2.4.3 Logout

Table 2.30 describes the use case for user logout, including its purpose, involved actors, preconditions, main workflow, and expected outcomes.

Table 2.30 Use Case Description-003 Logout

Use case ID: 003		
Use case name Logout Priority High		Primary Actor User Other Participating Actor None
Use Case Summary This use case describes how an authenticated user securely logs out of their session. The system invalidates the session token, clears any cached authentication data, and redirects the user to the login or home page.		Pre-condition The users must be logged into the system
Normal Course of Events		
1. User clicks the “Logout” button from the system interface. 2. System sends a logout request to the authentication handler. 3. System invalidates the session token on the server. 4. System clears any client-side authentication data (e.g., cookies or local Storage). 5. User is redirected to the login or landing page. 6. System optionally logs the logout event in the user’s activity log.		
Conclusion	The user is successfully logged out, their session is securely terminated, and they are returned to a neutral page (e.g., login or landing page).	
Post Conditions		
1. Session token is destroyed. 2. User is no longer authenticated in the system. 3. Logout is optionally recorded in the activity logs.		
Implementation, constraints, and Specifications		
1. Session tokens must be invalidated on the server side. 2. Logout should work consistently across tabs and devices. 3. Cached login tokens on the client side must be securely cleared. 4. Activity logs should timestamp the logout action.		
Use Case Cross References		
Includes Session Invalidation, Activity Logging	Extends None	Exceptions 1. Session token not found or already expired.

		2. Failure to clear authentication data. 3. Server error during session termination.
--	--	---

2.4.4 Password Recovery

Table 2.31 describes the use case for password recovery, including its purpose, involved actors, preconditions, main workflow, and expected outcomes.

Table 2.31 Use Case Description-004 Password Recovery

Use case ID 004	
Use case name Password Recovery Priority Medium	Primary Actor User Other Participating Actor None
Use Case Summary A user who has forgotten their password can use the “Forgot Password” feature. The system validates the email and sends a secure, time-limited password reset link. Upon clicking the link, the user can enter a new password and regain account access.	Pre-condition 1. User has an existing and registered account. 2. User does not remember their password. 3. Email system is operational.
Normal Course of Events	
1. User accesses the “Forgot Password” page. 2. User enters their registered email address. 3. System validates the email format and checks its presence in the database. 4. If the email is valid, the system generates a time-limited password reset token. 5. System sends a secure password reset link to the user’s email. 6. User opens the link and is redirected to the reset page. 7. User enters a new password. 8. System validates the new password for complexity. 9. Password is updated successfully. 10. User may now log in with the new password.	
Alternative Path	
1-User provides an invalid or unregistered email address. 2. System displays an error message indicating the email is not found in the system.	
Conclusion	The user securely resets their password via a validated email link, restoring account access after successful authentication.
Post Conditions	
1- User’s password is updated in the system. 2- Any previously issued sessions or tokens are invalidated. 3- User can now access their account using the new password.	
Implementation, constraints, and Specifications	
1. Password reset link must expire within 15–30 minutes. 2. Reset links must be one-time use only. 3. Password must meet minimum complexity requirements (8+ chars, symbol, number).	

4. System must invalidate all existing sessions after password reset (optional security feature).		
Use Case Cross References		
Includes Email Verification, Password Validation	Extends Login (after reset)	Exceptions 1. Invalid email format. 2. Email account not found.

2.4.5 Profile Management

Table 2.32 describes the use case for profile management, including its purpose, involved actors, preconditions, main workflow, and expected outcomes.

Table 2.32 Use Case Description-005 Profile Management

Use case ID 005	
Use case name Profile Management Priority High	Primary Actor User Other Participating Actor None
Use Case Summary The system allows users to edit their personal information, manage their profile picture, and configure privacy settings for profile visibility.	Pre-condition 1. User is logged in. 2. User has an existing profile in the system.
Normal Course of Events	
1. User accesses the profile management page. 2. User edits personal information such as name and contact details. 3. User uploads a profile picture and manages it. 4. User sets or updates notification preferences. 5. System saves the updated information and settings.	
Alternative Path	
1. User does not upload a profile picture. 2. System saves the profile with default or no picture. 3. User does not configure privacy controls 4. System saves the profile with default visibility settings	
Conclusion	The user updates their profile information, manages their profile picture, and configures privacy settings, with changes securely saved by the system.
Post Conditions	
1. User's personal information is updated. 2. Profile picture is uploaded and managed. 3. Privacy settings are configured.	
Implementation, constraints, and Specifications	
1. Image file size must not exceed 2 MB. 2. Supported image types: JPG, PNG. 3. Input fields must be validated (e.g., phone number, email format).	

4. Profile data should be stored securely and comply with privacy settings.		
Use Case Cross References		
Includes Input Validation, Image Upload Handler, Privacy Settings Module.	Extends User Dashboard	Exceptions 1. Profile picture upload failure. 2. Invalid information input (e.g., incorrect phone number or email format).

2.4.6 View Projects

Table 2.33 describes the use case for view project, including its purpose, involved actors, preconditions, main workflow, and expected outcomes.

Table 2.33 Use Case Description-006 View Projects

Use case ID 006	
Use case name View Projects Priority High	Primary Actor User Other Participating Actor None
Use Case Summary A logged-in user accesses the dashboard to view all projects linked to their account. The system fetches recent project data, showing the project name and target URL. If no projects exist, an appropriate message is shown.	Pre-condition 1-User is logged in. 2- User has existing projects or no projects.
Normal Course of Events	
1. User accesses the dashboard. 2. System displays a list of recent projects associated with the user. 3. System shows project name and target URL for each project. 4. If the user has no projects, the system shows the message “No projects found”.	
Alternative Path	
1. User has no projects. 2. System displays the message “No projects found”.	
Conclusion	The user views a dashboard displaying their recent projects or a “No projects found” message upon successful login.
Post Conditions	
1- Project list is loaded and visible. 2- User may now open or manage a project.	

Implementation, constraints, and Specifications		
1. System must handle empty datasets and server latency gracefully. 2. System should support pagination for large project lists. 3. Display should auto-refresh if projects are added or deleted in-session.		
Use Case Cross References		
Includes Project Retrieval, UI Rendering	Extends None	Exceptions 1. Error in fetching user projects. 2. Server timeout or dashboard loading failure.

2.4.7 Add Project

Table 2.34 describes the use case for add project, including its purpose, involved actors, preconditions, main workflow, and expected outcomes.

Table 2.34 Use Case Description-007 Add Project

Use case ID 007	
Use case name Add Project Priority Medium	Primary Actor User Other Participating Actor None
Use Case Summary A logged-in user creates a new project by providing a project name and a valid target URL. The system validates the URL, checks reachability, and saves the project under the user's account. Upon success, the user is redirected to the project details page.	Pre-condition 1. User is logged in. 2. User has access to the "Add Project" form.
Normal Course of Events	
1. User navigates to the "Add Project" form. 2. User inputs the project name and target URL. 3. System validates the input format of the target URL. 4. System performs an HTTP reachability check on the URL. 5. If valid and reachable, the system saves the project data. 6. System redirects the user to the project details page.	
Alternative Path	
1. User enters an invalid URL format. 2. System displays an error message indicating invalid URL. 3. User enters a non-reachable URL. 4. System displays an error message indicating the URL is not reachable.	
Conclusion	The user successfully creates a project after system validation of the URL's format and reachability, redirecting to the project details page.
Post Conditions	
1- Project is saved in the database under the current user's account.	

2- User is redirected to the project details view.		
Implementation, constraints, and Specifications		
1. Target URL must follow standard URL format. 2. All fields must be mandatory and validated before save.		
Use Case Cross References		
Includes	Extends	Exceptions
URL Format Validation, HTTP Status Check	View Projects	1. Invalid URL format. 2. Unreachable domain/server 3. Database error when saving project

2.4.8 Open Project

Table 2.35 describes the use case for open project, including its purpose, involved actors, preconditions, main workflow, and expected outcomes.

Table 2.35 Use Case Description-008 Open Project

Use case ID 008	
Use case name Open Project Priority Medium	Primary Actor User Other Participating Actor None
Use Case Summary A logged-in user selects a project from their dashboard. The system fetches and displays that project's details (name, target URL, creation date) and provides access to associated modules such as scan configuration, execution, and report viewing.	Pre-condition 1. User is authenticated and logged in. 2. User has one or more projects saved in their account. 3. Project data exists and is not corrupted.
Normal Course of Events	
1. User accesses the dashboard. 2. User selects a project from the list. 3. System fetches project data from the database. 4. System displays the following: Project name, Target URL, Date and time of creation 5. System provides navigation buttons/links for: • Scan Configuration • Scan Execution • Report Viewing 6. User interacts with any of the provided modules.	
Alternative Path	
1. User selects a project, but the project details cannot be loaded due to server issues. 2. System displays an error message indicating failure to load project details.	
Conclusion	The user successfully accesses a project's details and navigation modules after selecting it from the dashboard, provided valid data is available.

Post Conditions		
User is redirected to the project details page, where project information and navigation options are displayed.		
Implementation, constraints, and Specifications		
Project details must be loaded via authenticated, secure database queries. System must handle invalid project IDs gracefully. Navigation modules (scan, report) must be dynamically linked to the selected project.		
Use Case Cross References		
Includes Project Detail Retrieval, Scan Module, Report Module	Extends View project	Exceptions 1. Project not found or deleted 2. Database timeout or error on retrieval

2.4.9 Target Configuration

Table 2.36 describes the use case for target configuration, including its purpose, involved actors, preconditions, main workflow, and expected outcomes.

Table 2.36 Use Case Description-009 Target Configuration

Use case ID 009	
Use case name Target Configuration Priority High	Primary Actor User Other Participating Actor None
Use Case Summary A user configures a website URL as the scan target, sets scan parameters, customizes crawling settings, and optionally enables Passive Information Gathering (PIG) features such as WHOIS, DNS, and header collection.	Pre-condition 1- User is logged in. 2- User is on the target configuration page.
Normal Course of Events	
1. User accesses the target configuration page. 2. User inputs the website URL as the scan target. 3. System verifies the reachability of the website URL through an HTTP status check. 4. User selects scan depth, intensity, and whether to respect robots.txt. 5. User configures the user-agent, delay between requests, and custom headers for crawling. 6. User selects whether to enable Passive Information Gathering. 7. If enabled, user chooses PIG options (e.g., WHOIS, DNS lookup, IP info, headers). 8. System saves the entire scan configuration including PIG preferences.	
Alternative Path	
1. User enters an invalid or unreachable website URL. 2. System displays an error message indicating the website is not reachable. 3. User configures invalid parameters (e.g., incorrect user-agent). 4. System shows validation feedback for passive information settings.	

Conclusion	The user configures scan parameters and crawling settings after system validation of the URL reachability and input criteria, enabling customized scanning workflows		
Post Conditions			
1. Website URL is set as the scan target. 2. Scan parameters, crawling options, and passive information gathering settings are stored.			
Implementation, constraints, and Specifications			
1. The system should support standard passive recon modules. 2. PIG settings should not interfere with scan execution if disabled. 3. URL must respond to an HTTP check.			
Use Case Cross References			
Includes URL Format Validation, HTTP Status Check, Passive Recon Configuration	Extends Scan Execution	Exceptions 1. Invalid target format 2. HTTP check fails 3. Passive info modules unavailable 4. Server error while saving configuration.	

2.4.10 Passive Information Gathering

Table 2.37 describes the use case for passive scanning, including its purpose, involved actors, preconditions, main workflow, and expected outcomes.

Table 2.37 Use Case Description-010 Passive Information Gathering

Use case ID 010	
Use case name Passive Information Gathering Priority High	Primary Actor Security Analyst Other Participating Actor Target Web Application
Use Case Summary This use case describes how the system performs non-intrusive reconnaissance on a configured target URL by gathering publicly available data using OSINT methods.	Pre-condition 1. A valid target URL must be configured. 2. Scan mode must be set to Passive Reconnaissance. 3. The target must be reachable (no connection errors).
Normal Course of Events	
1. Security analyst requests passive information gathering for configured target 2. System resolves target domain's DNS records 3. System retrieves robots.txt and sitemap.xml from root path 4. System sends HEAD requests to collect server headers	

- System performs passive technology fingerprinting - System logs and stores metadata in scan database - System presents collected data to analyst - Data becomes available for ML analysis and reporting			
Alternative Path			
- If analyst cancels scan then System stops collection and saves partial data - If target blocks requests then System logs blocking behavior as intelligence - If DNS resolution fails then System attempts alternative resolution methods			
Conclusion	The system completes passive reconnaissance and stores the retrieved data for future reference or extended scanning workflows.		
Post Conditions			
- Analyst can review collected passive intelligence - Data is structured and available for analysis modules - Scan session contains complete passive reconnaissance results			
Implementation, constraints, and Specifications			
- Must avoid sending payloads that could be flagged as intrusive. - Should comply with legal and ethical OSINT boundaries. - Only uses GET/HEAD methods. - Data should be structured for easy parsing and report generation.			
Use Case Cross References			
Includes DNS Resolver, robots.txt and sitemap.xml Fetcher, HTTP Header Parser	<table> <tr> <td>Extends Can be extended by Active Scan or Technology Detection.</td><td>Exceptions - Network unreachable. - Invalid/malformed URL. - HTTP error response. - Timeout exceeded during lookup. </td></tr> </table>	Extends Can be extended by Active Scan or Technology Detection.	Exceptions - Network unreachable. - Invalid/malformed URL. - HTTP error response. - Timeout exceeded during lookup.
Extends Can be extended by Active Scan or Technology Detection.	Exceptions - Network unreachable. - Invalid/malformed URL. - HTTP error response. - Timeout exceeded during lookup.		

2.4.11 Scan Execution

Table 2.38 describes the use case for scan execution, including its purpose, involved actors, preconditions, main workflow, and expected outcomes.

Table 2.38 Use Case Description-011 Scan Execution

Use case ID 011	
Use case name Scan Execution Priority High	Primary Actor User Other Participating Actor None
Use Case Summary A user initiates an automated scan on the configured website target. The system first performs passive information gathering (like DNS records, headers, WHOIS), then crawls the website actively, logs progress, and allows manual stoppage of the scan.	Pre-condition - User has configured the target website. - User has initiated the scan process.

Normal Course of Events		
1. User initiates the scan on the configured target. 2. System performs passive information gathering (e.g., DNS records, HTTP headers). 3. Passive data is logged and visualized for user review. 4. System begins crawling the website. 5. System collects forms, cookies, HTTP headers, and discovered URLs. 6. System logs progress and provides live status during the scan. 7. System continues scanning until complete or manually stopped.		
Alternative Path		
1- User manually stops the scan. 2- System halts the scan and provides status update. 3- System saves the data collected until that point, including passive recon.		
Conclusion	The user successfully executes an automated scan. The system collects both passive and active data, logs progress, and saves results until completion or manual stoppage.	
Post Conditions		
1- The scan is either completed or stopped. 2- Passive and active scan data (DNS, WHOIS, headers, forms, URLs, etc.) is collected, logged, and stored.		
Implementation, constraints, and Specifications		
1. Passive information gathering must precede active scanning. 2. Scan execution must not proceed without valid target URL. 3. System must provide real-time status and logs during scanning. 4. User can interrupt the scan at any time.		
Use Case Cross References		
Includes	Extends	Exceptions
Passive Information Gathering, Website Crawling.	None	1- Scan failure due to server issues. 2- Website crawling fails due to connection problems. 3- Scan is interrupted unexpectedly. 4- Passive recon fails due to domain resolution issues.

2.4.12 Vulnerability Analysis and Prediction

Table 2.39 describes the use case for vulnerability analysis and prediction, including its purpose, involved actors, preconditions, main workflow, and expected outcomes.

Table 2.39 Use Case Description-012 Vulnerability Analysis and Prediction.

Use case ID 012		
Use case name Vulnerability Analysis and Prediction. Priority High		Primary Actor Security Analyst Other Participating Actor None
Use Case Summary The system automatically analyzes crawled website data to identify security vulnerabilities including SQL Injection, XSS, and other OWASP Top 10 issues, using both active testing and passive reconnaissance data.		Pre-condition 1- The user has started a scan on the target website. 2- The website is successfully crawled for analysis. 3- Scan execution (including passive info gathering and crawling) has been completed.
Normal Course of Events		
1- System automatically begins vulnerability analysis after scan completion. 2- System processes collected scan data from crawling and passive reconnaissance. 3- System performs active testing for SQL Injection vulnerabilities. 4- System conducts XSS detection through JavaScript injection attempts. 5- System checks for other OWASP Top 10 vulnerabilities. 6- System correlates findings with passive data (headers, robots.txt, etc.). 7- System Labels the detected vulnerabilities with severity level and confidence score. 8- System Stores the result and triggers machine learning prediction analysis. 9- System sends notification to analyst that analysis is ready. 10- Results are sent to Remediation Suggestion module.		
Alternative Path		
1. Target application blocks analysis attempts. 2. SSL/TLS certificate issues prevent secure analysis 3. Network connectivity issues during active testing		
Conclusion	It does the complete cycle of vulnerability detection using both active and passive inputs, and prepares detailed data for reporting and further ML based prediction.	
Post Conditions		
1. All the vulnerabilities detected are stored in the results of a scan. 2. Data becomes available to the Reporting and machine learning modules. 3. Results are then passed to the Remediation Suggestion module.		
Implementation, constraints, and Specifications		
1. Detection engine should support numerous vulnerability types, such as OWASP Top 10. 2. The scan engine should timeout if delays in responses exceed the threshold.		
Use Case Cross References		
Includes Passive Information Gathering, Input Validation, Scan Execution.	Extends Target Configuration	Exceptions 1- Scan failure due to server issues.

2.4.13 Data Collection and Model Training

Table 2.40 describes the use case for data collection and model training, including its purpose, involved actors, preconditions, main workflow, and expected outcomes.

Table 2.40 Use Case Description-013 Data Collection and Model Training

Use case ID 013		
Use case name Data Collection and Model Training Priority High		Primary Actor ML Administrator/Data Scientist Other Participating Actor None
Use Case Summary The system processes data from past scan results, employs machine learning algorithms to identify security threats, and then learns from the data by updating the algorithms using new data that has been collected. - Processes data from past scan results. - Analyzes patterns and trends in historical scan data		Pre-condition 1- Previous scan results are available. 2- ML models are set up for classification and risk prediction.
Normal Course of Events		
1- ML Administrator starts the process of model training. 2- The system retrieves the historical scanning data from the storage. 3- System performs data cleaning and preprocessing. 4- System splits data into training/validation sets. 5- System trains ML algorithms on the dataset. 6- System evaluates model performance using test data. 7- ML Administrator reviews training results and accuracy metrics.		
Alternative Path		
1- Data preprocessing fails due to incomplete or corrupted scan results. 2- System logs an error and requests data correction.		
Conclusion	The system collects and processes scan data to train and update machine learning models, enabling ongoing security threat classification and prediction.	
Post Conditions		
1- ML models are updated with new data. 2- The system is able to classify and predict security threats		
Implementation, constraints, and Specifications		
1. Scan data must be labeled or structured for supervised training. 2. Training must not block live operations (done asynchronously). 3. Model updates must be version-controlled. 4. Accuracy must meet minimum threshold before deployment. 5. Must support rollback to previous model if training fails.		
Use Case Cross References		
Includes Data Preprocessing Module, Model Evaluation Module	Extends Vulnerability Prediction	Exceptions 1- Incomplete or corrupted scan results. 2- Failure in the model training process. 3- System errors during data collection or preprocessing.

2.4.14 Remediation Suggestion Based on ML Prediction

Table 2.41 describes the use case for remediation suggestion based on ML prediction, including its purpose, involved actors, preconditions, main workflow, and expected outcomes.

Table 2.41 Use Case Description-014 Remediation Suggestion Based on ML Prediction

Use case ID 014		
Use case name Remediation Suggestion Based on ML Prediction Priority High		Primary Actor Security Analyst/User Other Participating Actor None
Use Case Summary The system automatically generates tailored remediation suggestions based on ML-predicted vulnerabilities and presents them to the security analyst through the dashboard and reports.		Pre-condition 1. ML model has completed vulnerability prediction. 2. A scan session with results is available.
Normal Course of Events		
1. Actor requests remediation suggestions (either as part of scan completion or separately) 2. System retrieves ML vulnerability predictions with confidence scores 3. System matches each vulnerability against the Remediation Knowledge Base 4. System generates appropriate suggestions for each vulnerability type 5. System displays suggestions to the user in the results dashboard 6. System includes suggestions in the generated PDF report under "Remediation Guidance".		
Alternative Path		
1. If Remediation Knowledge Base is unavailable then System provides generic security guidance 2. If user lacks permissions then System displays limited suggestions		
Conclusion	The system successfully provides clear, actionable remediation steps tailored to the user’s scan results, improving security posture awareness.	
Post Conditions		
1. User sees relevant suggestions linked to their vulnerabilities. 2. Report contains a remediation guidance section.		
Implementation, constraints, and Specifications		
1. Knowledge base of suggestions must be kept updated with OWASP and security best practices. 2. Suggestions should be prioritized by severity of the corresponding issue. 3. UI and report format must display them clearly under each vulnerability section.		
Use Case Cross References		
Includes Vulnerability Prediction, Report Generation.	Extends Scan Execution	Exceptions 1. No suggestions found for a particular issue then fallback to generic guidance. 2. Report generation fails to include suggestions then notified to user.

2.4.15 Report Generation

Table 2.42 describes the use case for report generation, including its purpose, involved actors, preconditions, main workflow, and expected outcomes.

Table 2.42 Use Case Description-015 Tournament Analysis Reporting

Use case ID 015	
Use case name Report Generation Priority High	Primary Actor User Other Participating Actor None
Use Case Summary The system generates a detailed report based on scan results. The report includes vulnerabilities detected during both active and passive scanning, classifies them by severity level, integrates ML-based predictions with confidence scores, and provides actionable remediation guidance. The report can be viewed or exported in PDF format. - Includes vulnerabilities detected during both active and passive scanning. - Classifies vulnerabilities by severity level. - Allows the report to be viewed or exported in PDF format.	Pre-condition - A scan has been completed. - Scan results (including passive recon and active crawling) are available. - Vulnerability detection and ML-based prediction has been completed.
Normal Course of Events	
- User navigates to the report section of a completed scan. - User asks for generating a scan summary report. - The System pulls data from passive and active scanning phases. - System retrieves ML based vulnerability predictions (if enabled). - System classifies all discovered vulnerabilities by severity (Critical, High, Medium,Low). - System offers remediation guidance for each vulnerability that is identified. - System shows a summary including: - Passive Recon Info - Discovered Vulnerabilities - Predicted Vulnerabilities (with confidence scores) - Remediation Steps - System allows users to download or export the report in PDF format.	
Alternative Path	
- No vulnerabilities detected - System generates a report indicating "No vulnerabilities detected."	
Conclusion	The system generates a detailed report classifying detected vulnerabilities by severity and providing remediation guidance, even if no vulnerabilities are found.
Post Conditions	
- A report is produced, including all vulnerabilities that is identified. - Report is linked with the scan and included in report history. - The report includes contextual remediation steps under each vulnerability. - User can download or review the report anytime	

Implementation, constraints, and Specifications		
1. The system must support both on-screen viewing and PDF export. 2. ML confidence scores must be shown where applicable. 3. Passive scan sections must be clearly separated and labeled. 4. Reports must be timestamped and linked to the scan instance.		
Use Case Cross References		
Includes	Extends	Exceptions
1. Vulnerability Detection 2. Passive Info Gathering 3. ML-Based Vulnerability Prediction 4. Remediation Suggestion	Scan Execution	1. Report generation fails due to system errors. 2. Incomplete scan data leading to missing report details. 3. ML prediction unavailable then system skips that section and continues. 4. Remediation suggestions fail to load (fallback to general advice)

2.4.16 Report Management

Table 2.43 describes the use case for report management, including its purpose, involved actors, preconditions, main workflow, and expected outcomes.

Table 2.43 Use Case Description-016 Report Management

Use case ID 016	
Use case name Report Management Priority High	Primary Actor User Other Participating Actor None
Use Case Summary The system allows users to view, save, organize, and export scan reports under each project, and maintains historical reports linked to each scan and project.	Pre-condition 1. User has completed a scan and generated a report.
Normal Course of Events	
1. User accesses the Report Management section from the dashboard. 2. System retrieves and displays reports associated with the user's projects. 3. User views available reports and selects actions: • View report content • Rename or organize under folders • Mark as important 4. User clicks "Export Report". 5. System generates a PDF version of the selected report. 6. Exported file is downloaded or saved locally by the user. 7. System maintains versioned and timestamped historical logs for each report.	
Alternative Path	
1. No reports are available for the selected project. 2. System displays a message indicating no reports are available.	

Conclusion	The system allows users to manage, export, and organize scan reports while maintaining historical records linked to projects and scans.	
Post Conditions		
1. Reports are saved, organized, and available for viewing or exporting. 2. Historical reports are linked to their respective scans and projects		
Implementation, constraints, and Specifications		
1. PDF export must preserve original formatting and charts. 2. Reports must be stored securely and versioned automatically. 3. User should only access their own reports (authorization enforced). 4. System should support at least 30 historical reports per project.		
Use Case Cross References		
Includes	Extends	Exceptions
Report Generation, PDF Export Module, Scan History.	View Projects detail	1. Report export fails due to system errors. 2. User encounters issues viewing or organizing reports.

Chapter 3. System Design/Methodology

In this chapter , we discuss the activity diagrams, software architecture diagram, database diagram, dataset details and model selection.

3.1 Activity Diagram

This subsection presents the activity diagrams of the system's modules.

3.1.1 User Registration

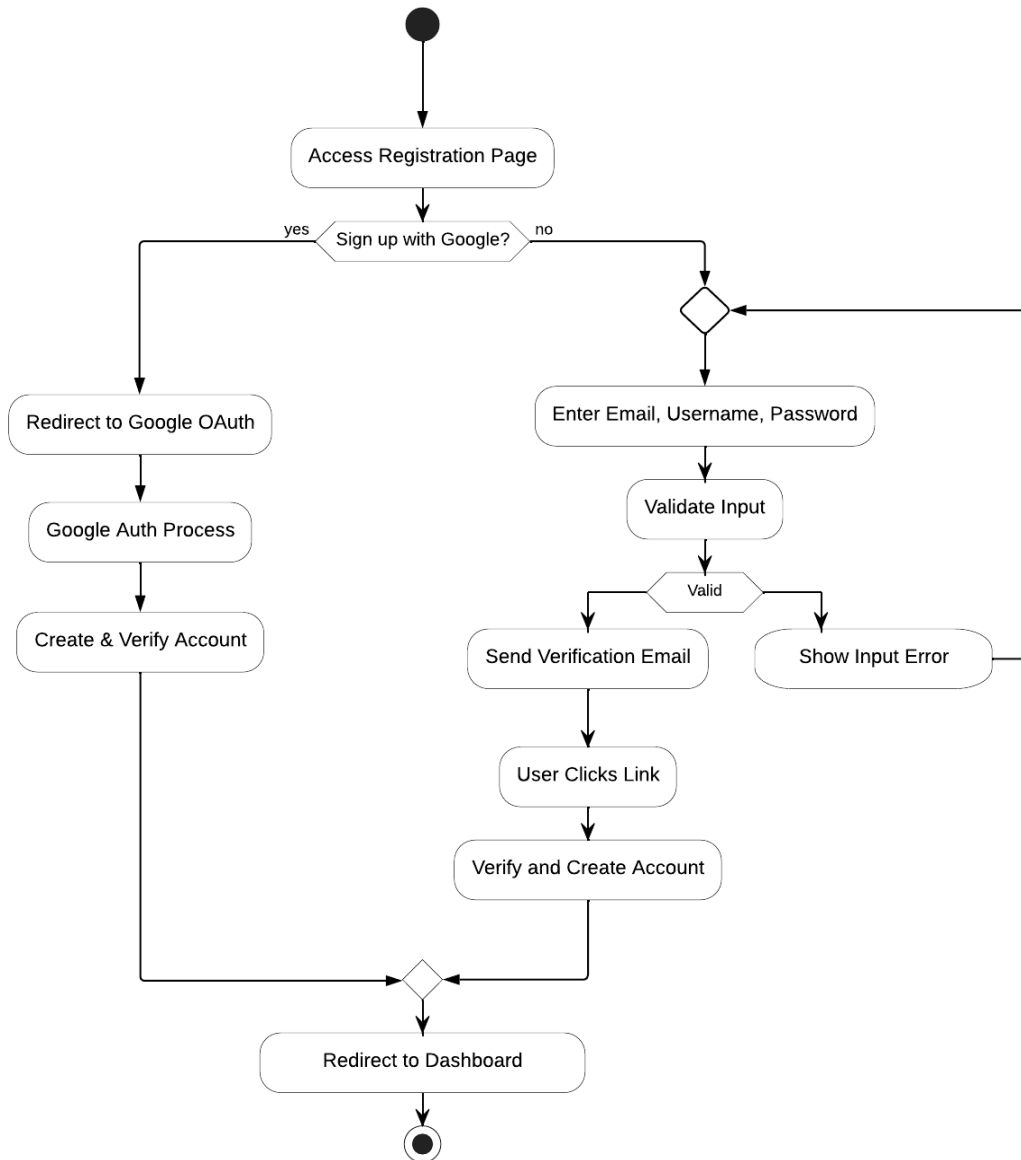


Figure 3.1 Activity Diagram-001 User Registration

Figure 3.1 shows, in detail, the activity flow for the user registration process, wherein a user may register through Google OAuth or by manually creating their credentials. It gives an overview of all activities, such as input validation, verification via email, account creation, and final redirection to the dashboard.

3.1.2 Login

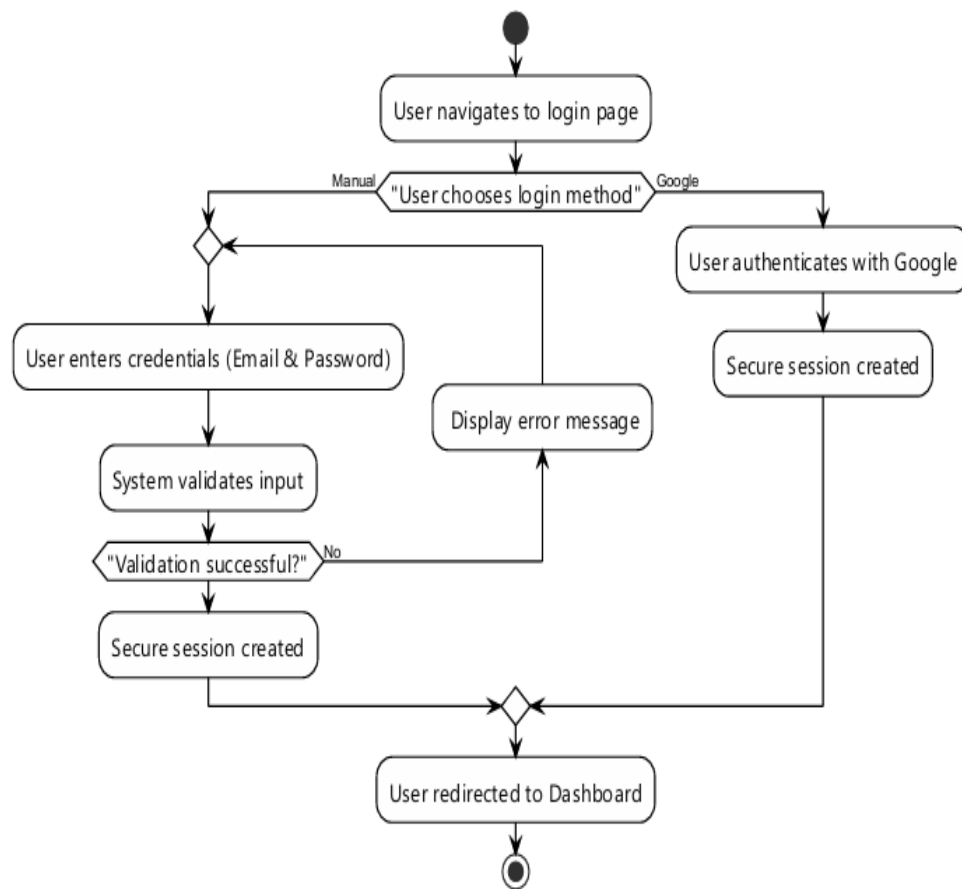


Figure 3.2 Activity Diagram-002 Login

Figure 3.2 shows the user login flow. The user can authenticate using either manual credentials or through Google OAuth. In this diagram, major activities include the validation of credentials, error handling in case of any problem, and the creation of secure sessions for both conditions. Once authentication succeeds, the user is directed to the dashboard.

3.1.3 Logout

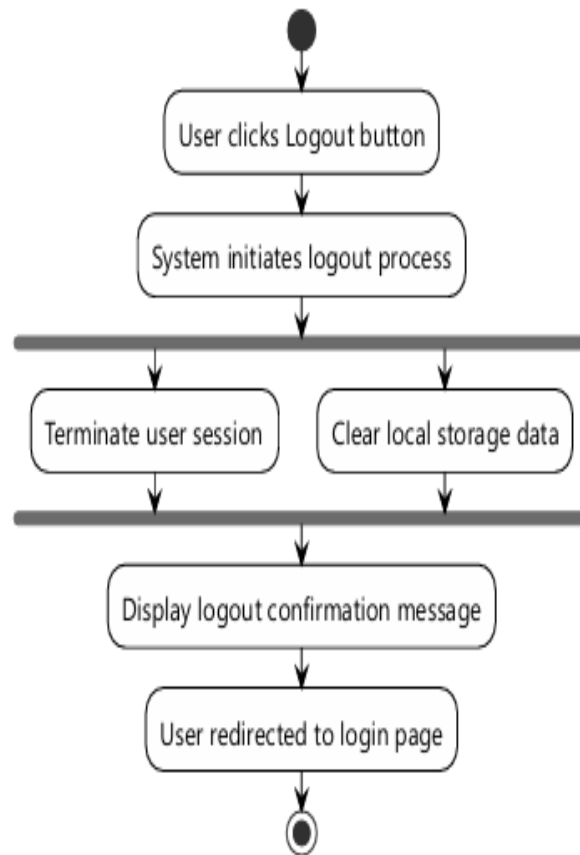


Figure 3.3 Activity Diagram-003 Logout

Figure 3.3 shows the activity flow of the User Logout system. The diagram outlines steps such as terminating the active session and clear local storage data. After those above actions, the user is securely redirected back to the login page.

3.1.4 Password Recovery

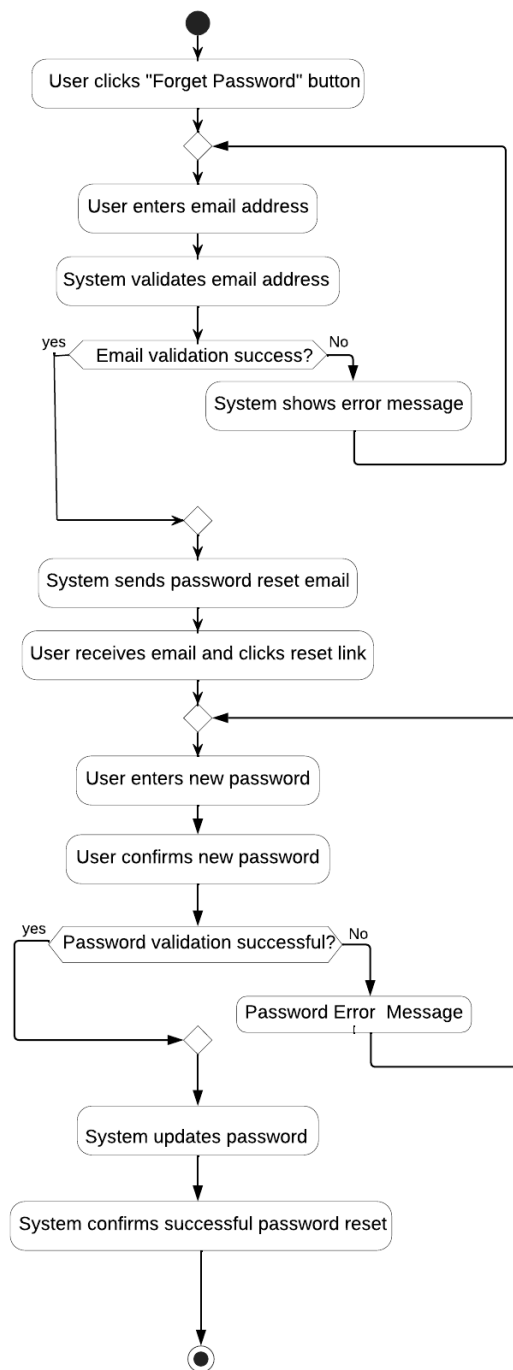


Figure 3.4 Activity Diagram-004 Password recovery

Figure 3.4 shows the activity diagram that describes the Forgot Password process. A user submits their email for verification; after that, a reset link is sent to the email. The user opens the link and sets up a new password, confirming it. Afterwards, the system verifies the newly setup password. Once verified, it updates the password and confirms successful password resetting.

3.1.5 Profile Management

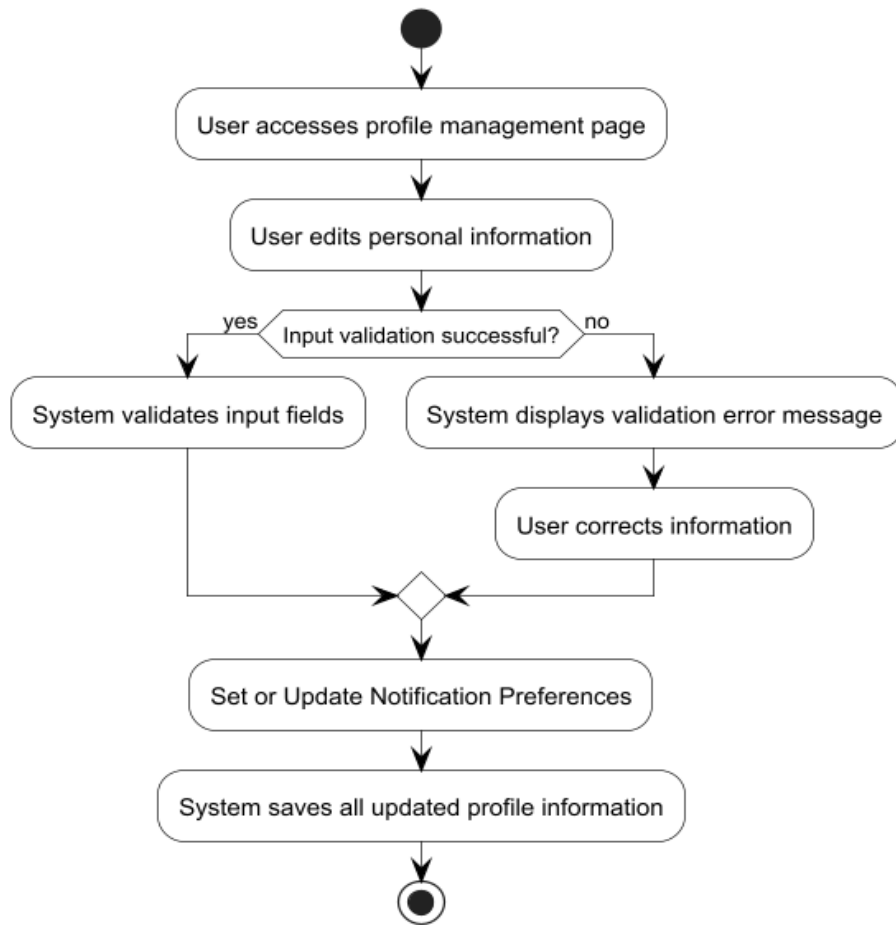


Figure 3.5 Activity Diagram-005 Profile Management

Figure 3.5 shows the activity diagram that depicts the activity of updating one's profile by editing personal information that needs to be validated by the system. For any incorrect input, the user may proceed after correcting it. Having validated the entry, the user selects notification preferences before the system saves all updated details of the profile.

3.1.6 View Projects

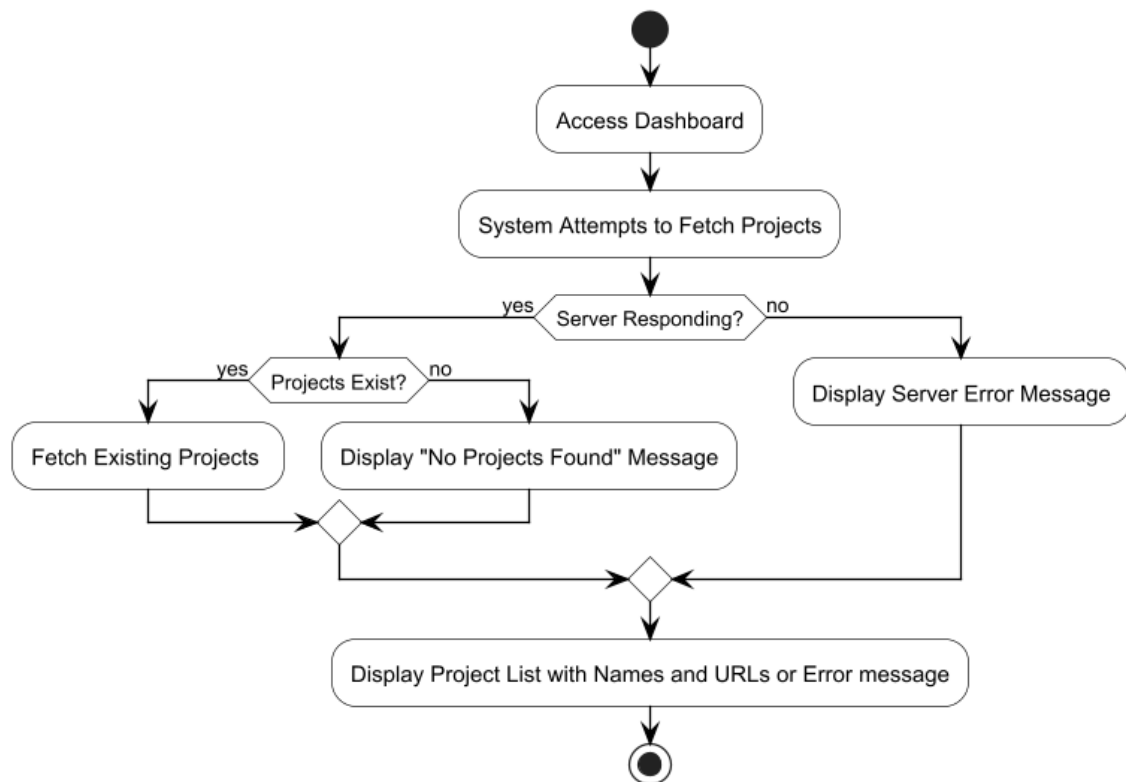


Figure 3.6 Activity Diagram-006 View Projects

Figure 3.6 shows the activity diagram that illustrates how the dashboard loads projects by checking the server's response and fetching available data. If there are projects, the system shows them; if not, a message is displayed. In case of a server failure, an error is shown, and the system outputs either the list of projects or an error message.

3.1.7 Add project

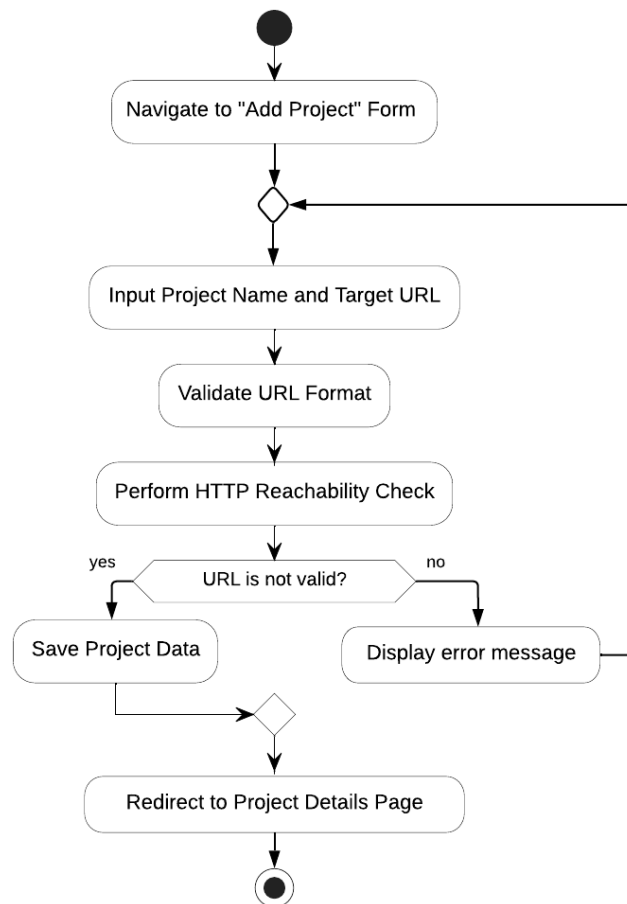


Figure 3.7 Activity Diagram-007 Add project

Figure 3.7 depicts the adding of a new project. The user navigates to the "Add Project" form and fills in the details. The system verifies the correctness of the format of the URL, whether it is reachable, and if the input has the appropriate format. If valid, the project gets saved, and the user gets sent back to the project details page; otherwise, an error message appears, and the user is asked to correct the input.

3.1.8 Open Project

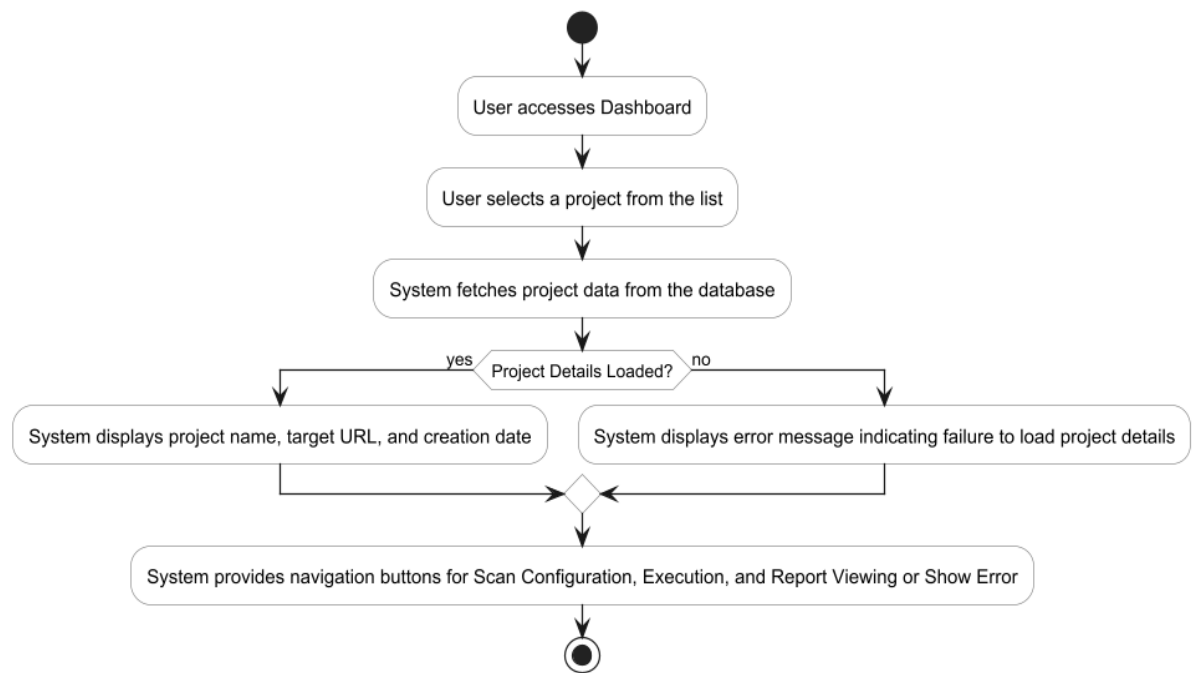


Figure 3.8 Activity Diagram-008 Open Project

Figure 3.8 depicts the view project details use case, which begins when the user enters the dashboard and selects any of the projects appearing in the list. The system retrieves information about the project and then renders the information of the project or an error screen in case of a loading failure. Finally, the system takes the user to the interface to configure, execute, and view the report or the respective error interfaces.

3.1.9 Target Configuration

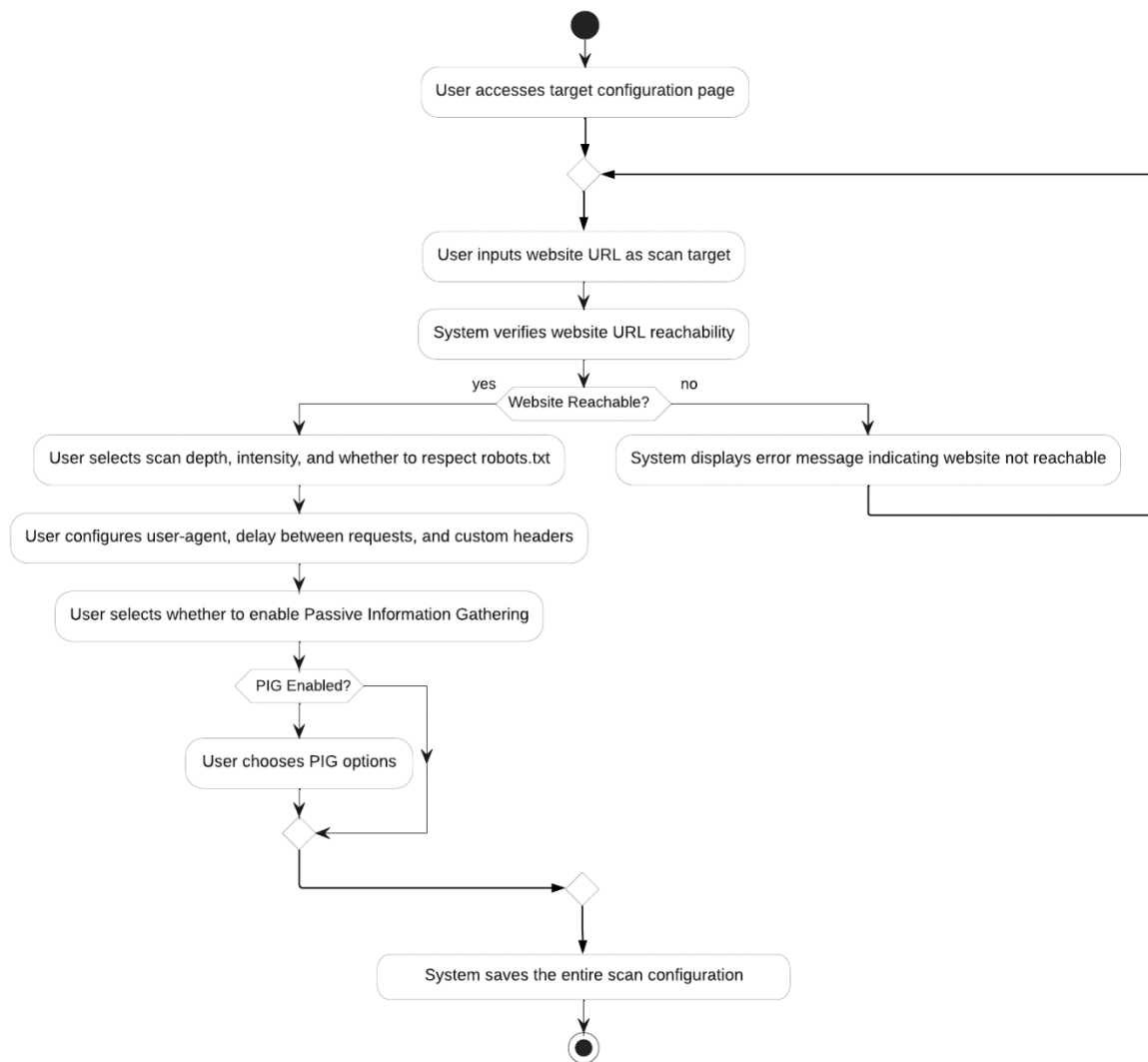


Figure 3.9 Activity Diagram-009 Target Configuration

Figure 3.9 depicts the scan configuration flow, which begins when a user enters the configuration page with a target URL to be validated. After confirmation that the website is reachable, the user defines scan parameters, configures request settings, and may enable Passive Information Gathering. In case all the choices have been made, the system saves the finalized scan configuration or shows an error if the target is not reachable.

3.1.10 Passive Information Gathering

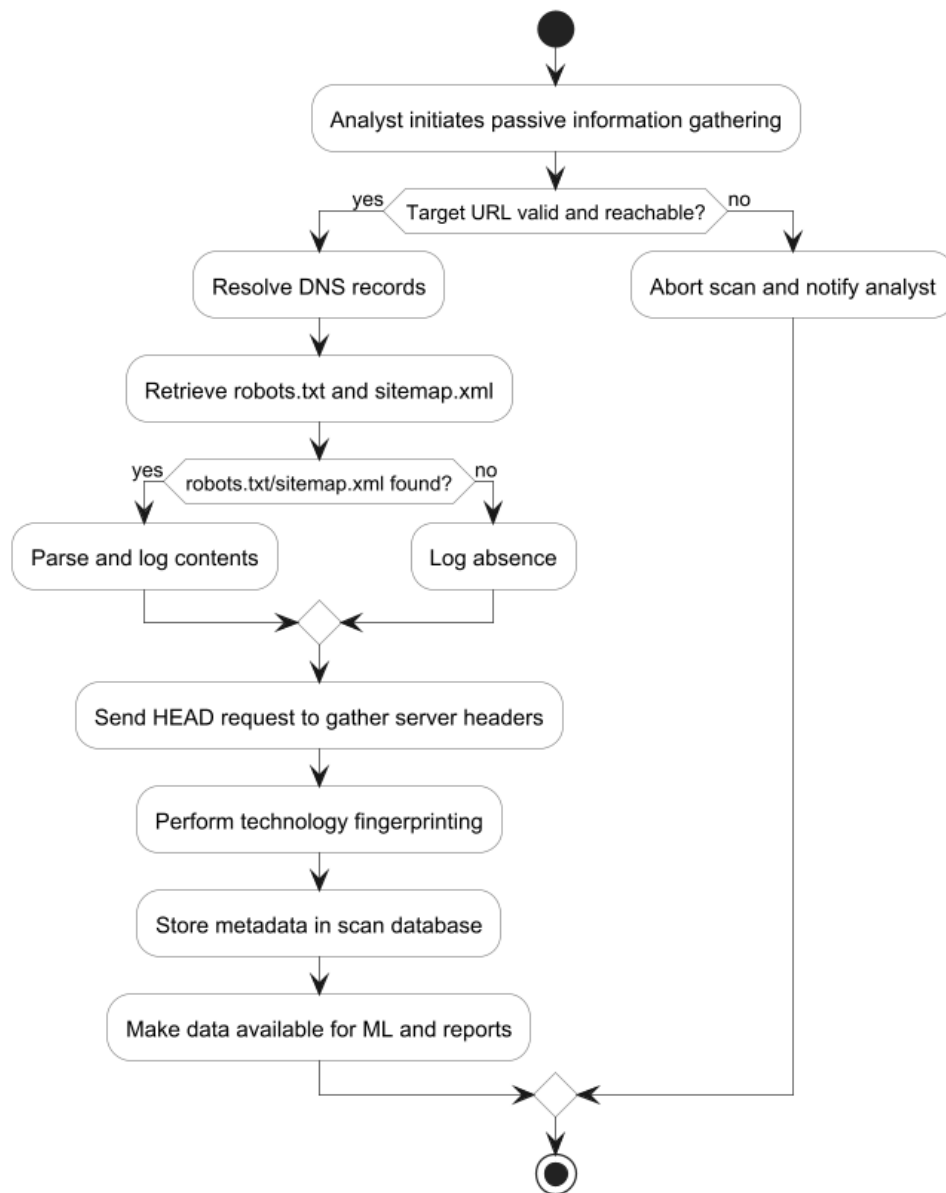


Figure 3.10 Activity Diagram-010 Passive Information Gathering

Figure 3.10 depicts the Passive information gathering workflow, passive information gathering workflow starts by an analyst request, where the system first checks if the target URL is reachable. If the target URL is valid, the system subsequently resolves DNS records and pulls discovery files to scrape or log their contents and capture the server headers for technology fingerprinting. The scraped metadata will be persisted in the Scan database for machine-learning analysis and reporting, while invalid URLs will terminate the scan and notify the analyst.

3.1.11 Scan Execution

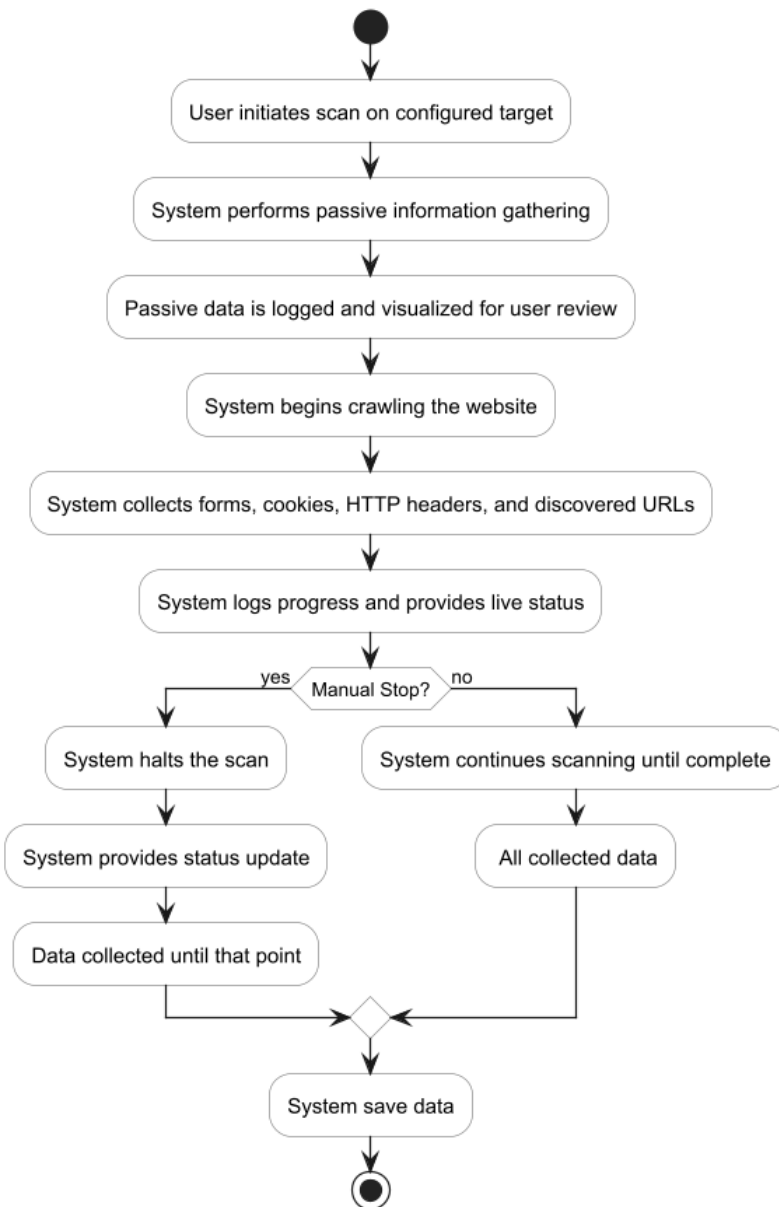


Figure 3.11 Activity Diagram-011 Passive Information Gathering

Figure 3.11 shows the active scan workflow, which starts when the user launches a scan against the configured target, and the system begins with passive information gathering. While it is gathering forms, cookies, headers, and URLs that have been discovered, it logs all events and status changes live. The scan then proceeds to either completion or manual termination. Upon doing so, the system saves all collected data for further analysis and reporting.

3.1.12 Vulnerability Analysis and Prediction

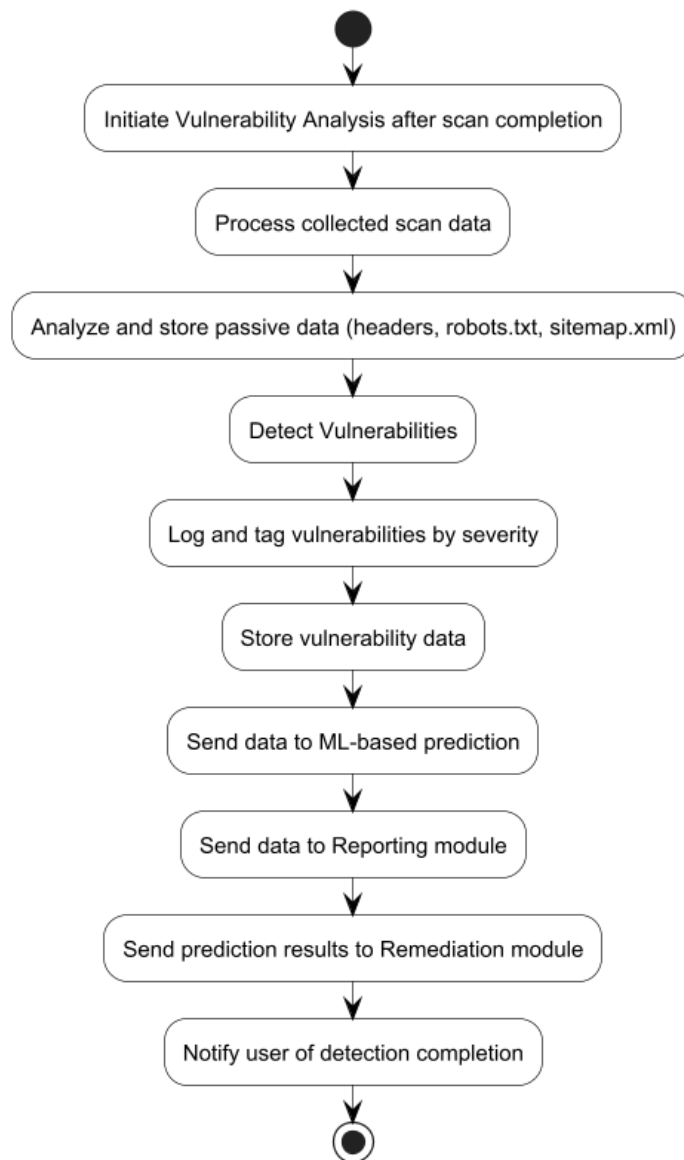


Figure 3.12 Activity Diagram-012 Vulnerability analysis and prediction

Figure 3.12 shows the vulnerability analysis workflow, starting directly after the scan, when the system starts processing the data it has collected and analyzing passive information. It then identifies vulnerabilities, adds severity tags, saves the results, and pushes relevant data to the machine-learning prediction and reporting modules. The prediction output is then fed into the remediation module, with notification to the user that vulnerability detection has completed.

3.1.13 Data Collection and Model Training

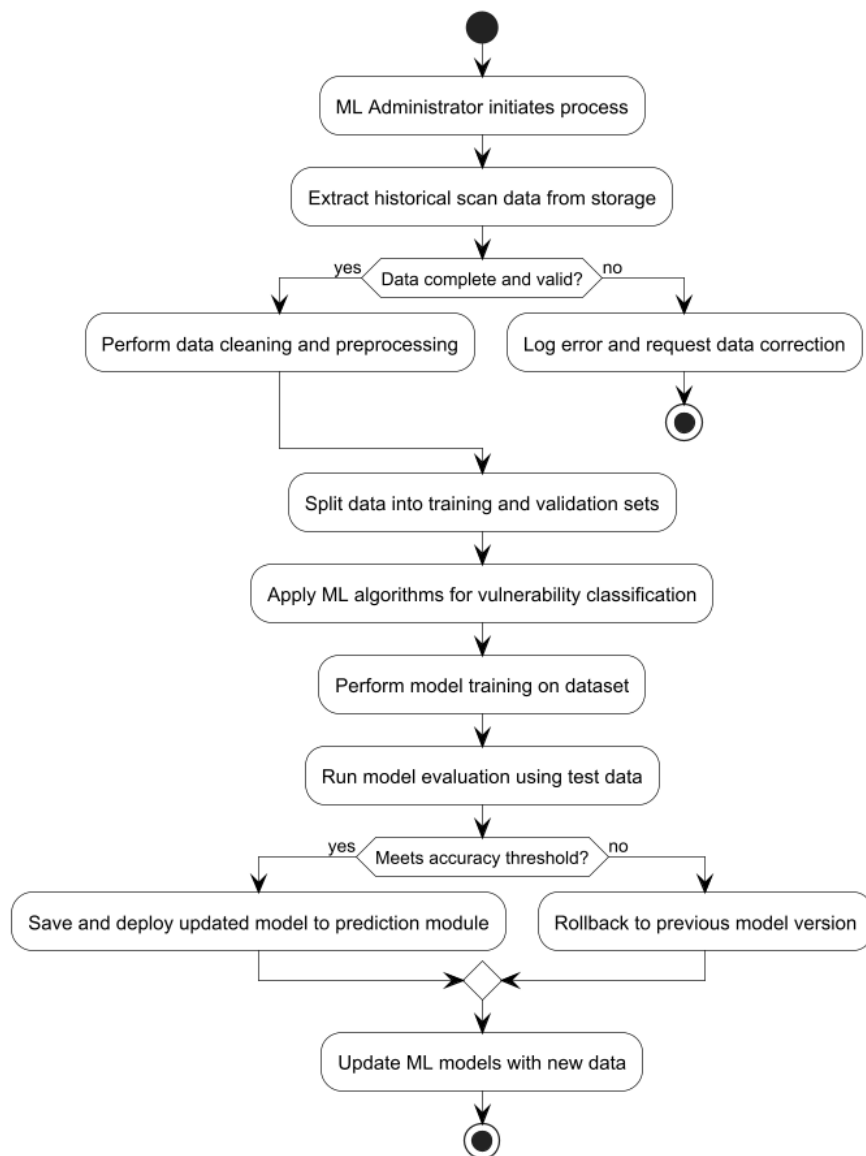


Figure 3.13 Activity Diagram-013 Data Collection and Model Training

Figure 3.13 shows the different steps in machine learning model updates, starting with the ML administrator's initiation of the workflow and retrieval of historical scan data for validation. In cases where the data is complete and valid, the system will perform cleaning, preparation of training and validation sets, classification algorithm training, and model performance evaluation against accuracy thresholds. Depending on the results, the model update is either deployed or rolled back, and then refreshes the ML models with newly processed data.

3.1.14 Remediation Suggestion Based on ML Prediction

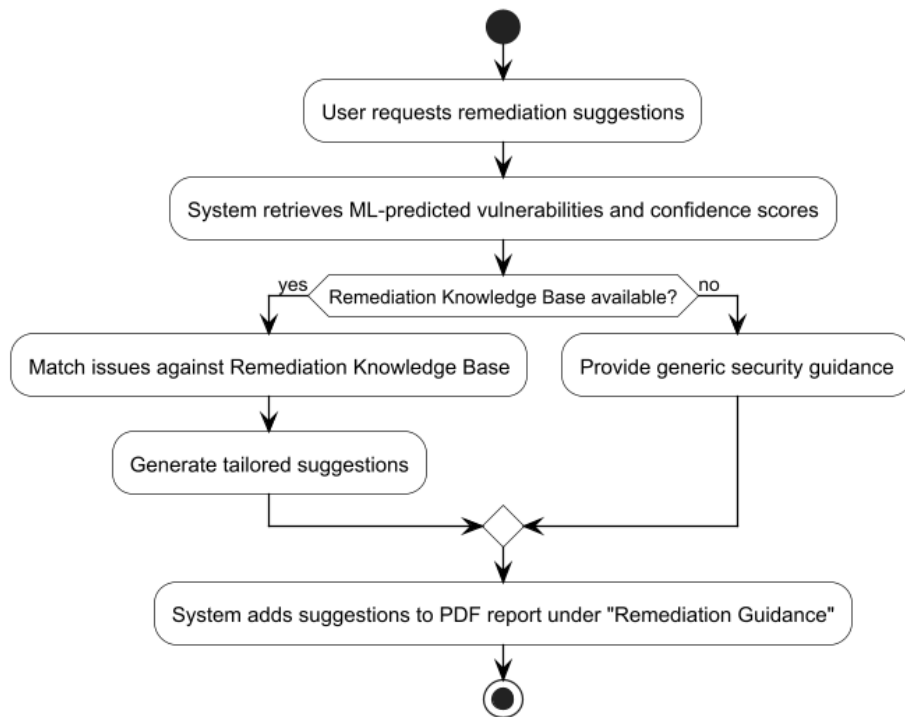


Figure 3.14 Activity Diagram-014 Remediation Suggestion Based on ML Prediction

Figure 3.14 shows the remediation suggestion workflow, which starts when a user requests guidance. It fetches ML-predicted vulnerabilities and their confidence scores. The system then uses either an available remediation knowledge base to draft customized recommendations or offers general security suggestions based on the information present. The outcome of these suggestions is then aggregated and added in the final PDF report under the “Remediation Guidance” section.

3.1.15 Report Generation

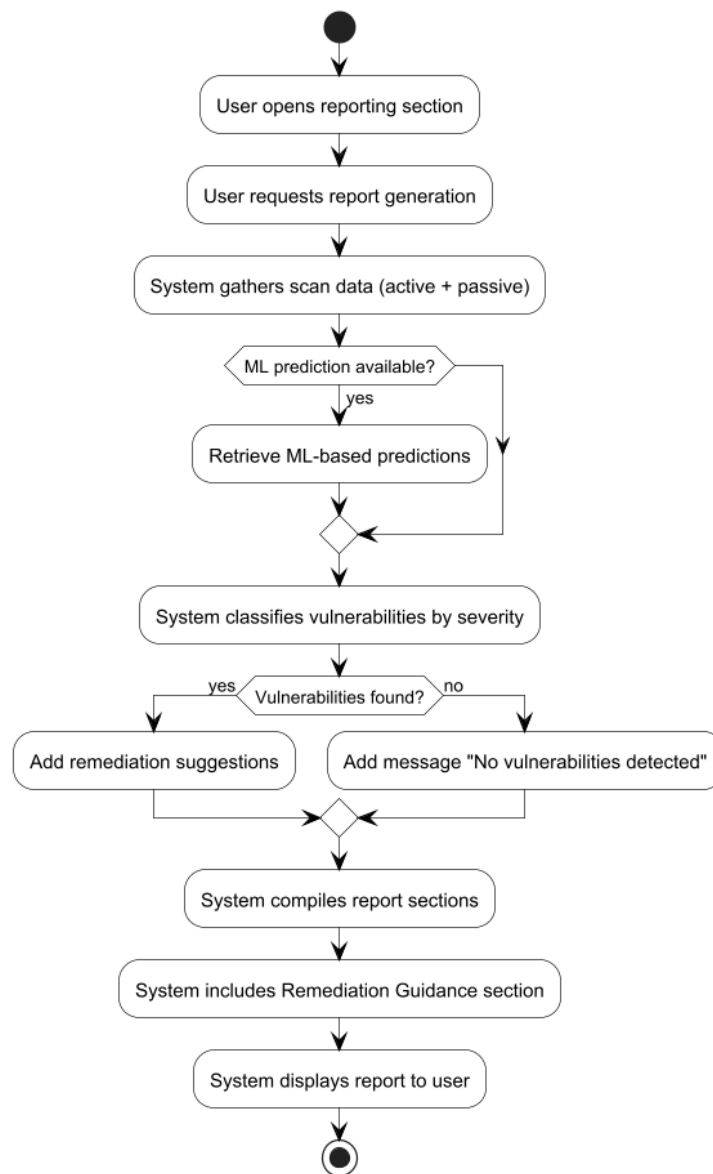


Figure 3.15 Activity Diagram-015 Report Generation

Figure 3.15 Activity Diagram on the workflow of an automated Vulnerability Reporting System. The process starts when a user opens the reporting section and requests report generation. Then, the system collects scan data and pulls out ML-based predictions if any exist, along with the severity classification of the vulnerabilities. Depending on whether any vulnerabilities have been identified or not, two options are shown, namely adding remediation suggestions and displaying the "No vulnerabilities detected" message. Finally, it will compile all the sections, including remediation guidance, and will show the final report to the user.

3.1.16 Report Management

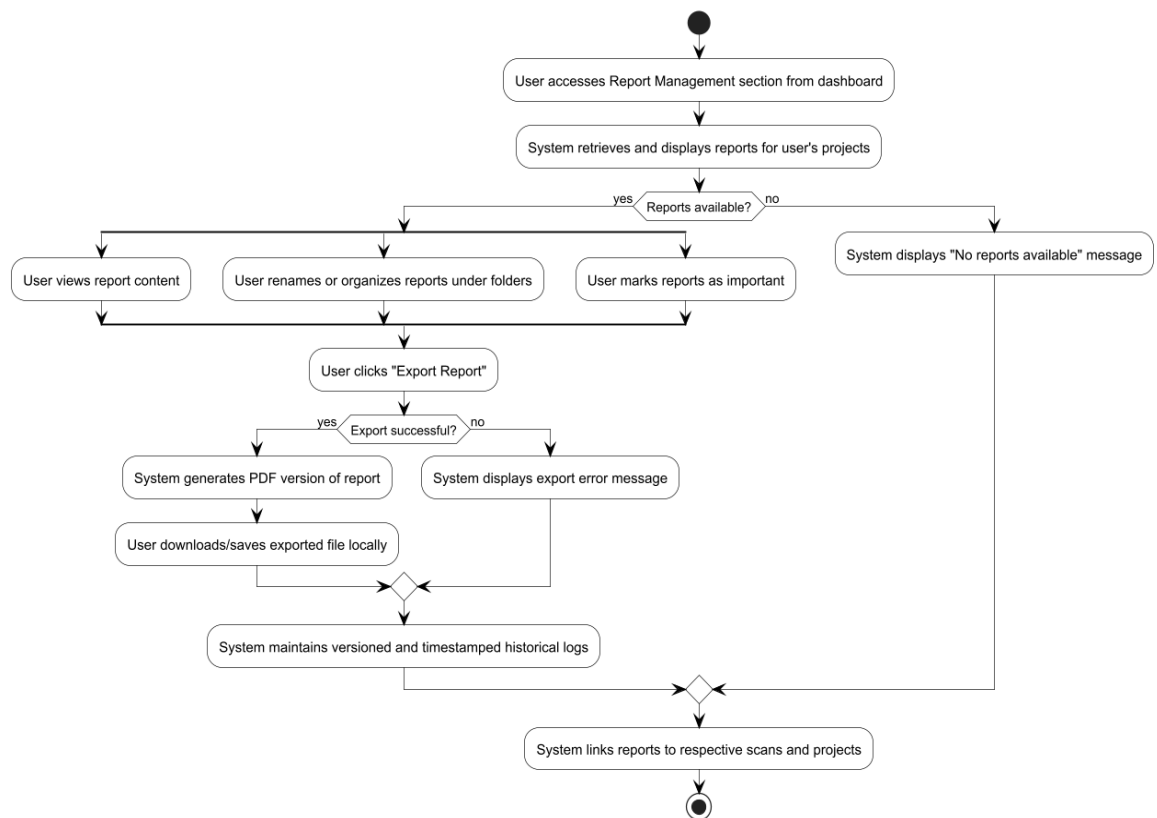


Figure 3.16 Activity Diagram-016 Report Generation

Figure 3.16 shows the Activity diagram for the Report management system that allows users to access, organize, and export vulnerability reports from their dashboard. The workflow starts when a user accesses the Report Management section. The system retrieves available reports for the user's projects and displays them. If there are available reports, options to view the content, rename, or set reports in folders and mark them as important are displayed; otherwise, a "No reports available" message is displayed. Upon receipt of an "Export Report" click, the system will try to generate a PDF version for local download if it's successful, with an export error message if it's unsuccessful. It will maintain versioned, timestamped historical logs of all changes and link reports to their respective scans and projects.

3.2 Software Architecture Diagram

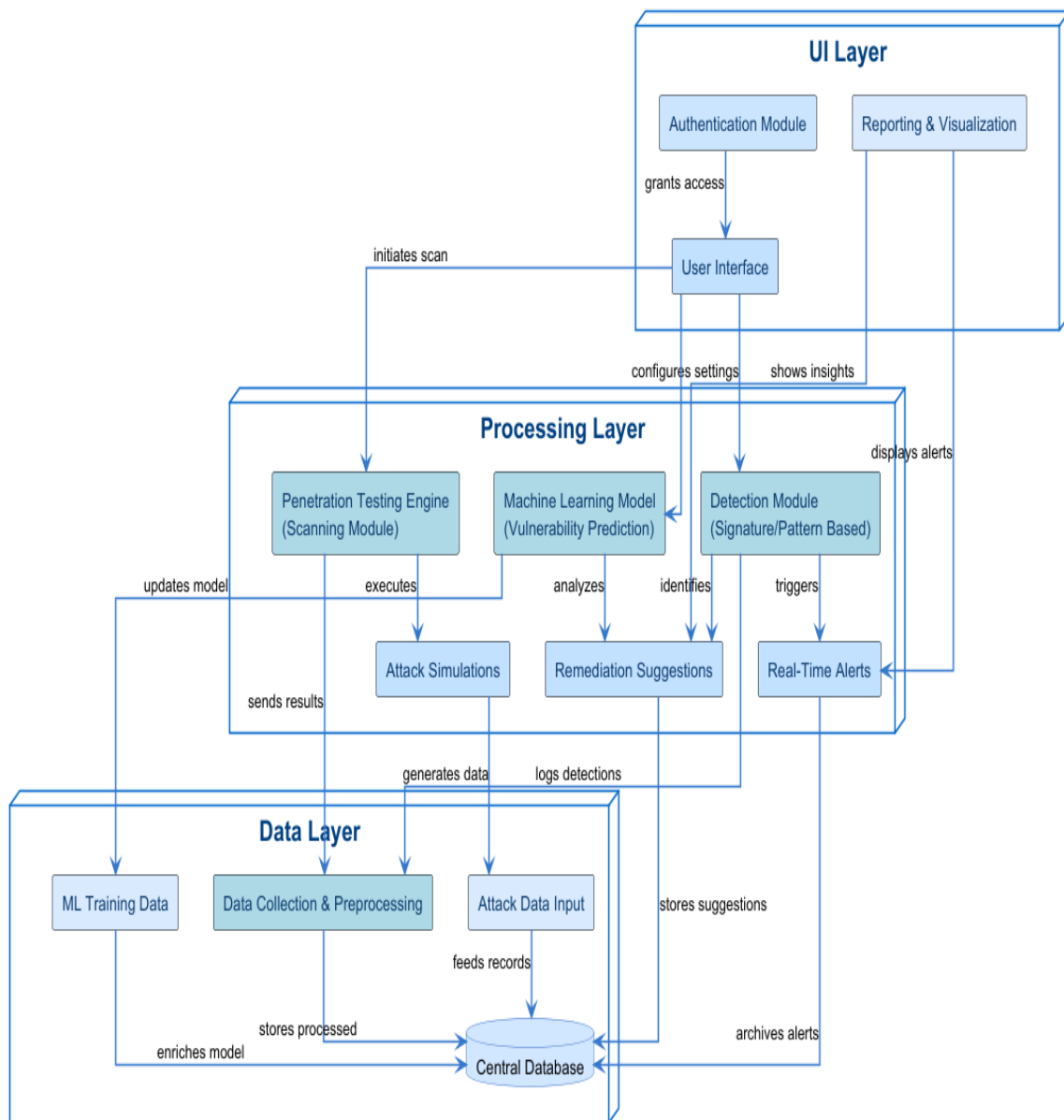


Figure 3.17 Software Architecture Diagram

Figure 3.17 shows the System Architecture Diagram with three layers: UI Layer, Processing Layer, and Data Layer. The UI Layer is responsible for user interactions and initiation of scans. The Processing Layer is responsible for scanning, vulnerability prediction through ML, and real time alerts. The Data Layer is responsible for training data, preprocessing, and centralized storage of all scan and detection results.

3.3 Database Diagram

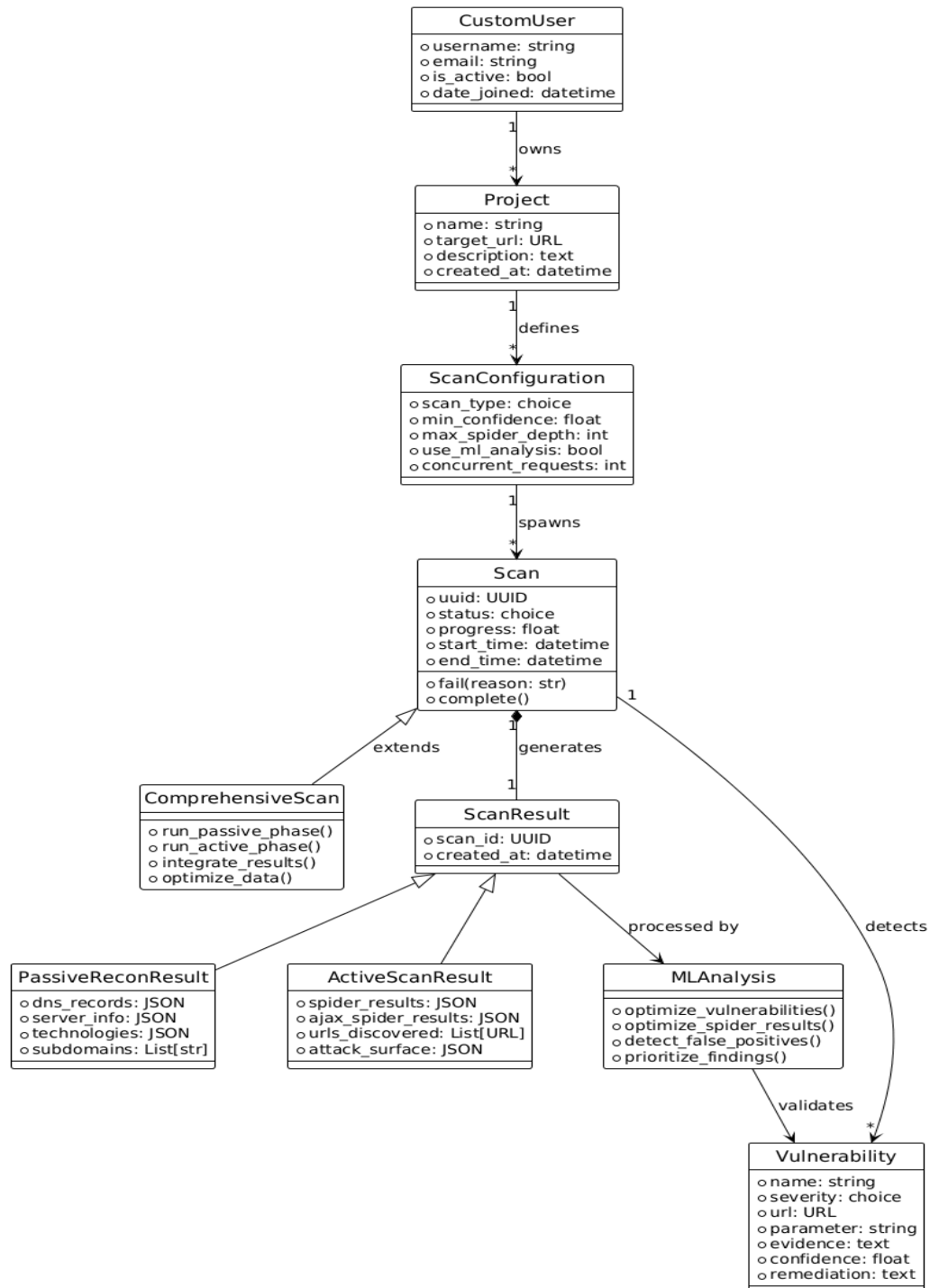


Figure 3.18 Database Diagram

Figure 3.18 shows the Entity Relationship Diagram for the Automated Web Application Penetration Testing system. From the database schema, it is evident how users are related to projects, projects to scans, scans to results, and how vulnerability identification is linked to machine learning models. It is also clear from the database schema that the attributes of the entities have been thoroughly

identified along with the corresponding data types. This database design enables the functionality of penetration testing, management of users, vulnerability identification, and generation of reports..

3.4 Dataset Details

In this subsection ,the description of the dataset on which this research was based will be presented: its source, the procedure of creation, its size and structure, and the preprocessing steps taken.

3.4.1 Dataset Source

The dataset is created from vulnerability scan data collected in a variety of environments that range from development environments to vulnerable applications and productive web applications on the internet. The data collected are produced by web application vulnerability scan software producing a list of identified issues along with additional metadata as a result of each scan. In addition, gathering data from scan environments that were productive and simulation environments means that a valid scenario of vulnerabilities is included along with false positives in the dataset.

3.4.2 Dataset Creation Process

The dataset creation process include the following steps outlined below:

- **Collection of Vulnerability Records:** Overall, the vulnerability findings were collected from several scan databases across various environments of the application. Each record represents one single finding of a vulnerability issue reported by the scanner.
- **Dataset Unification:** Records collected were merged together into a single dataset, which was stored in CSV format. It unified the representation of vulnerability attributes across various scans and projects.
- **Handling Missing Values:** Many attributes, like request parameters, evidence, and classification identifiers, had missing values since the vulnerabilities which got detected had contextual differences. These were standardized with a uniform placeholder, such as Not Applicable, in order to maintain consistency and avoid ambiguities during analysis.
- **Save and Remove Duplicates:** Duplicated records were identified and removed according to complete row duplication. In this step, redundant scan results were removed, which may cause bias during the statistical analysis and model training.
- **Feature Engineering Preparation:** Additional domain-specific features were derived from raw data after cleaning to support machine learning analysis. Features capture scanner behavior and vulnerability context rather than the raw vulnerability content.

3.4.3 Dataset Size and Structure

After preprocessing, the final dataset consists of over 100,000 vulnerability records, where each record representing a single scanner finding. The dataset represents various categorical and numerical fields.

Key attributes of dataset are:

- Vulnerability name
- Severity level
- Target URL
- Evidence and parameter information
- Classification identifiers (CWE, WASC)
- Engineered features used for analysis

3.4.4 Dataset Cleaning Summary

The following dataset cleaning steps were applied to the dataset:

- Standardization of missing values
- Removal of duplicate records
- Normalization of text fields
- Preparation of structured features for machine learning

These steps ensured data quality, consistency, and suitability for subsequent analysis.

3.5 Algorithm/Model Selection

An explanation of the reasoning related to the machine learning approach selected based on the objective to reduce the number of false positives in vulnerability scan results.

3.5.1 Problem Framing

False positives in vulnerability scans are not considered a traditional classification problem since there are no ground truth labels. Vulnerabilities for labeling by hand require security expertise and are not scalable. As such, the problem statement was formulated to be a confidence estimation problem whereby the objective was to estimate the likelihood that the scanner indicated vulnerability has been called a false positive.

3.5.2 Model Selection Rationale

A **Random Forest** model was selected for this task due to following reasons:

- It performs well with noisy and weakly labeled data
- It can learn non-linear decision boundaries
- It is robust against overfitting through ensemble learning
- It produces probabilistic outputs suitable for confidence-based filtering

These properties make Random Forests well-suited for modeling scanner behavior patterns rather than strict vulnerability correctness.

3.5.3 Feature-Based Learning Approach

The model relies on domain driven features that reflect scanner behavior and context, instead of using raw vulnerability content. Examples include indicators of evidence availability, vulnerability category (header-based or injection-based), and deployment environment. This feature-based approach allows the model to learn patterns associated with false positives while remaining independent of specific application implementations.

3.5.4 Threshold-Based Decision Strategy

For every vulnerability, the model generates a false-positive confidence score. To identify probable false positives, this score is compared to a configurable threshold. Without permanently eliminating vulnerabilities from the scan results, this technique enables flexible tailoring of alarm suppression.

Chapter 4. Implementation

In this Chapter, we discuss about the work breakdown structure, team roles and responsibilities, tools and technologies, implementation details and challenges during implementation.

4.1 Work Breakdown Structure (WBS)

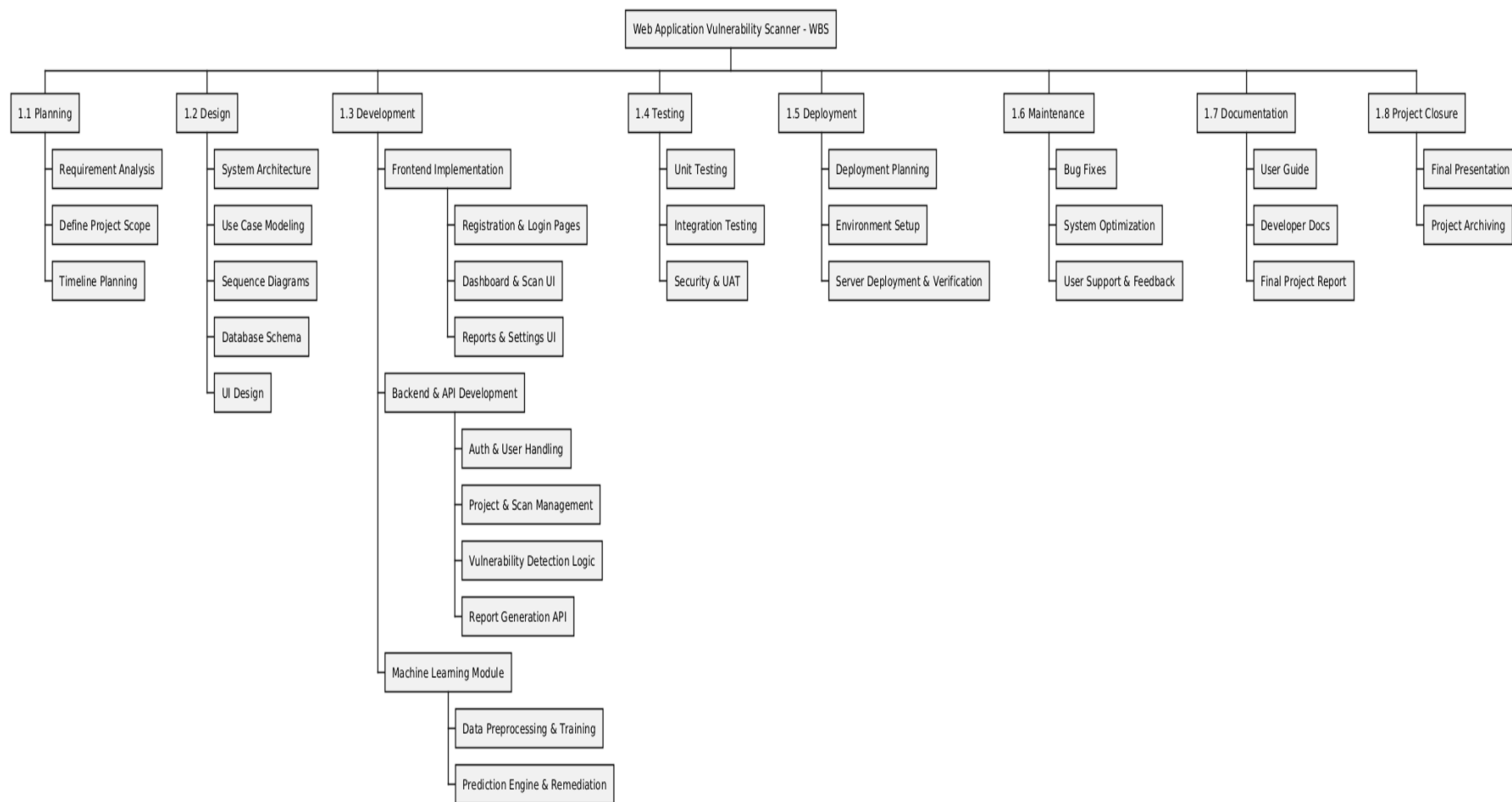


Figure 4.1 Work Breakdown Structure

Figure 4.1 illustrates the hierarchical decomposition of the Web Application Vulnerability Scanner project into major phases, including planning, design, development, testing, deployment, maintenance & support, documentation, and project closure, along with their associated tasks and sub-task

4.2 Team Roles and Responsibilities

Table 4.1 presents a comprehensive breakdown of individual tasks involved in the development and implementation of the Automated Web Vulnerability Scanner.

Table 4.1. Individual Tasks

Team Member	Activity
Hannan Ali , Ahmad Naeem , Mujtaba Kamil Mehmood	Project Planning and Requirements Analysis
Ahmad Naeem	User Registration and Authentication Design
Mujtaba Kamil Mehmood	UI/UX Design for Security Interface
Hannan Ali , Ahmad Naeem	Project Management Module
Mujtaba Kamil Mehmood	Scan Configuration Module
Hannan Ali	Passive Information Gathering Module
Hannan Ali	Active Scanning Engine Development
Hannan Ali , Ahmad Naeem	Data Collection and Preprocessing
Hannan Ali	ML Model training and Testing
Hannan Ali	Comprehensive Scan with ML
Ahmad Naeem , Mujtaba Kamil Mehmood	Report Generation Module
Hannan Ali , Ahmad Naeem , Mujtaba Kamil Mehmood	Documentation
Ahmad Naeem , Mujtaba Kamil Mehmood	System Testing
Hannan Ali , Ahmad Naeem	System Deployment
Hannan Ali , Ahmad Naeem , Mujtaba Kamil Mehmood	Future Implementation Goals

Activities	Semester 7			Summer Break			Semester 8			
	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec
Requirement Analysis and Planning										
Database Schema Design and Setup										
User Interface Design										
Authentication and User Role Integration										
Project and Scan Management Module										
Vulnerability Detection and ML Prediction										
Passive Information Gathering										
Scanning Engine Development										
Remediation Suggestions & Alert System										
Report Generation Module										
Testing, Logging & Evaluation										
Final Documentation and Presentation										

Figure 4.2 Gantt Chart

Figure 4.2 shows the Gantt chart displaying the planned sequence and duration of key development activities. The chart shows how foundational work including requirements analysis, database design, and user interface development precedes the technical implementation of authentication, scan management, and vulnerability detection modules. The middle phases focus on intelligence components including passive information gathering, ML prediction, and scanning engine development. The project concludes with remediation suggestions, reporting functionality, and comprehensive testing. Documentation activities run parallel to technical development throughout the project.

4.3 Tools and Technologies

The software, frameworks, programming languages, libraries, platforms, and hardware chosen to construct the Web App Scanner are listed in this section. The project makes use of a contemporary, containerized architecture that divides the heavy scanning engines, frontend details, and backend functionality.

4.3.1 Core Technologies

- **Programming Languages Python (v3.10+):** The primary language for the backend API and scanning logic, chosen for its rich ecosystem of security and data analysis libraries.
- **TypeScript:** Used for the frontend to ensure type safety and scalable code maintenance.
- **Backend Framework Django (v5.2+):** A robust, secure implementation of the web framework, handling API requests, database interactions, and authentication (via django-allauth and dj-rest-auth).
- **Django REST Framework (DRF):** For assembling the RESTful API endpoints consumed by the frontend.
- **Frontend Framework Next.js (v16):** A React based framework used for server side rendering, static site generation, and optimized performance.
- **React (v19):** The library for building interactive user interfaces.
- **Tailwind CSS (v4):** A utility-first CSS framework for rapid and responsive UI design.

4.3.2 Database & Storage

- **Primary Database:** SQLite (Development) is used for storing user data, scan configurations, and results.
- **Object Storage:** Django Storages is used for controlling and managing file uploads and scan logs.

4.3.3 Security & Scanning Tools

The security scanning system integrates industry standard security tools via a custom adapter pattern:

- **OWASP ZAP (Zed Attack Proxy):** The core engine for active vulnerability scanning and active/passive crawling.
- **SSLyze:** For analyzing SSL/TLS configuration of target servers.
- **Nuclei:** Integrated for template-based vulnerability scanning.
- **Python-dotenv:** For managing environment variables and secrets.

4.3.4 DevOps & Deployment

- **Docker & Docker Compose:** Used to containerize the application services (Backend, Frontend, ZAP, Database) ensuring compatible environments across development and production.
- **Gunicorn:** WSGI HTTP Server for serving the Django application in production.
- **WhiteNoise:** For serving static files efficiently.

4.3.5 Data Analysis & Utilities

- **Scikit-learn, Pandas, NumPy:** Used for scan results processing , deduplication, and false positive reduction.

4.4 Implementation Details

This section describes how the chosen tools make up the functional Web Application Scanner System. The architecture is based on a microservices inspired approach in which specialized containers for security functions are arranged by the primary backend.

4.4.1 System Architecture

There are four primary containers make up the application's coherent system:

- **Frontend Service:** This is the Next.js app responsible for the dashboard service. This service interacts with the backend using RESTful API protected by JWT token authentication.
- **Backend API:** This is a Django app that has functionality for scan related operations, user authentication, and reporting. It serves as a central coordinator.
- **Scanning Engine (ZAP + Discovery):** A special container that runs OWASP ZAP as a daemon (-daemon) listening on port 8080. The backend communicates with it via ZAP API to activate spiders and active scanning.
- **Database:** A SQLite instance storing relational data including users, projects, scan profiles, and vulnerabilities.

4.4.2 Scanning Logic Implementation

The core intelligence resides in the backend/scanning module, centered around the UnifiedScanningEngine class.

The Unified Engine

The Unified_Scanning_Engine is the orchestrator class responsible for managing the entire scan lifecycle. It supports three scans:

- **Passive Scan:** Performs reconnaissance without directly attacking the target url.
- **Active Scan:** Aggressively probes the target for vulnerabilities.
- **Comprehensive Scan:** A two phase approach combining passive scanning seamlessly with active scanning and uses machine learning for reducing the false positive.

The Adapter Pattern

The system relies on an Adapter Pattern, which is contained within "backend/scanning/integrations," to ensure that it maintains its modularity. For example, a particular adapter class is designated for each external software tool, which could be ZAP, SSLyze, or Nuclei, namely ZAPActiveAdapter, SSLyzeAdapter, and so on. The engine allows these adapters to be accessed through an interface..

Scan Workflow

- **Initialization:** When a user initiates a scan, the UnifiedScanningEngine initializes and registers itself with the ScanTracker.
- **Reconnaissance (Passive Phase):** The engine triggers PassiveScanningEngine, which uses adapters to run DNS checks, active/passive discovery, and technology detection. Results are saved to PassiveReconResult.
- **Integration:** The URLs, subdomains, and API endpoints that were found in the passive phase are now seeded into the ZAP context. This makes sure that the active scanner hits the hidden assets identified during the recon phase.
- **Vulnerability Testing (Active Phase):** It launches the Active Scanning Engine. It tells the container to spider and attack the seeded URLs. The progress status, which goes from 0 to 100%, is monitored in real-time and stored in the database.
- **Result Normalization:** Raw results from all tools are parsed and normalized into a standard Vulnerability model. Deduplication logic ensures the same vulnerability isn't reported multiple times. Spider data is optimized (large response bodies removed) to keep the database lightweight while retaining discovery data.

4.4.3 Data Flow & Storage

- **Input:** Users provide a Target URL and Scan Configuration via the frontend.
- **Processing:** The backend spawns a thread/task for UnifiedScanningEngine. This engine makes HTTP API calls to the ZAP container and CLI calls to other tools.
- **Output:** Real-time progress is polled by the frontend. Final results are aggregated into ScanLog, ActiveScanResult, and PassiveReconResult tables.

4.4.4 Pseudocode

In this subsection, we basically describe pseudocode of our main modules.

Unified Scanning Engine (Orchestrator)

This module acts as the central controller, checking the configuration and routing the scan to the appropriate engine.

python

Class UnifiedScanningEngine:

Function Start(scan_id):

Initialize scan inputs (target_url, configuration)

Register scan with ScanTracker

Try:

If configuration.scan_type == "passive":

Run Passive_Scan_Module()

Else If configuration.scan_type == "active":

Run Active_Scan_Module()

Else If configuration.scan_type == "comprehensive":

Step 1: Run Passive Phase

success_passive = Run Passive_Scan_Module()

Step 2: Run Active Phase (using passive results)

success_active = Run Active_Scan_Module(input=passive_results)

Step 3: Integrate Results

Integrate_Comprehensive_Results()

Catch Error:

Log error

Mark scan status as "FAILED"

Finally:

Stop all running tools (ZAP, etc.)

Unregister scan

Save final logs

Passive Scanner (Reconnaissance)

This module gathers information without attacking the target. It runs sequentially through multiple analysis phases.

python

Class PassiveScanner:

Function Run_Scan(target_url):

Initialize results_container

Phase 1: Basic Info

Analyze_Target_URL(target_url)

Phase 2: Infrastructure

DNS_Results = Analyze_DNS(domain)

SSL_Results = Analyze_SSL_Certificate(domain, port)

Phase 3: Stack Detection

Tech_Results = Detect_Technologies(target_url, headers, html_content)

(Identifies CMS, Frameworks, Servers, etc.)

Phase 4: Security Configuration

Header_Results = Analyze_Security-Headers(response_headers)

Cookie_Results = Analyze_Cookies(response_cookies)

Phase 5: Enhanced Discovery

Discovery_Results = Run_External_Tools(Subfinder, Waybackurls)

Aggregation

Vulnerabilities = Aggregate_Findings(SSL_Results, Header_Results, Cookie_Results)

Save_Results_To_Database(dns, ssl, tech, headers, vulnerabilities)

Return Results

Active Scanning Engine (Vulnerability Testing)

This module performs the actual attacks. It connects to the ZAP tool, runs spiders to find pages, and then launches attacks.

python

Class ActiveScanningEngine:

Function Start(scan_id):

 Initialize ZAP_Adapter

 Validate ZAP Connection

Phase 1: Discovery (Finding Attack Surface)

 Load Passive_Discovery_Results() # Import URLs from passive phase

 Run ZAP_Spider(target_url) # Traditional crawler

 Run ZAP_AJAX_Spider(target_url) # Crawler for modern JS apps

 Discovered_URLs = Consolidate_URLs(Passive + Spider + AJAX)

Phase 2: Vulnerability Testing (Attacking)

 For each URL in Discovered_URLs:

 Run ZAP_Active_Scan(URL, configured_policy)

 # Tests for SQLi, XSS, CSRF, etc.

Phase 3: Processing

 Raw_Findings = Extract_ZAP_Alerts()

 Cleaned_Vulnerabilities = Deduplicate_And_Verify(Raw_Findings)

 Save_Results_To_Database(Cleaned_Vulnerabilities)

 Generate_Scan_Statistics()

Scan Data Optimizer (Reporting)

This module processes the raw data (which can be huge) into a format that is fast and readable for the user frontend.

python

Class ScanDataOptimizer:

Function Optimize_Spider_Results(raw_data):

 URLs = Extract_And_Deduplicate(raw_data)

 Filtered_URLs = Filter_Important_Endpoints(URLs) # Keep login, admin, api

 Paginate_Results(Filtered_URLs)

 Return Optimized_Json

Function Optimize_Vulnerabilities(vulnerability_list):

 Sort_By_Severity(Critical -> High -> Medium -> Low)

 For each vulnerability:

 Truncate_Large_Evidence_Text() # Prevent massive JSON responses

 Check_For_False_Positives()

```
Calculate_Summary_Stats(Counts by severity and category)
```

```
Return Optimized_List
```

Enhanced Discovery Module

This module runs before the Active Scanner to find more attack surface than ZAP can find alone. It uses specialized tools for subdomains, modern crawling, and API discovery.

```
python
```

```
Class EnhancedDiscoveryEngine:
```

```
Function Run_Comprehensive_Discovery(target_url):
```

```
Initialize results_container
```

```
Step 1: Subdomain Enumeration
```

```
    If Tool_Available("subfinder"):
```

```
        Subdomains = Run_Subfinder(domain)
```

```
        Add_To_Scope(Subdomains)
```

```
Step 2: Technology & Probing
```

```
    If Tool_Available("httpx"):
```

```
        Live_Hosts = Probe_HTTP_Services(Subdomains)
```

```
        Tech_Stack = Detect_Technologies(Live_Hosts)
```

```
Step 3: Modern Crawling (JavaScript-aware)
```

```
    If Tool_Available("katana"):
```

```
        # Crawls SPAs better than traditional spiders
```

```
        Crawled_URLs = Run_Katana_Crawler(target_url, depth=3)
```

```
    Else:
```

```
        Crawled_URLs = Run_GoSpider(target_url)
```

```
Step 4: Vulnerability & Endpoint Discovery
```

```
    If Tool_Available("nuclei"):
```

```
        # Uses templates to find exposed git, env, config files
```

```
        Endpoints = Run_Nuclei(Crawled_URLs, templates="exposed-panels, misconfig")
```

```
Step 5: API Analysis
```

```
    API_Endpoints = Analyze_Patterns_For_APIs(Crawled_URLs)
```

```
    # Looks for /api/v1, graphql, swagger.json, etc.
```

```
Step 6: Form Analysis
```

```
Forms = Analyze_HTML_For_Forms(Crawled_URLs)
```

```
# Identifies Login, Register, Upload, Contact forms for targeted attacking
```

```
# Compile and Return
```

```
Final_Results = Deduplicate_And_Merge(
```

```
    Subdomains, Live_Hosts, Crawled_URLs, Endpoints, Forms
```

```
)
```

Return Final_Results

4.5 Screenshots of Prototype / System

4.5.1 Home Page



Figure 4.3 Home Page

Figure 4.3 shows the home page of the Security Scanner web application.

4.5.2 Landing Page

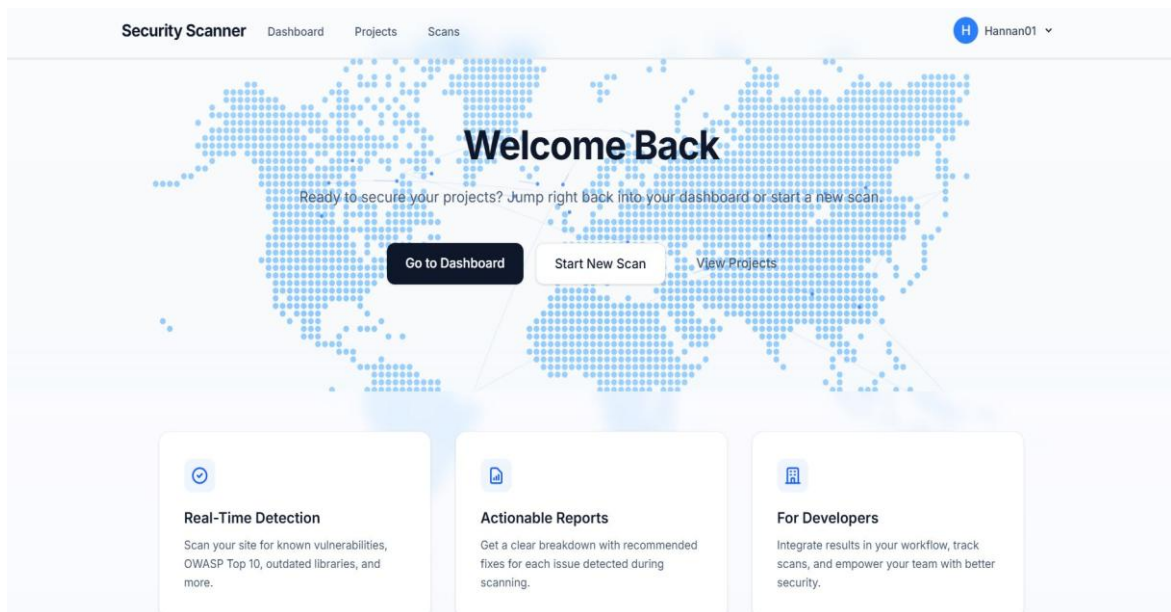


Figure 4.4 Landing Page

Figure 4.4 shows the main dashboard options, such as project navigation, scan initiation, and important features like real-time detection, actionable reports, and developer-focused integrations, are displayed in the Security Scanner welcome interface.

4.5.3 Authentication & Authorization

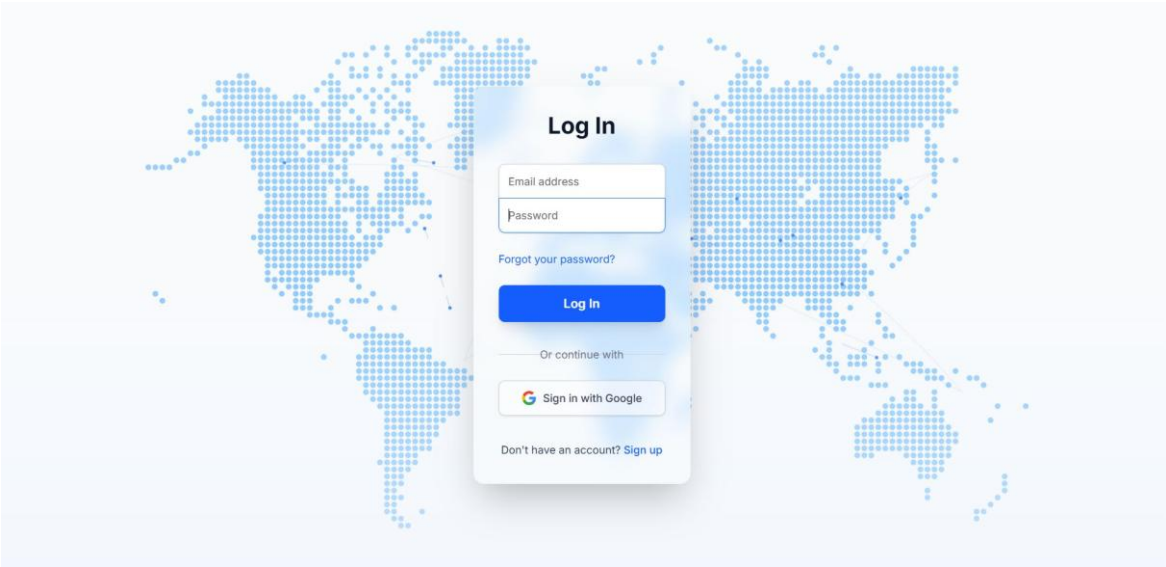


Figure 4.5 Authentication and Authorization

Figure 4.5 shows the Security Scanner system's login interface displaying user authentication choices, such as Google sign-in, password recovery, and email/password login

4.5.4 Dashboard

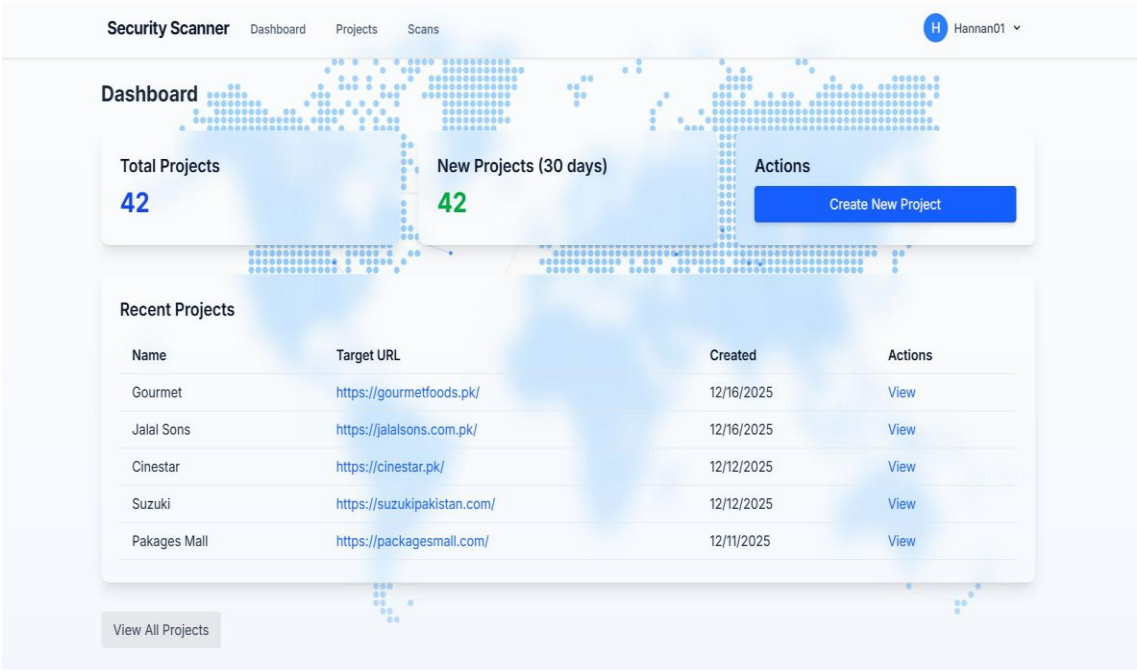


Figure 4.6 Dashboard

Figure 4.6 shows the Security Scanner system's dashboard, which includes a list of recent projects, available management actions, and an overview of all ongoing and new projects.

4.5.5 Password Recovery

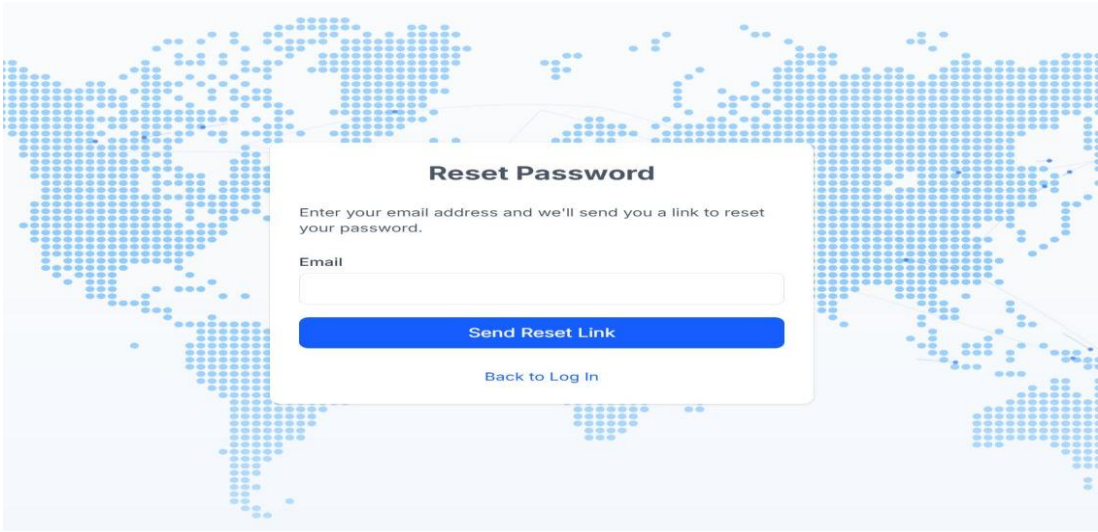


Figure 4.7 Password Recovery

Figure 4.7 shows the Password reset interface of the Security Scanner system allowing users to request a reset link by entering their registered email address.

4.5.6 Project Management Interface

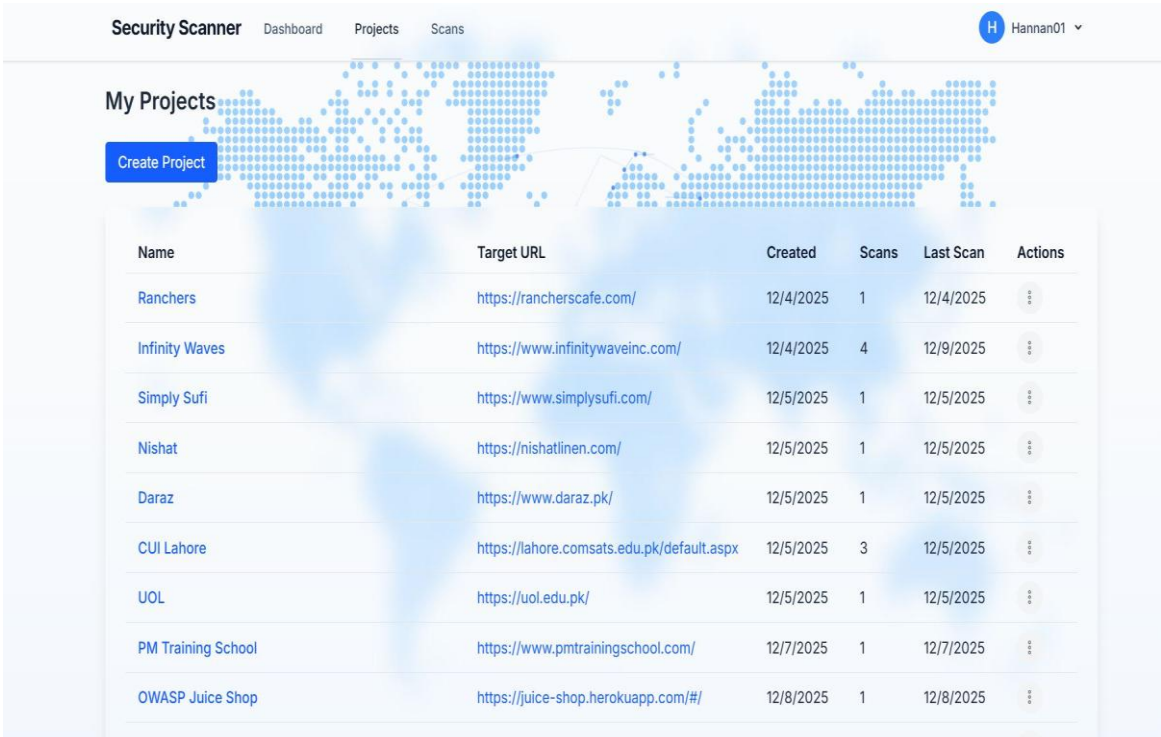


Figure 4.8 Project Management Interface

Figure 4.8 shows a list of user created projects along with their target URLs, creation dates, scan histories, and available management actions.

4.5.7 Security Scan Management Interface

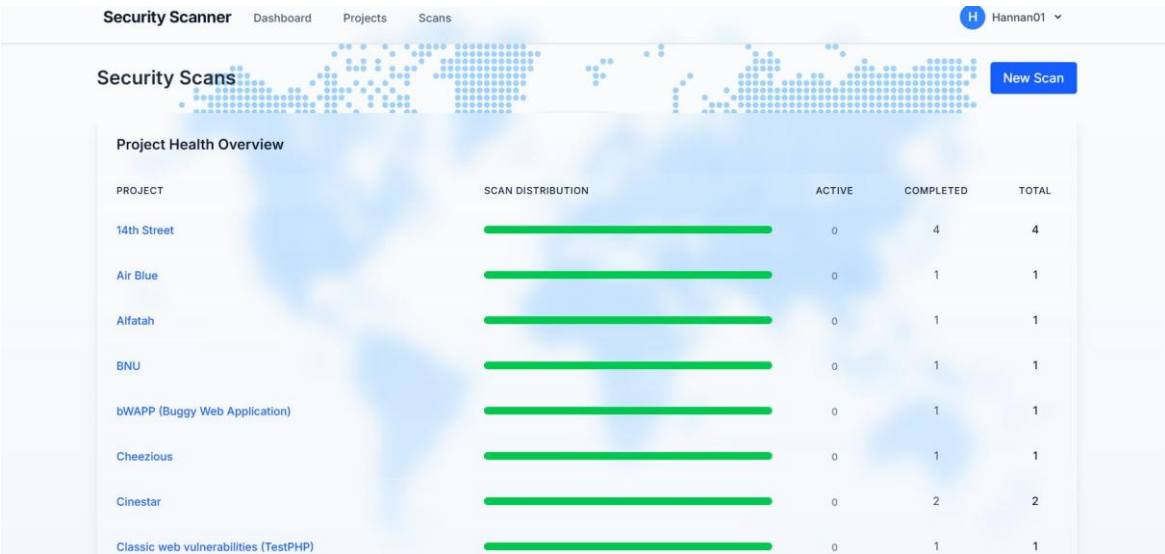


Figure 4.9 Project Health Overview Dashboard

Figure 4.9 shows the Project Health Overview dashboard of the security scanner, displaying scan distribution across multiple projects along with the number of active, completed, and total scans for each project.

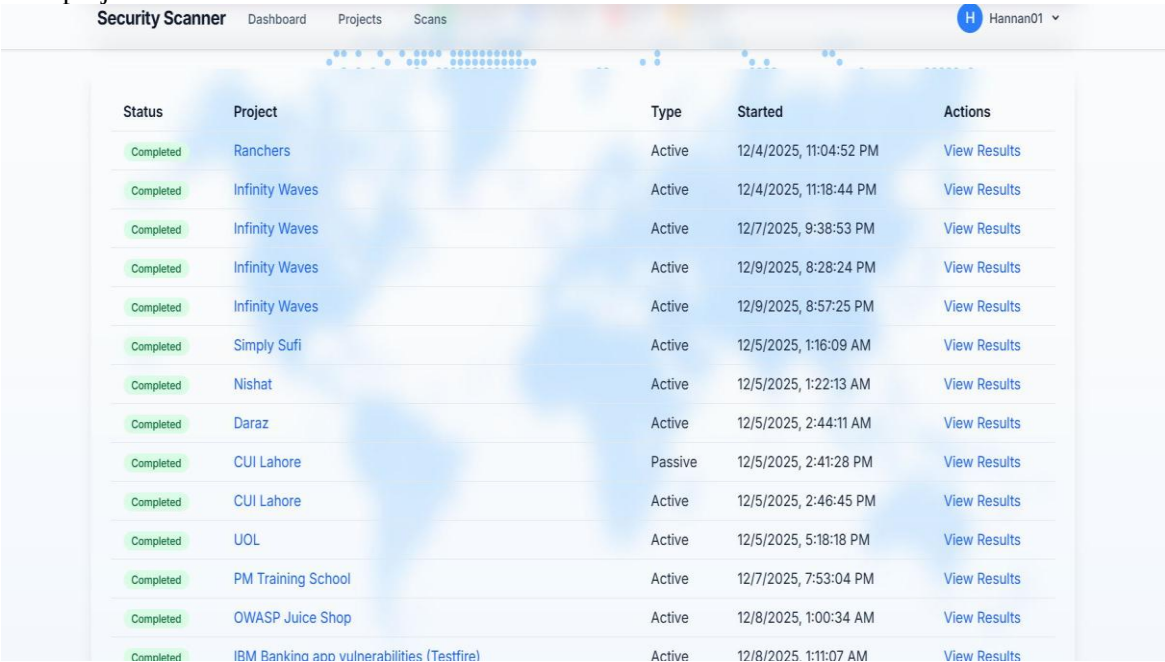


Figure 4.10 Security Scan Status and Results Listing

Figure 4.10 shows the security scan status page displaying a list of completed scans along with the projects, scan types, times, and actions to view scan results.

4.5.8 User Profile Management

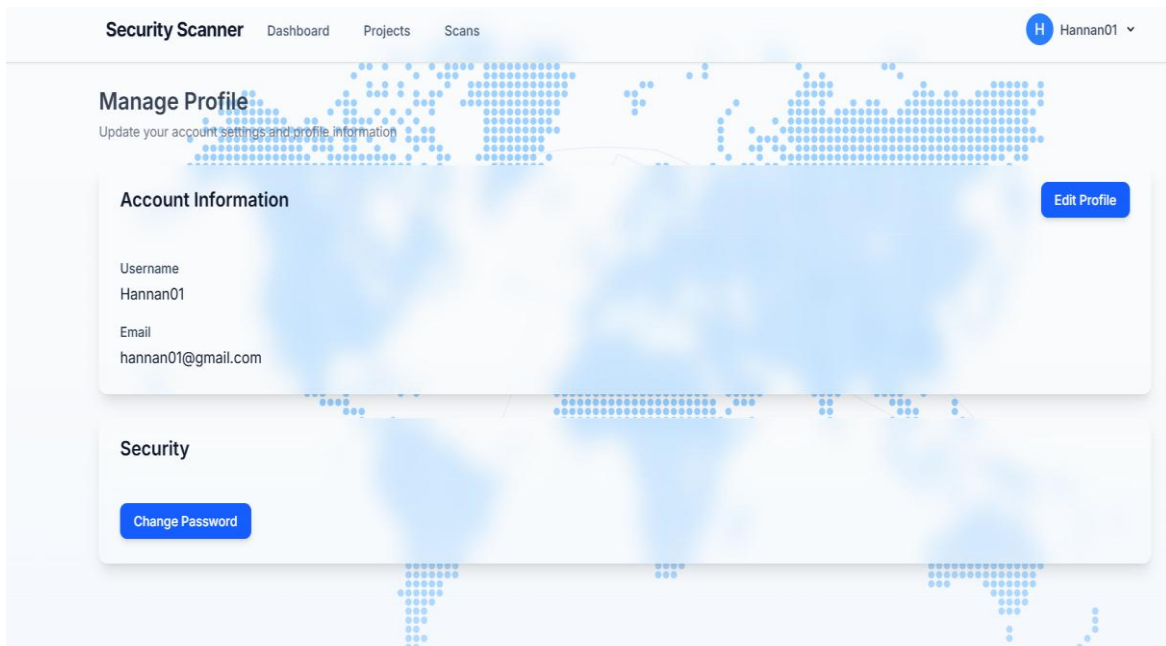


Figure 4.11 User Profile Management

Figure 4.11 shows the manage profile page in the security scanner, which allows users to view and modify profiles such as usernames and email, besides other security parameters including password updates.

4.5.9 Project Creation and Target URL Configuration Interface

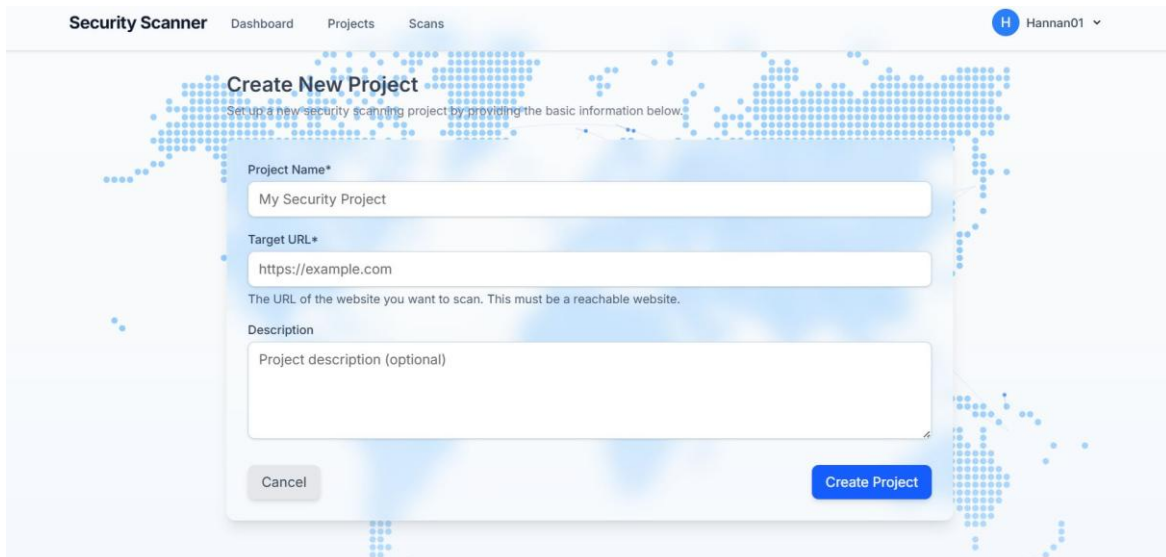


Figure 4.12 Project Creation and Target URL Configuration Interface

In the create new project page of the security scanner, shown as Figure 4.12 above , the user needs to introduce the name of the project, the URL of the target, and a description of the project.

4.5.10 Project Details and Scan Overview Interface

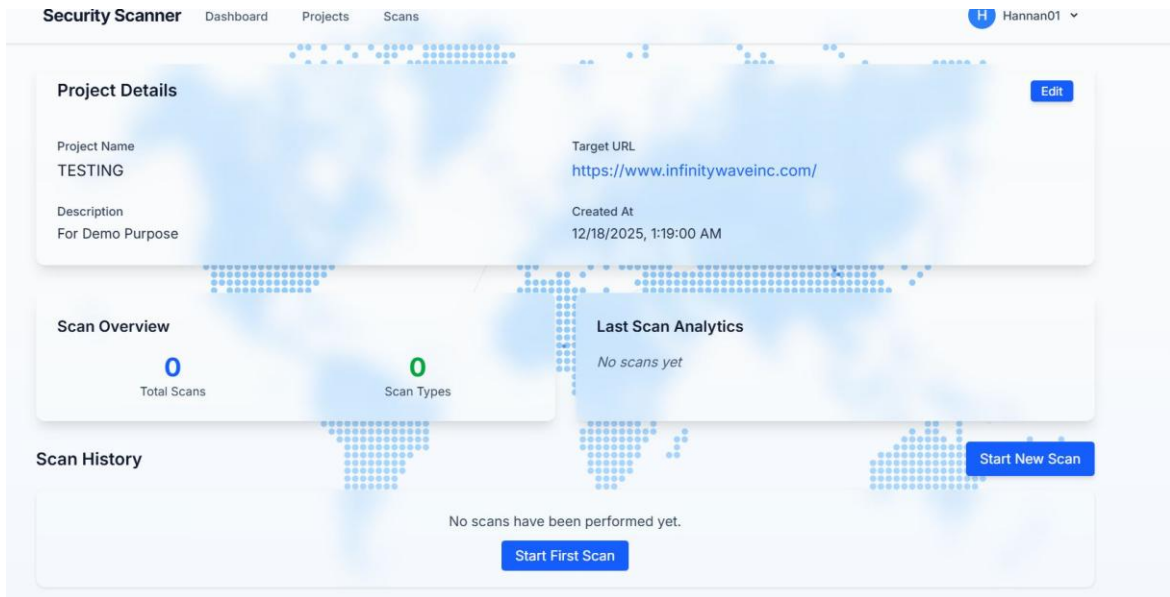


Figure 4.13 Project Details and Scan Overview Interface

Figure 4.13 shows the project details page of the security scanner, displaying project metadata such as project name, target URL, description, and creation date, along with an overview of scan statistics.

4.5.11 Security Scan Configuration Interface

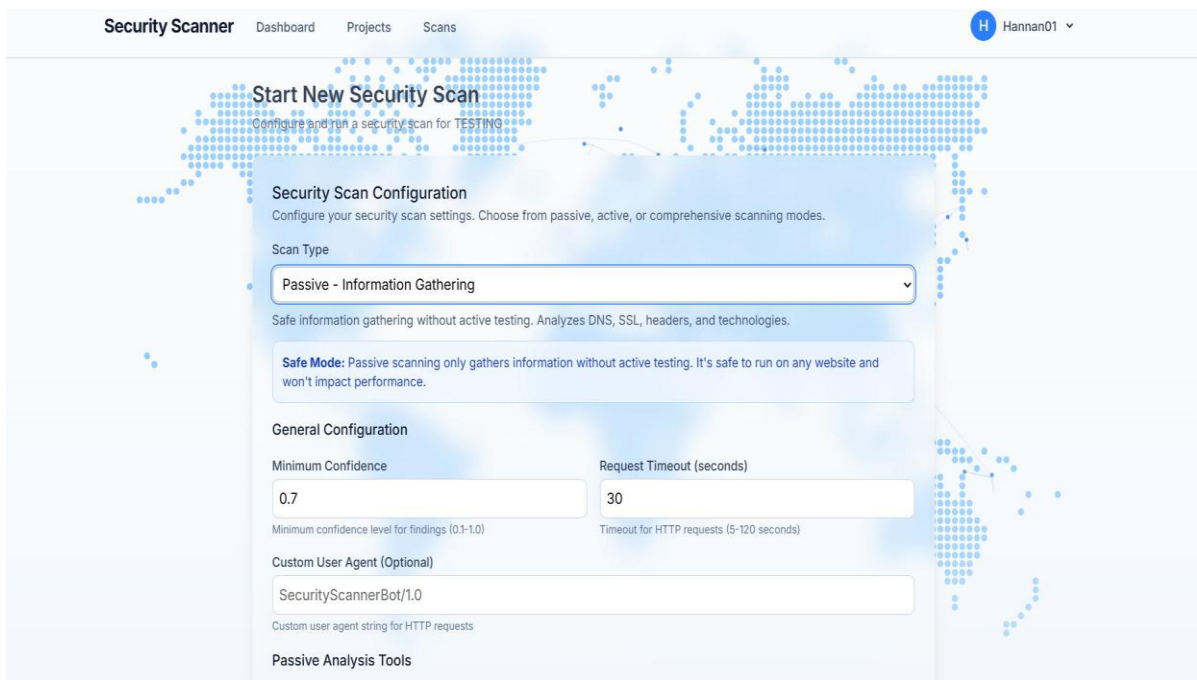


Figure 4.14 Passive Scan Configuration

The Passive Information Gathering Scan Configuration is depicted by Figure 4.14. It is a scanning technique that ensures harmless and non-intrusive information gathering, including the analysis of DNS, SSL, and header information, which does not impact the targeted system.

Security Scanner Dashboard Projects Scans Hannan01

Start New Security Scan

Configure and run a security scan for TESTING

Security Scan Configuration

Configure your security scan settings. Choose from passive, active, or comprehensive scanning modes.

Scan Type

Active - Vulnerability Testing

Comprehensive vulnerability testing with active probing. May send test payloads to the target.

Active Testing: This scan actively tests for vulnerabilities by sending specialized requests. Only scan websites you have permission to test.

General Configuration

Minimum Confidence: 0.7
Request Timeout (seconds): 30

Minimum confidence level for findings (0.1-1.0) Timeout for HTTP requests (5-120 seconds)

Custom User Agent (Optional): SecurityScannerBot/1.0
Custom user agent string for HTTP requests

Passive Analysis Tools

Figure 4.15 Active Scan Configuration

Figure 4.15 shows the Active Vulnerability Testing Scan Configuration.

Security Scanner Dashboard Projects Scans Hannan01

Start New Security Scan

Configure and run a security scan for TESTING

Security Scan Configuration

Configure your security scan settings. Choose from passive, active, or comprehensive scanning modes.

Scan Type

Comprehensive - Complete Assessment

Combines passive reconnaissance with active vulnerability testing for complete coverage.

Complete Assessment: Combines passive reconnaissance with active vulnerability testing. Ensure you have explicit permission before running this scan type.

General Configuration

Minimum Confidence: 0.7
Request Timeout (seconds): 30

Minimum confidence level for findings (0.1-1.0) Timeout for HTTP requests (5-120 seconds)

Custom User Agent (Optional): SecurityScannerBot/1.0
Custom user agent string for HTTP requests

Passive Analysis Tools

☒ SSLyze - SSL/TLS analysis ☒ Nuclei - Template-based scanning

☒ Wappalizer - Technology detection ☒ ZAP - Passive security analysis

Active Scanning Configuration

Figure 4.16 Comprehensive Scan Configuration

Figure 4.16 shows the comprehensive Security Scan Configuration. The above figure is the Comprehensive Security Scan Configuration screen, which performs both the passive reconnaissance scan and the active vulnerability scan tests and should be done only upon authorization.

4.5.12 Website Vulnerability Assessment and Security Analysis Results

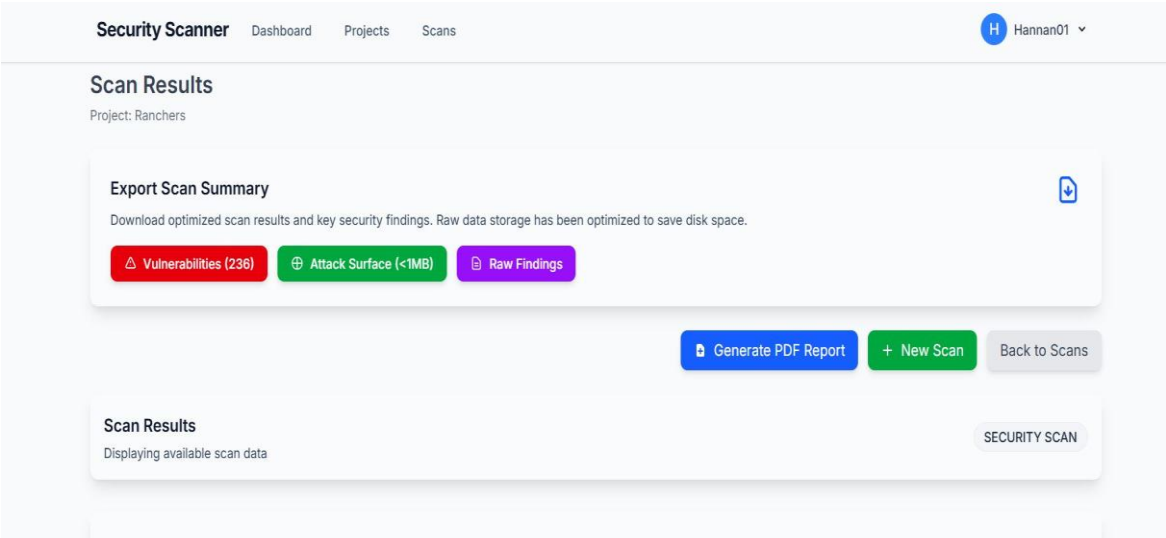


Figure 4.17 Security Scan Results Dashboard

Figure 4.17 shows a summary of discovered vulnerabilities, attack details, and scan analytics, with options to export report and generate detailed PDF results.

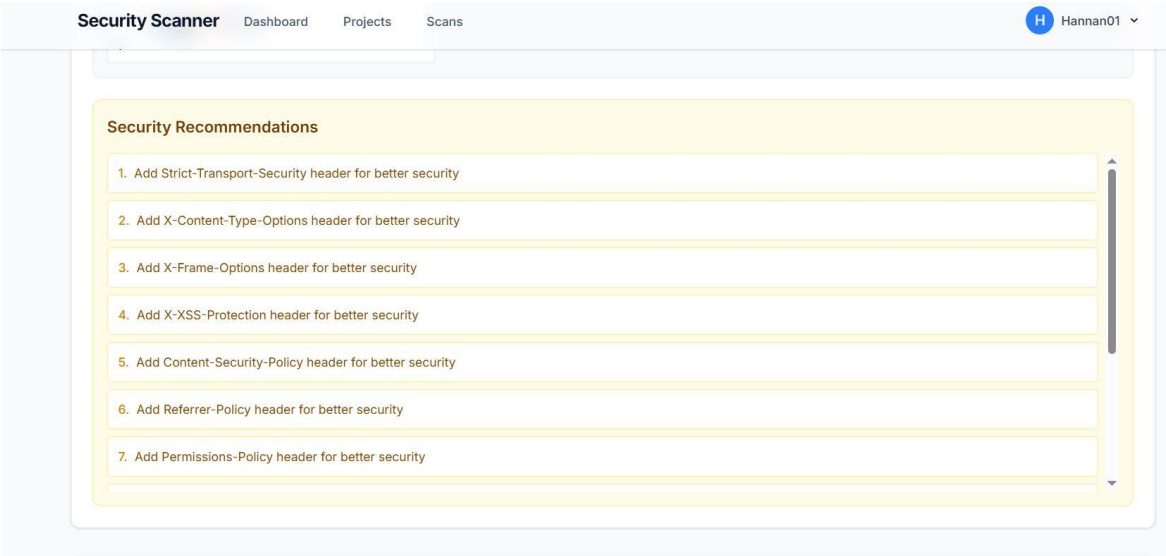


Figure 4.18 Security Recommendations

Figure 4.18 shows the web Security Scanner dashboard listing recommendations for improvement of security.

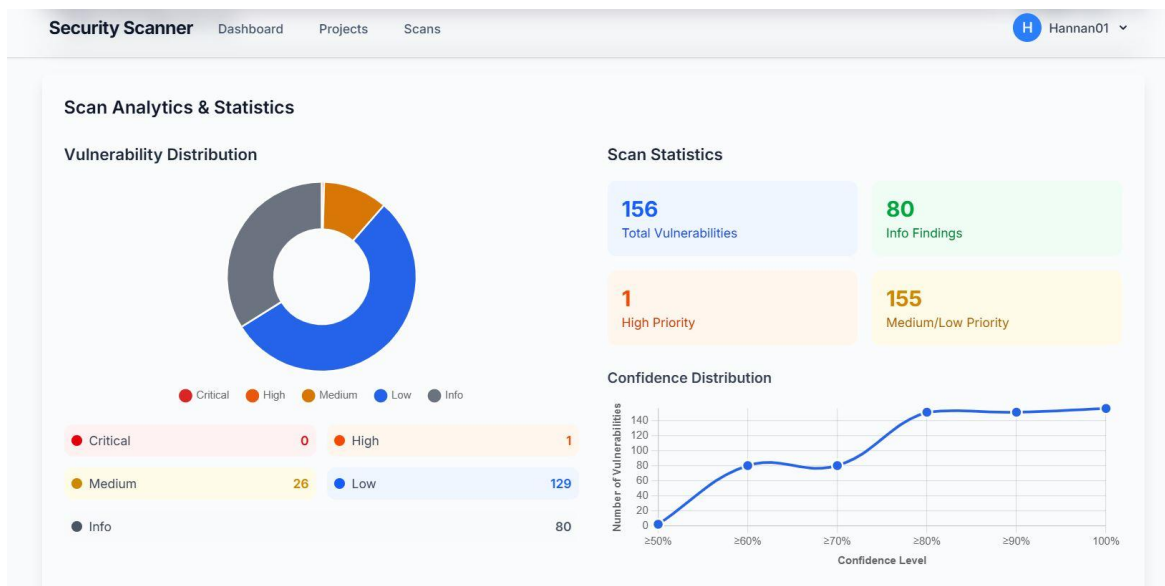


Figure 4.19 Scan Analytics and Statistics

Figure 4.19 highlights “Scan Analytics & Statistics” with a donut chart illustrating the distribution of vulnerabilities and important statistics such as total vulnerabilities and their priorities. There is also confidence distribution that provides a line graph of the number of vulnerabilities against their confidence levels.

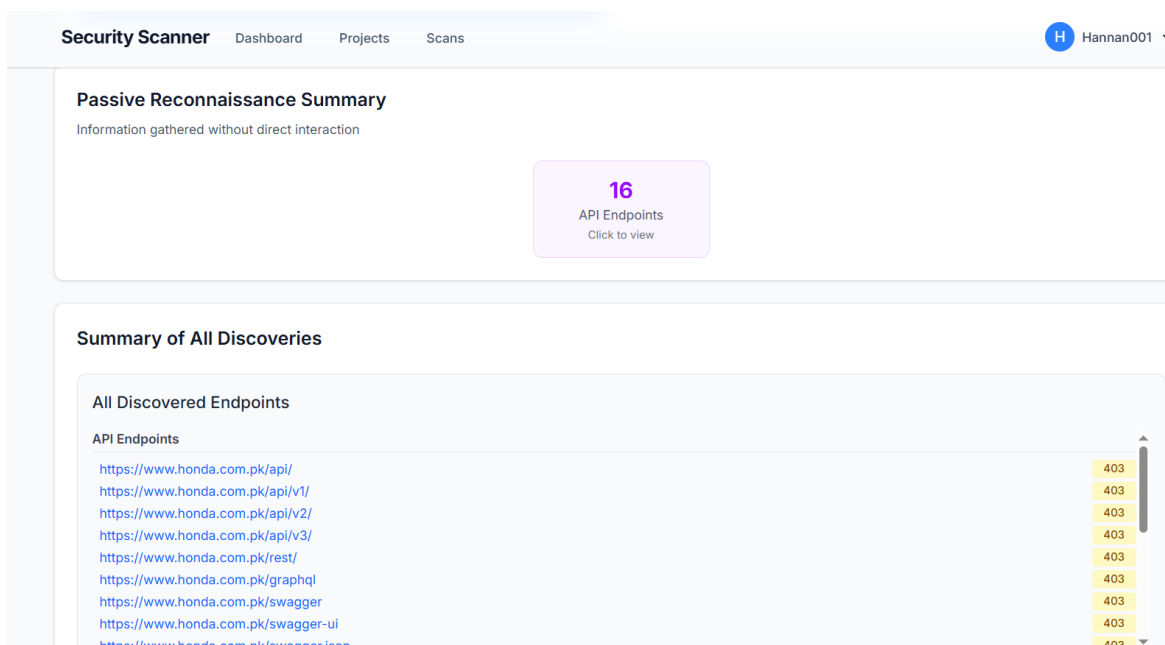


Figure 4.20 Passive Reconnaissance Summary

Figure 4.20 shows the web Security Scanner dashboard displaying a passive reconnaissance summary report.

4.6 Challenges During Implementation

4.6.1 Integration of Multiple Security Tools

One of the main challenges was to integrate various security tools into a single unified automated scanning pipeline. The system included interaction between crawling modules, the machine learning layer, data storage components, and active vulnerability scanners. Handling these interdependencies made the system intricate and hard to maintain. The troubleshooting of issues usually involved analyzing the full scan logs across the entire pipeline when errors or inconsistencies at one stage in processing often had implications for subsequent stages.

4.6.2 High Computational and Resource Requirements

Accordingly, active scanning in automated vulnerability scanning imposed considerable computational overhead: many scans necessitated high CPU utilizations and augmented execution times with respect to the size and complexity of the target application. This makes the development and testing cycles slower, knowing even small changes in scan configuration or pipeline logics require long re-execution times. Resource management needed to be done efficiently, and the scheduling of scans had to be carefully performed so these limitations could be mitigated.

4.6.3 Data Quality and Inconsistent Scanner Output

Another challenge was that input data generated by scanners was inconsistent. Vulnerability findings often had absent or incomplete fields, such as missing evidence or parameters. Because the absence of evidence itself is meaningful for false positive analysis, careful preprocessing had to be done to preserve this information without injecting bias or noise into the dataset.

4.6.4 Lack of Ground-Truth Labels for Machine Learning

One of the challenges facing machine learning application to outputs from vulnerability scanners is the absence of ground-truth labels that differentiate between true positives and false positives. Because this manually requiring security expert labels is very costly and consumes so much time, which is inappropriate to put under supervised learning. Because of this challenge, weak supervision and heuristic-based pseudo-labeling needed to be adopted, introducing uncertainty in the learning process and bounding traditional metrics such as accuracy and precision.

4.6.5 Feature Selection and Information Leakage Risk

Another challenging task was to select appropriate features for machine learning. Detailed payloads, URLs, or textual vulnerability descriptions also threatened to leak information and cause overfitting, reducing the generalizability of the model. The challenge was thus to identify security-aware features that capture the scanner behavior yet are generic and transferable across different applications and environments.

4.6.6 Model Evaluation and Interpretation

Another critical difficulty was evaluating the machine learning model's performance in the absence of reliable ground truth. It was not possible to use traditional classification metrics, which meant the need to adopt alternative strategies for evaluation, anchored on alert reduction, severity preservation, injection vulnerability retention, and confidence distribution analysis. Interpreting model outputs as probabilistic confidence scores rather than definitive predictions also required careful explanation to ensure correct understanding by end users and examiners.

Chapter 5. Results & Analysis

The experimental setup, results, comparative analysis, and discussion of findings for the suggested false positive reduction framework based on unsupervised and weakly supervised learning approaches are presented in this chapter.

5.1 Experimental Setup

This subsection briefly discusses the experimental setup for the system.

5.1.1 Test Cases

Below are the test cases of the system.

User Registration

Table 5.1 presents the details of the test case 001: User registration.

Table 5.1 Test Case 001-User Registration

Test Case ID 001			
Description		To verify that a new user can successfully register using the email/password method or Google Sign-In, including form validation and account verification.	
Pre-Condition		User is not logged in. No existing account with the same email or username.	
Post-Condition		A new user account is created and stored in the database. User account is verified and ready for login. User may proceed to login using valid credentials.	
Step	Action	Input Data	Response
1.	Open registration page	-	Registration page is displayed
2.	Fill out registration form	Email: ahmad.naeem@gmail.com Username: Ahmad Naeem Password: Ahmad@1234	Form is filled with valid data
3.	Submit form	Click "Register"	System validates inputs and sends verification email
4.	Click verification link	Click verification link received via email within 15 minutes	System verifies the user and activates the account
5.	Alternative: Sign up with Google	Google Account: hannan.ali@gmail.com	System redirects to Google OAuth screen
6.	Approve Google access	User grants Google access	System auto-creates and verifies user account
7.	Validation result	-	Account is created and user is redirected to login/dashboard

Login

Table 5.2 presents the details of the test case 002: User login.

Table 5.2 Test Case 002-Login

Test Case ID 002			
Description		To verify that a registered user can successfully log in using a valid email/username and password, and the system authenticates them.	
Pre-Condition		User has already registered and created an account. User is currently logged out.	
Post-Condition		User is logged into the system. A session is created for the user. Login activity is optionally logged for auditing.	
Step	Action	Input Data	Response
1.	Open login page	-	Login page is displayed
2.	Enter credentials	Email: mujtaba.kamil@gmail.com Password: Mujtaba@1234	Input is accepted for processing
3.	Submit login form	Click "Login"	System validates input format
4.	Validate credentials	System checks email and password against the database	Credentials matched
5.	Login successful	-	User is authenticated and logged in
6.	Redirect to dashboard	-	User is redirected to dashboard/home page
7.	Alternative: Invalid login	Email: wrong.email@gmail.com Password: wrongpass123	System displays error message: "Invalid email or password"

Logout

Table 5.3 presents the details of the test case 003: User logout module.

Table 5.3 Test Case 003-Logout

Test Case ID 003			
Description		To verify that a logged-in user can securely log out, terminating the session and clearing authentication data.	
Pre-Condition		The user must be logged into the system.	
Post-Condition		Session token is destroyed. User is no longer authenticated. Logout event is optionally recorded in activity logs.	
Step	Action	Input Data	Response
1.	Click "Logout" button	-	System registers the logout request
2.	Send logout request	Session token: ahmad-session-001	Request is sent to the server's authentication handler

3.	Invalidate session on server	-	Session token is invalidated successfully
4.	Clear client-side data	Cookies: auth= true Local Storage: userToken	Authentication data cleared from browser
5.	Redirect to login/home page	-	User is redirected to login or landing page
6.	Log the logout activity	User: ahmad.naeem@gmail.com	Logout time recorded in the activity log with timestamp
7.	Exception (if token expired)	Expired session token	System handles gracefully and still redirects to login page

Password Recovery

Table 5.4 presents the details of the test case 004: Password recovery.

Table 5.4 Test Case 004-Password Recovery

Test Case ID 004			
Description		To verify that a registered user can reset their forgotten password using a secure email-based recovery process.	
Pre-Condition		User has a registered account. User forgot their password. Email system is operational.	
Post-Condition		Password is updated in the system. Old sessions or tokens are invalidated. User can log in using the new password.	
Step	Action	Input Data	Response
1.	Access “Forgot Password” page	-	System displays password recovery form
2.	Enter registered email	hannan.ali@gmail.com	System validates email format and checks existence in database
3.	Generate password reset token	Email: hannan.ali@gmail.com	System creates a secure, one-time token with 30-minute expiry
4.	Send reset link to user	Email: hannan.ali@gmail.com	Email sent with password reset link
5.	Open reset link	Token: reset-hannan-001	User is redirected to secure reset page
6.	Enter new password	NewPass@123	System validates password complexity
7.	Confirm password reset	-	Password updated and stored securely using hashing
8.	Invalidate old sessions (optional security)	-	All existing user sessions are invalidated

9.	Attempt login with new password	Email: hannan.ali@gmail.com, Password: NewPass@123	Login successful
10.	Exception: Enter unregistered email	Email: unknown.user@gmail.com	System shows “Email not found” error
11.	Exception: Expired reset link	Expired token	System shows “Reset link expired” error
12.	Exception: Weak new password	Password: 123	System prompts to enter a stronger password

Profile Management

Table 5.5 presents the details of the test case 005: Profile management.

Table 5.5 Test Case 005-Profile Managemnet

Test Case ID 005			
Description		To verify that the user can successfully manage their profile by updating personal details, uploading a profile picture, and configuring privacy settings.	
Pre-Condition		User is logged in. User has an existing profile in the system.	
Post-Condition		User's profile information is updated. Profile picture is uploaded and stored. Privacy settings are saved successfully.	
Step	Action	Input Data	Response
1.	Access profile management page	-	Profile management interface is displayed
2.	Edit name and contact info	Name: Ahmad Naeem, Phone: 03184121652	System validates input and displays updated form
3.	Upload profile picture	File: ahmad.jpg (JPG, 1.5MB)	System validates size/type and uploads image
4.	Configure privacy settings	Visibility: Only Friends, Show Email: No	System updates visibility preferences and confirms
5.	Save profile	-	System stores all profile updates and confirms success
6.	Alternative: Skip picture upload	-	System saves with default/no image
7.	Alternative: Skip privacy settings	-	System applies default visibility settings
8.	Exception: Invalid phone number	Phone: 123abc	System shows validation error
9.	Exception: Unsupported image format	File: mujtaba.bmp	System shows error: unsupported file type

10.	Exception: Image exceeds size limit	File: hannan_large.png (3.2MB)	System shows error: file exceeds maximum allowed size
11.	Exception: Empty required fields	Name: (blank), Phone: (blank)	System prompts user to fill required fields
12.	Exception: Server error during save	Valid input, but backend server crashes	System shows “Unable to save profile. Please try again later.”

View Projects

Table 5.6 presents the details of the test case 006: Project management module.

Table 5.6 Test Case 006-View Projects

Test Case ID 006			
Description		To ensure that a logged-in user can view their recent projects on the dashboard. If no projects exist, the system should show a relevant message.	
Pre-Condition		User is logged in. User has either existing projects or none.	
Post-Condition		User sees a project list or a “No projects found” message. User may proceed to manage or open a project.	
Step	Action	Input Data	Response
1.	User accesses the dashboard	-	Dashboard loads
2.	System queries and retrieves recent projects	User ID: u2343	Server returns list of projects or empty list
3.	System displays project list	Projects: [“Test1”, “Scan2”]	Names and target URLs shown in dashboard grid
4.	User clicks on a project to view/manage	Project ID: scan2	System redirects to project detail view
5.	Alternative: User has no projects	Projects: []	Message “No projects found” is displayed
6.	System auto-refreshes if a project is added	Action: Create new project in another tab	New project appears in the list without full reload
7.	Exception: Error fetching projects	Simulate DB failure or network issue	System shows error: “Unable to load projects. Please try again later.”
8.	Exception: Dashboard timeout	Simulate long delay or crash	System shows: “Dashboard loading failed due to timeout.”

Add Project

Table 5.7 presents the details of the test case 007: Project management module.

Table 5.7 Test Case 007-Add Projects

Test Case ID 007			
Description		To verify that a logged-in user can successfully create a new project by providing a valid name and a reachable target URL.	
Pre-Condition		User is logged in. User has access to the “Add Project” form.	
Post-Condition		Project is saved under the user's account in the database. User is redirected to the project details page.	
Step	Action	Input Data	Response
1.	User navigates to the “Add Project” page	-	Add Project form is displayed
2.	User enters project name and target URL	Name: “AcmeScan” URL: https://acme.com	System proceeds to validate input
3.	System validates the URL format	URL: https://acme.com	URL is syntactically valid
4.	System performs HTTP reachability check	Ping https://acme.com	Site responds successfully
5.	System saves project data to the database	Project metadata and user ID	Data stored successfully
6.	System redirects user to the project detail page	Redirect trigger	User is sent to /projects
7.	Alternative: Invalid URL format	URL: http://badurl	System shows: “Please enter a valid URL.”
8.	Alternative: Unreachable URL	URL: https://nonexistent.domain	System shows: “Target URL is unreachable. Check the domain and try again.”
9.	Exception: Database error during save	Simulate DB failure	System shows: “An error occurred while saving the project. Please try again later.”

Open Project

Table 5.8 presents the details of the test case 008: Project management module.

Table 5.8 Test Case 008-Open Project

Test Case ID 008	
Description	To verify that a logged-in user can select and open a project from the dashboard, view its details, and access related modules like scan configuration, execution, and report viewing.

Pre-Condition		User is authenticated and logged in. User has at least one project saved. Project data is valid and intact.	
Post-Condition		User is redirected to the project details page where project information and action modules are accessible.	
Step	Action	Input Data	Response
1.	User accesses the dashboard	-	Dashboard with a list of projects is shown
2.	User selects a project from the list	Project ID: PJT123	System begins to fetch selected project details
3.	System retrieves project data	Query using project ID	Returns project name, target URL, and creation date
4.	System displays project details	Data from DB	Details (Name, URL, Date) are shown on screen
5.	System displays navigation to modules	-	Links/buttons for Scan Configuration, Scan Execution, Report Viewing are displayed
6.	User interacts with any available module	Click "Scan Configuration"	User is redirected to relevant module tied to the project
7.	Alternative: Project retrieval fails	Invalid ID or DB timeout	System shows: "Unable to load project details. Please try again later."
8.	Exception: Unauthorized project access	Project owned by another user	System shows: "Access denied. You are not authorized to view this project."

Target Configuration

Table 5.9 presents the details of the test case 009: Target Configuration Module.

Table 5.9 Test Case 009-Target Configuration

Test Case ID 009			
Description		To verify that a user can configure a website as a scan target, set scanning and crawling preferences, and optionally enable Passive Information Gathering (PIG) settings.	
Pre-Condition		User is logged in. User is on the target configuration page.	
Post-Condition		The scan configuration, including target URL, scan parameters, crawling options, and PIG settings (if any), is stored successfully.	
Step	Action	Input Data	Response
1.	User accesses the target configuration page	-	Configuration interface is loaded

2.	User enters website URL as scan target	https://example.com	System initiates reachability check
3.	System performs HTTP status check	Target URL	Status 200 OK proceed; else, error message
4.	User selects scan parameters	Depth: Deep, Intensity: High, Respect robots.txt: Yes	UI accepts scan settings
5.	User customizes crawling options	User-agent: CustomBot, Delay: 5s, Headers: Key=Val	Inputs accepted if valid
6.	User enables Passive Information Gathering (optional)	Toggle: ON	PIG settings panel appears
7.	User selects PIG modules	WHOIS, DNS Lookup, IP Info, Headers	Selections validated and accepted
8.	System saves the configuration	All parameters and options	Confirmation message: "Configuration Saved Successfully"
9.	Alternative: Invalid or unreachable URL	http://invalid-url.test	Error: "Target unreachable. Please enter a valid URL."
10.	Alternative: Invalid input in crawling/PIG options	e.g., empty user-agent field	UI shows validation errors and prevents submission
11.	Exception: Passive modules unavailable	WHOIS API timeout	Warning: "WHOIS service temporarily unavailable. Try again later."
12.	Exception: Server error saving configuration	Internal DB write failure	Error: "Failed to save configuration. Please retry."

Passive Information Gathering

Table 5.10 presents the details of the test case 010: Passive Information Gathering.

Table 5.10 Test Case 010-Passive Information Gathering

Test Case ID 010	
Description	In order to prove the system makes non-intrusive OSINT-based passive scans on a specific target URL and stores the acquired results for post-Analysis.
Pre-Condition	Target URL is configured. Scan mode is set to Passive Reconnaissance. Target is reachable (responds to network checks).
Post-Condition	Metadata is collected and stored under the scan session; available for report generation and machine learning modules.

Step	Action	Input Data	Response
1.	System resolves DNS records of the target	Target domain	A, MX, NS, TXT, CNAME records collected
2.	System fetches robots.txt and sitemap.xml	https://example.com/robots.txt	Files retrieved or logged as “not found”
3.	System sends HEAD request to gather HTTP headers	HEAD https://example.com	Response headers (e.g., Server, X-Powered-By) collected
4.	System performs technology fingerprinting using headers and HTML	Meta tags, response headers	Detected tech stack stored (e.g., PHP, WordPress, Nginx)
5.	System stores all passive data	Recon results	Data saved and indexed in scan session
6.	Data becomes available for ML engine and reporting module	-	Modules receive structured data input
7.	Alternative: robots.txt or sitemap.xml not found	404 response	Logged as missing; scan proceeds
8.	Alternative: Target unreachable	No response	Recon aborted; “Network unreachable” logged
9.	Exception: Timeout during DNS or HTTP	Delayed response > 10 seconds	Error logged: “Lookup timed out. Partial data saved.”
10.	Exception: Malformed or invalid URL	http://invalid-url	Error: “Malformed target URL. Recon skipped.”

Scan Execution

Table 5.11 presents the details of the test case 011: Scan Execution.

Table 5.11 Test Case 011-Scan Execution

Test Case ID 011			
Description		Ensures that a system has the ability to scan a target, conduct both passive and active reconnaissance, and also allows a user to interrupt it while data is logged.	
Pre-Condition		Target website is properly configured. User has initiated the scan.	
Post-Condition		Passive and active scan data is collected, logged, and stored. Scan completes or is halted with progress saved.	
Step	Action	Input Data	Response

1.	User initiates scan on configured target	Target URL	Scan initialized; system begins passive recon
2.	System performs passive info gathering (DNS, WHOIS, headers)	-	Metadata collected and logged
3.	Passive data visualized for user	Passive scan results	Recon shown in UI (e.g., DNS records, server headers)
4.	System begins website crawling	Target domain	Pages discovered and crawled
5.	System collects forms, cookies, headers, and internal URLs	Crawled pages	Structured data stored and indexed
6.	System logs progress and displays live scan status	-	Real-time updates visible to user
7.	System completes scan or proceeds until user interruption	-	Full or partial scan results saved
8.	Alternative: User manually stops the scan	“Stop” command from user	Scan halts safely; current progress stored
9.	Exception: Scan fails due to server/crawling error	Network/server fails	Error logged: “Scan interrupted. Data saved up to failure point.”
10.	Exception: Passive recon fails due to DNS issue	Invalid domain	Error: “Passive info gathering failed. Check target URL.”

Vulnerability Analysis and Prediction

Table 5.12 presents the details of the test case 012: Vulnerability Analysis and Prediction.

Table 5.12 Test Case 012 Vulnerability Analysis and Prediction

Test Case ID 012			
Description		It is responsible for the accurate scanning data analysis required in the identification of Top 10 OWASP vulnerability threats via active inputs and passive Recon.	
Pre-Condition		Scan execution has completed. Target website is reachable and crawled. Passive and active data are available.	
Post-Condition		Vulnerabilities are detected, tagged by severity, logged, and forwarded for ML-based prediction and remediation.	
Step	Action	Input Data	Response
1.	System begins analyzing scan data	Completed scan session	Analysis initiated, system loads passive + active data

2.	System tests inputs for SQL Injection	Form fields, URLs	Potential SQLi flagged and logged if detected
3.	System checks for XSS via JS injection attempts	Input fields, scripts	Reflected or stored XSS patterns identified and tagged
4.	System checks for IDOR via URL/parameter manipulation	URL/query parameters	Unauthorized access patterns flagged as IDOR if detected
5.	System checks for other OWASP Top 10 (e.g., misconfig, data exposure)	Headers, endpoints	Issues identified and tagged accordingly
6.	System correlates passive data to improve context and accuracy	Headers, sitemap, etc.	Context-aware tagging of findings (e.g., sensitive files, missing headers)
7.	Vulnerabilities are tagged with severity and stored	Detection results	Results saved to database and scan report
8.	System applies ML models to predict likelihood/severity	Raw findings	Prediction results generated and passed to Remediation module
9.	Alternative: No vulnerabilities found	Valid scan data	System generates a clean report indicating “No vulnerabilities detected.”
10.	Alternative: Only passive data available	Passive-only session	System provides limited insights and warns about skipped active testing
11.	Exception: Scan fails due to target blocking or server timeout	WAF/CAPTCHA/timeout	Scan halted, error logged: “Analysis incomplete due to target blocking or timeout.”
12.	Exception: Website layout prevents certain vulnerability detection	Complex SPA	Warning logged: “Some checks skipped due to dynamic content or structure limitations.”

Data Collection and Model Training

Table 5.13 presents the details of the test case 013: Data Collection and Model Training.

Table 5.13 Test Case 013-Data Collection and Model Training

Test Case ID 013			
Description		Verifies that the system retrieves historical scan data information, processes it, develops models for threat classification using ML, and updates models on its performance.	
Pre-Condition		Previous scan data is available in storage. ML model infrastructure is initialized and accessible.	
Post-Condition		Trained ML models are updated, deployed for threat prediction, and versioned properly.	
Step	Action	Input Data	Response
1.	System extracts historical scan data	Stored scan results	Data loaded for processing
2.	System performs data preprocessing (cleaning, normalization, feature extraction)	Raw scan logs	Cleaned dataset ready for training
3.	System splits data into training and validation sets	Processed dataset	Data split confirmed
4.	System applies ML algorithm to train threat classifier	Training set	Model trained with intermediate logs and progress tracked
5.	System evaluates model performance using test set	Test set	Accuracy, F1-score, and confidence score calculated
6.	If performance meets threshold, model is versioned and deployed	Evaluation metrics	Model saved and registered to prediction module
7.	Alternative: Preprocessing fails due to corrupted/incomplete data	Corrupted scan data	System logs error: "Preprocessing failed-check data integrity"
8.	Exception: Model training fails	Valid input, training error	Training halted, model rollback initiated if partial deployment exists
9.	Exception: Storage or system failure during preprocessing	Storage/connection error	Error logged, training aborted until next schedule

Remediation Suggestion Based on ML Prediction

Table 5.14 presents the details of the test case 014: Suggestion Based on ML Prediction.

Table 5.14 Test Case 014-Remediation Suggestion Based on ML Prediction

Test Case ID 014			
Description		This tests that the system is able to provide vulnerabilities predicted by machine learning and remediation steps mapped by a knowledge base and reported through the UI and reports.	
Pre-Condition		ML prediction for a completed scan session is available. Remediation knowledge base is accessible.	
Post-Condition		User receives accurate remediation guidance for each detected vulnerability in the dashboard and the final report.	
Step	Action	Input Data	Response
1.	System receives ML output with predicted vulnerabilities	Prediction results with confidence	Vulnerability list prepared for lookup
2.	System checks each vulnerability against remediation knowledge base	List of vulnerabilities	Matches found and linked with remediation steps
3.	System prioritizes suggestions based on vulnerability severity	Matched remediation entries	Critical issues get top-priority recommendations
4.	System displays suggestions in the scan results dashboard	User views results	UI shows actionable remediation for each detected issue
5.	System inserts the same suggestions into PDF report under "Remediation Guidance"	Report generation module	PDF report includes the guidance section
6.	Alternative: No vulnerabilities found	ML output empty	Message displayed: "No vulnerabilities detected. No action needed."
7.	Alternative: ML confidence low	Prediction < threshold	Suggestion marked: "Low confidence - verify manually"
8.	Exception: No match in remediation knowledge base	Unknown vulnerability type	Fallback message: "Generic remediation recommended - consult OWASP guide."
9.	Exception: Report generation fails to include suggestions	System/reporting error	System notifies user: "Report incomplete-remediation section missing."

Report Generation

Table 5.15 presents the details of the test case 015: Report Generation.

Table 5.15 Test Case 015- Report Generation

Test Case ID 015			
Description		Verifies that the system generates a structured, comprehensive scan report, including vulnerability details, ML predictions, severity classification, and remediation guidance, with export support.	
Pre-Condition		A scan has been completed. Scan results (passive and active) are available. ML-based prediction has been performed.	
Post-Condition		A timestamped report is generated, viewable in the system, exportable as PDF, and linked to the scan history.	
Step	Action	Input Data	Response
1.	User navigates to the scan reporting section	Completed scan session	Reporting dashboard loads with available scan records
2.	User clicks "Generate Report"	Scan ID selected	System begins report generation process
3.	System gathers passive and active scan data	Scan results from storage	All relevant data segments retrieved
4.	System includes ML prediction data	ML prediction module output	Predicted vulnerabilities and confidence scores fetched
5.	System classifies vulnerabilities by severity	Vulnerability list	Classification into Critical, High, Medium, and Low categories
6.	System fetches remediation suggestions and maps them to vulnerabilities	Prediction results + remediation KB	Actionable guidance attached to each issue
7.	System compiles and displays a report structure with the following sections: • Passive Recon Info • Detected Vulnerabilities • Predicted Vulnerabilities • Remediation Steps	N/A	Structured report rendered in the UI
8.	System provides an option to download the report as a PDF	Click on "Export as PDF"	PDF file is generated with all report sections
9.	Report is timestamped and added to report history	N/A	Report entry appears in user's history for future access
10.	Alternative: No vulnerabilities detected	Scan has no findings	Message displayed: "No vulnerabilities detected." Report still includes passive recon and metadata
11.	Exception: ML prediction unavailable	ML service offline or failed	System skips the prediction section; displays notice: "Prediction unavailable - skipped in report."

12.	Exception: Remediation suggestions fail to load	Knowledge base query fails	Fallback guidance shown: "Generic remediation advice - verify manually."
13.	Exception: Report generation fails due to system error	Internal error	User receives error: "Report generation failed-please try again later."
14.	Exception: Incomplete scan data detected	Missing scan segments	Warning message shown: "Some scan data missing - report may be incomplete."

Report Management

Table 5.16 presents the details of the test case 016: Report Management.

Table 5.16 Test Case 016-Report Management

Test Case ID 016			
Description		Verifies the user's ability to view, organize, and export scan reports, with proper access control and versioned report history.	
Pre-Condition		User is logged in. At least one scan and corresponding report is generated.	
Post-Condition		Reports are accessible for viewing, organizing, or exporting; historical data is maintained per project.	
Step	Action	Input Data	Response
1.	User navigates to "Report Management" via dashboard	Logged-in session	System loads the Report Management interface
2.	System fetches reports linked to user's projects	User ID and project ID	List of reports displayed (title, timestamp, status)
3.	User selects a report and chooses an action: • View • Rename • Organize	Report ID	Corresponding action is executed (content displayed, name updated, folder assigned)
4.	User clicks "Export Report"	Selected report	System generates a PDF version with charts and formatting
5.	User downloads or saves the exported file	Download location	File is saved locally with timestamped filename
6.	System logs report version and export event	Report ID	Report history updated with timestamp and version info

7.	Alternative: No reports found for selected project	Project has no reports	System displays: “No reports available for this project.”
8.	Exception: Export fails	System error	Error message shown: “Export failed – please try again later.”
9.	Exception: User attempts to access another user’s report	Unauthorized access	System denies access and redirects to dashboard with warning
10.	Exception: Organizing reports fails due to sync/storage issue	Folder update fails	System displays: “Unable to organize report -retry or contact admin.”

5.1.2 System Under Test

The system under test is an automated web application vulnerability scanning pipeline that has been upgraded with a machine learning based false positive reduction component. The component can be described as a post-processing stage of the scanning activity that receives vulnerability reports produced by the scanner and tries to assess the probability of each report being a false positive.

The experiments were performed on vulnerability scan data collected from intentionally vulnerable applications, as well as from production environments. The analysis will concentrate on the behavior of the models on new real world vulnerability scans..

5.1.3 Learning Paradigm

Because there were no ground truths to label whether a vulnerability is a correct result or a false positive, the task was modeled using the notions of unsupervised learning and weakly supervised learning.

First, the domain heuristics were applied to determine the potential patterns of the false positives (for example, header-related problems with no evidence on the web). This enabled the pseudo-labeling, which directed the learning.

The last model refrains from trying to predict the correctness of vulnerabilities and, instead, estimates the confidence of the false positives, which makes it much more similar to an unsupervised approach to anomaly scoring.

5.1.4 Evaluation Metrics

Classical supervised metrics such as accuracy are not suitable in this context due to the lack of true labels. However, to analyze internal consistency and model behavior, the following metrics and indicators were examined:

- Precision and F1-score computed on pseudo-labeled data (used only for internal validation)
- Alert reduction rate
- Preservation of high-risk vulnerabilities
- Stability across confidence thresholds

These metrics are interpreted cautiously and are not treated as indicators of real-world correctness.

5.2 Results

5.2.1 Alert Reduction Performance

The proposed approach was evaluated on a previously unseen real-world scan and the results are presented in Table 5.17 .

Table 5.17 Results of Scan

Metric	Value
Total alerts before ML	1489
Alerts retained after ML	1114
Alerts filtered as likely false positives	375
Alert reduction	25.18%

This result demonstrates that the model effectively reduces scanner noise while maintaining a substantial portion of potentially actionable findings.

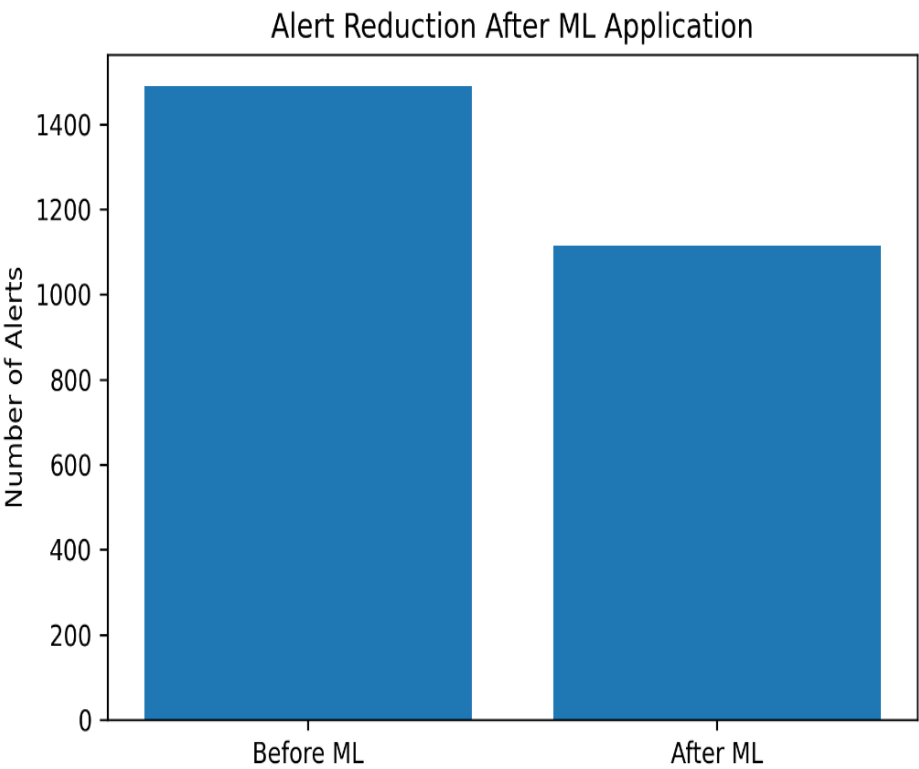


Figure 5.1 Test results of previous unseen real-world scan

Figure 5.1 shows the success of ML in lowering false positives and raising alert accuracy is demonstrated by this graphic, which contrasts the total number of security alerts produced by the vulnerability scanner before and after the machine learning model was integrated.

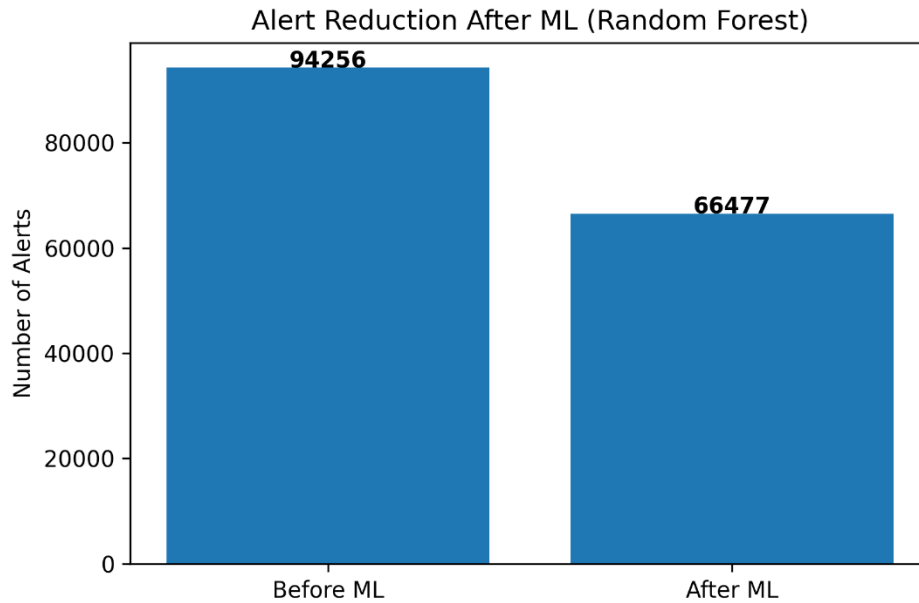


Figure 5.2 Test results random forestt model

Figure 5.2 illustrates the comparison of the number of security alerts generated before and after applying the Random Forest machine learning model, demonstrating a significant reduction in alerts and improved filtering of false positives.

5.2.2 Precision and F1-Score (Internal Validation)

Precision and F1-score were computed using pseudo-labels generated through weak supervision rules. The model achieved high precision values during internal evaluation. However, these results should be interpreted as consistency with heuristic rules rather than true predictive accuracy. The high precision indicates that the model successfully learns the same false positive patterns encoded by the domain rules.

5.2.3 Overfitting Analysis

An apparent concern arises when precision or F1-score approaches very high values. In a supervised learning, this could indicate overfitting. However, in this work, such behavior is described by the following factors:

- The pseudo-labels are derived from deterministic domain rules
- The feature space is intentionally low-dimensional
- The model learns dominant scanner noise patterns rather than complex decision boundaries

Threshold sensitivity analysis further showed stable behavior across multiple confidence thresholds, suggesting that the model is not memorizing individual samples but rather capturing generalizable patterns.

5.2.4 FP Confidence Distribution

The model produced false positive confidence scores that were strongly separated, with most of the findings receiving either high or low confidence values. A polarized distribution is indicative of stable learning behavior and suggests that the model has captured the dominant scanner noise patterns without relying on fragile thresholds.

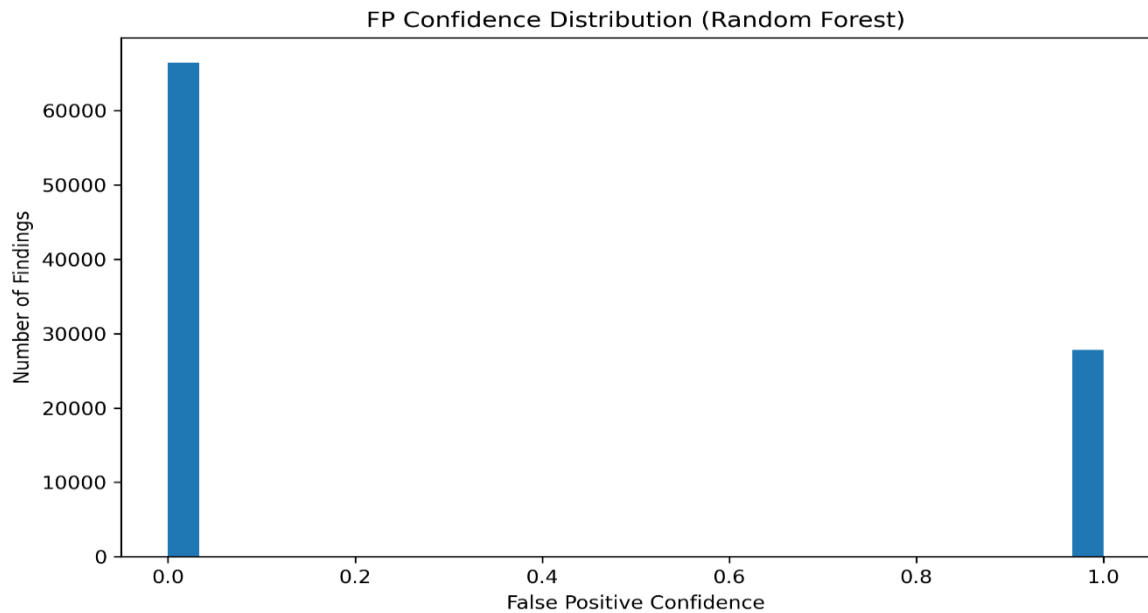


Figure 5.3 FP Confidence Distribution

Figure 5.3 shows the confidence score distribution of false positive findings generated by the Random Forest-based classifier. It can illustrate how the model marks the confidence level for the wrong alerts and support the analysis related to reducing false positives..

5.2.5 Risk Preservation

Analysis of the filtered results showed that, indeed, exploit-based vulnerabilities like injection and cross-site scripting issues were preserved for the most part. High-severity findings weren't disproportionately filtered, suggesting that this model prefers safety over aggressive noise reduction.

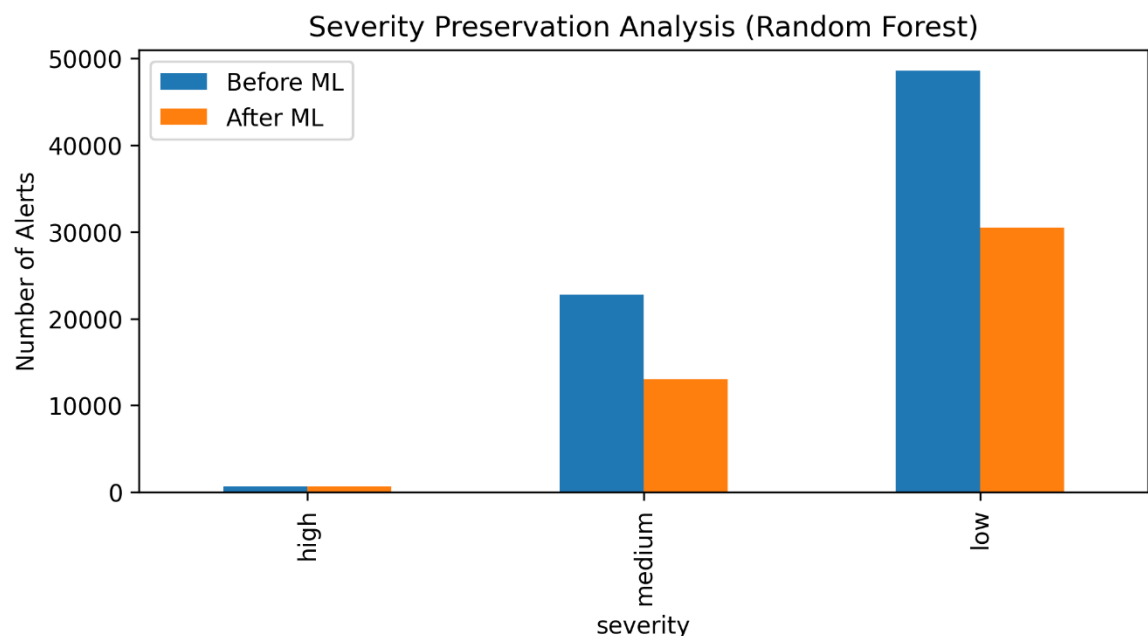


Figure 5.4 Severity Predervation Analysis

Figure 5.4 shows the distribution of security alerts with regard to their severity level-high, medium, and low-before and after the application of the Random Forest-based machine learning model. It appears from here that the reduction will affect mostly the lower-severity findings, thus preserving critical vulnerabilities..

5.3 Comparative Analysis with Existing Work

Traditional vulnerability scanners primarily rely on rule based detection mechanisms, which often result in uncontrolled false positives. Existing approaches to the false positive reduction typically fall into one of the following categories:

- Manual rule tuning and whitelisting
- Expert driven vulnerability validation
- Fully supervised learning models requiring labeled datasets
- Compared to these approaches, the proposed method offers several advantages:
- It does not require expert labeled ground truth
- It adapts to scanner behavior across environments
- It provides probabilistic confidence scores instead of binary decisions
- It integrates seamlessly as a post-processing step

While supervised models may achieve higher accuracy under ideal labeling conditions, such datasets are rarely available in real-world security environments.

5.4 Discussion on Findings

These experiments have shown that unsupervised learning and weakly supervised learning can be effective at reducing the number of false positives in the context of automatic vulnerability scanning. This method does not have the limitation associated with the traditional classification approach.

The utility value of a given approach is demonstrated by its ability to detect an alert decrease exceeding 25% upon unseen scan data, while a close scrutiny confirms that the necessary vulnerabilities are still maintained.

Precision and F1-score are not used as primary metrics for performance, although they are being used in the reporting for internal validation. The operational effect, robustness, and safety are used as metrics to assess the efficacy of the proposed method.

As far as overall performance is concerned, the experimental outcome supports that unsupervised learning provides a highly efficient, robust, and realistic solution for the problem of FPs in web vulnerability scanning applications.

Chapter 6. Conclusions

In this chapter, we discuss about summary of the project and recommendations for future work.

6.1 Summary

This project aims to develop a web based vulnerability scanning system with machine learning characteristics. It performs passive and active scanning automatically to discover prevalent web application vulnerabilities. The system processes scan results through trained ML models to identify possible threats and produces comprehensive reports with remediation recommendations. Important modules are user management, handling projects, scanning, detection using ML, and report generation.

6.2 Recommendations for Future Work

Though the proposed approach shows its efficacy for reducing false positives using weak supervision and machine learning, there are certain areas where future work can enhance this system.

To begin with, the kinds of vulnerabilities that can be considered might be extended to encompass more types of vulnerability patterns, rather than being limited by configuration and headers. Adding more patterns of vulnerability, including authentication, access, and application programming interfaces, would allow the neural network more opportunities to learn different patterns of the application scanners.

Secondly, the future work may involve the use of manual labeling by the domain expert for the subset of the vulnerability findings to improve the performance of the model. Although the project avoided labeling the findings due to the constraints, the manual labeling process by the security expert may help in the application of the supervised learning methods.

Future work would also be interested in exploring Hybrid methods of learning, whereby weak supervision is done in conjunction with human-labeled data. This will act to update and refine the model. Active learning methods will also be used to selectively label significant discoveries.

Chapter 7. References

- [1] “O. Z. P. OWASP Foundation”, [Online]. Available: <https://owasp.org/www-project-zap>
- [2] “B. S. PortSwigger Ltd”, [Online]. Available: <https://portswigger.net/burp>
- [3] “A. W. V. S. Invicti Security”, [Online]. Available: <https://www.acunetix.com>
- [4] “N. V. S. Tenable Inc”, [Online]. Available: <https://www.tenable.com/products/nessus>
- [5] “N. W. S. CIRT.net”, [Online]. Available: <https://cirt.net/Nikto2>
- [6] “OWASP Foundation, *OWASP Top Ten 2021*”, [Online]. Available: <https://owasp.org/www-project-top-ten/>
- [7] “S. S. W. A. Snyk Ltd”, [Online]. Available: <https://security.snyk.io/>
- [8] “R. G. Rapid7”, “Metasploit Framework”, [Online]. Available: <https://www.metasploit.com>
- [9] “G. McGraw”, “Software Security: Building Security In”, Addison-Wesley, 2006.
- [10] “I. Goodfellow, Y. Bengio, and A. Courville”, “Deep Learning”, MIT Press, 2016. Available: <https://www.deeplearningbook.org/>

SP25SE38_Automated Web Application Penetration Testing Using Machine Learning Models



Document Details

Submission ID

trn:oid::2945:338696042

106 Pages

Submission Date

Dec 22, 2025, 1:02 PM GMT+5

22,440 Words

Download Date

Dec 22, 2025, 1:08 PM GMT+5

135,031 Characters

File Name

SP25SE38_Automated Web Application Penetration Testing Using Machine Learning Models.docx

File Size

4.2 MB



Page 2 of 108 - AI Writing Overview

Submission ID trn:oid::2945:338696042

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.



5% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text

Match Groups

-  **110 Not Cited or Quoted** 5%
Matches with neither in-text citation nor quotation marks
-  **1 Missing Quotations** 0%
Matches that are still very similar to source material
-  **0 Missing Citation** 0%
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted** 0%
Matches with in-text citation present, but no quotation marks

Top Sources

- 4%  Internet sources
- 2%  Publications
- 0%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.